

JANOME DESKTOP ROBOT

JR3000 Series

JANOME CARTESIAN ROBOT

JC-3 Series

JANOME SCARA ROBOT

JS3 Series

取扱説明書

機能編Ⅱ

(変数・命令・関数)

このたびは本製品をお買い上げいただきまして、誠にありがとうございます。

- ご使用になる前に取扱説明書をよくお読みの上、正しくお使いください。
特に「安全上のご注意」は安全に関する重要な内容ですので、ご使用前に必ずお読みください。
- 本書はお読みになった後も大切に保管し、お使いになる方がいつでも見られるようにしてください。

JANOME

まえがき

本書では、JR3000、JC-3、JS3 の各シリーズについて説明しています。

取扱説明書は以下の各編により構成されています。

名称	内容	JR3000	JC-3	JS3
Read This First (はじめにお読み ください)	安全に関する重要な内容です。ご使用前に必ずお読み ください。(特殊仕様をお使いの場合は、特殊仕様編を ご参照ください。)	○	○	○
設置編	本機の設置方法について説明します。 ■ 設置する際には必ずお読みください ■ 注) ロボットに関する安全教育と、本機の設置に関す る教育を受けた方向けです。	○	○	○
保守編	本機のメンテナンスについて説明します。 ■ 保守を行う際には必ずお読みください ■ 注) ロボットに関する安全教育と、本機の保守に関す る教育を受けた方向けです。	○	○	○
基礎編	各部の名称やデータ構成など、本機を操作するために 必要な基礎的知識を説明します。 ■ 設置・保守を行う際には必ずお読みください ■	○ (共通)		○
入門編	簡単なプログラムを実際に作成し運転することで、本 機の操作方法を説明します。	○ (共通)		○
ティーチングペン ダント操作編	ティーチングペンダントからの本機の操作方法を説明 します。	○ (共通)		○
機能編Ⅰ	ポイントティーチングについて説明します。	○ (共通)		
機能編Ⅱ	命令・変数・関数について説明します。	○ (共通)		
機能編Ⅲ	全プログラム共通設定や PLC プログラム等、運転時の 機能を説明します。	○ (共通)		
機能編Ⅳ	カスタマイズ機能の説明をします。	○ (共通)		
外部制御編	I/O およびフィールドバスについて説明します。	○	○	○
通信制御編 (COM, LAN)	COM, LAN の通信制御について説明します。	○ (共通)		
カメラ・センサ 機能編	カメラ、Z センサを取り付けた場合の機能を説明します。	○ (共通)		
資料編	本機の動作範囲や質量など、仕様全般が掲載されてい ます。	○	○	—
補助軸機能編	補助軸機能について説明します。	○ (共通)		
用途仕様編	各用途仕様専用の機能を説明します。	標準仕様： - 用途仕様： ○		

名称	内容	JR3000	JC-3	JS3
特殊仕様編	各特殊仕様専用の設置およびメンテナンス方法について説明します。 ■ 設置・保守を行う際には必ずお読みください ■ 注) ロボットに関する安全教育と、本機の設置・保守に関する教育を受けた方向けです。	○	○	—

⚠ 警告



リストにある取扱説明書に記載された以外の方法で、本機を操作、運転しないでください。修理の際は、弊社（連絡先は本書の裏表紙を参照）または代理店までご連絡ください。
感電、ケガの原因になります。

⚠ 注意



本機の機能・性能を十分に発揮できるように、取扱説明書に従った正しい取り扱い方法、および操作方法によりご使用ください。



本機は、設定やデータの変更を行った後そのまま電源を OFF すると、変更部分が消失し、変更前の状態に戻ってしまいます。
設定やデータの変更を行った際は、忘れずにデータを保存してください。



初めて本機をご使用になる際は、必ず事前にロボットデータのバックアップを実行して、ロボットの個体情報を保存してください。ロボットの個体情報は、ロボットの内部基板を交換した場合に必要となります。
バックアップ方法については、JR3000 シリーズでは取扱説明書『設置編』「3. ロボットデータバックアップ・復元」、JC-3 シリーズでは取扱説明書『設置編』「7.1 ロボットデータバックアップ・復元」、JS3 シリーズでは取扱説明書『設置編』「9.1 ロボットデータバックアップ・復元」をご参照ください。

- 本書に記載されている内容は標準仕様に準じています。ご使用の機種によってはメニュー項目等が異なる場合があります。
- 2 軸仕様では Z 軸関連の設定メニューが表示されることがありますが、設定は無効です。
- オプションについては、JR3000 シリーズでは取扱説明書『資料編』「24. 仕様」、JC-3 シリーズでは取扱説明書『資料編』「13. 仕様」、JS3 シリーズでは取扱説明書『基礎編』「15. 仕様」にてご確認ください。図示を除き、本文中でのオプション表記については省略しています。
- 製品改良のため、断り無く仕様を変更することがありますので、ご了承ください。

凡例：

- 本取扱説明書では、操作方法の手順を以下のように表記しています。

TP ティーチングペンダントでの操作方法

PC PC（JR C-Points II）での操作方法

- 青色 + 下線で表記されている文字列をクリックすると、参照先に移動できます。

例：「[1. ポイント作業データの作成手順](#)」をご参照ください。

目次

まえがき	1
安全上のご注意	7
1. ポイント作業データの作成手順	44
1.1 ロボット本体の TP を使用する	44
2. 式の構成	47
2.1 式	47
2.2 定数	47
2.3 変数	47
2.4 関数	48
2.5 演算子	48
3. 命令一覧	49
3.1 ポイント作業	49
3.2 実行条件	55
3.3 PLC プログラム	55
4. 変数一覧	56
5. 関数一覧	65
6. システムフラグ一覧	70
6.1 JR3000/JC-3 シリーズ	70
6.2 JS3 シリーズ	73
7. 変数	75
7.1 組み込み変数	75
7.1.1 自由変数	75
7.1.2 入力変数	76
7.1.3 出力変数	77
7.1.4 ダウンタイマー	78
7.1.5 移動後作業開始高さ	78
7.1.6 パレット	80
7.1.7 ワーク補正	81
7.1.8 ポイント座標	83
7.1.9 指定ポイント座標	84
7.1.10 指定プログラム指定ポイント座標	84
7.1.11 指定プログラムツールデータ	85
7.1.12 ポイント属性データ番号	85
7.2 ユーザ定義変数	86
7.2.1 グローバル変数・キープ変数	86

8. 関数.....	87
8.1 組み込み関数.....	87
8.1.1 ロボット系関数.....	87
8.1.2 算術関数.....	93
8.1.3 文字列関数.....	97
8.2 ユーザ定義関数.....	99
9. エイリアス定義.....	101
9.1 I/O エイリアス.....	101
9.2 COM エイリアス.....	102
10. ON/OFF 出力制御.....	104
10.1 I/O へ出力.....	104
10.2 X 秒後に出力.....	106
10.3 ブザーを鳴らす.....	107
10.4 緑 LED を点滅する.....	108
10.5 赤 LED を点滅する.....	108
10.6 I/O から数値を出力.....	109
10.7 動力 ON、サーボ ON/OFF.....	110
11. 条件分岐・条件待ち.....	111
11.1 条件分岐.....	111
11.2 条件待ち.....	113
12. 条件演算.....	115
12.1 条件設定.....	115
13. ディレイ、データ IN、スタート待ち.....	118
13.1 時間遅延.....	118
13.2 スタート指示を待つ.....	119
13.3 I/O から入力.....	121
14. パレット操作.....	123
14.1 パレット命令.....	123
15. 実行流れ制御.....	126
15.1 種別設定作業をサブルーチンコール.....	126
15.2 ポイント作業データをサブルーチンコール.....	128
15.3 ポイント作業終了.....	130
15.4 プログラムをサブルーチンコール.....	131
15.5 ポイント列呼び出し.....	135
15.6 プログラムの強制終了.....	136
15.7 関数の戻り値を代入.....	137
15.8 指定ポイントへ飛ぶ.....	137
15.9 指定した命令へ飛ぶ.....	140
16. 繰り返し.....	141

17. 駆動制御.....	143
17.1 Z (J3) 軸だけを上昇／下降.....	143
17.2 ポイント作業で CP 直線駆動.....	145
17.3 ポイント作業でメカ初期化実行.....	147
17.4 位置ずれ検出.....	148
17.5 指定した 1 軸だけを駆動.....	149
18. LCD、7SLED.....	155
18.1 ティーチングペンダントに指定文字列を表示.....	155
18.2 7 セグメント LED に任意の数値を表示.....	156
19. COM/Ethernet 入出力.....	157
20. 変数、コメント、システム制御.....	162
20.1 変数宣言、代入.....	162
20.2 コメント挿入.....	164
20.3 ポイント作業でプログラム番号変更.....	164
20.4 ポイント作業で PLC プログラム変更.....	165
21. Appendix.....	166
21.1 イーサネットクライアント機能.....	166
21.1.1 イーサネットクライアントポートの設定.....	167
21.1.2 接続処理.....	168
21.1.3 実用例.....	170

安全上のご注意

ここに示した注意事項は、製品を安全に正しくお使いいただき、お使いになる人や他の人々への危害や財産への損害を未然に防止するためのものです。

．．．．．必ずお守りください．．．．．

注意事項は、いろいろな表示を用いて説明しています。表示の意味は下記をご参照ください。

■ 危害・損害の程度を表わす表示

注意事項を無視して、誤った取り扱いを行ったときに生じる危害や損害の程度を、次の表示で区分しています。

 危険	この表示の欄は「死亡または重傷などを負う切迫した可能性が想定される」内容です。
 警告	この表示の欄は「死亡または重傷などを負う可能性が想定される」内容です。
 注意	この表示の欄は「傷害を負う可能性または物的損害が発生する可能性が想定される」内容です。

■ 危険の内容や回避方法を表わす表示

お守りいただく注意事項の内容の種類を、次の図記号で区分しています。

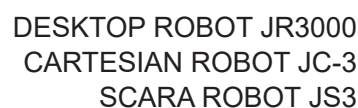
	記号は、気を付けていただきたい「注意」の内容です。
	注意してください。（一般的な注意）
	記号は、行ってはいけない「禁止」の内容です。
	絶対に行わないでください。（一般的な禁止）
	分解（改造・修理）しないでください。
	手を触れないでください。（接触禁止）
	記号は、必ず実行していただく「強制」の内容です。
	必ず指示にしたがい、実行してください。（一般的な強制）
	必ず電源ケーブルを外してください。
	必ずアースの接続を確認してください。

[illegible]

【補助軸機能を使用した時に安全策が必要となった場合】



〈例〉



安全上のご注意

■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ 〈JR3000 シリーズ〉 **■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■**

【補助軸機能を使用した時に安全策が必要となった場合】

 危險



ロボットの電源が ON しているときは、安全防護柵の内側に入ったり、手や頭部等を入れたり等、絶対にしないでください。

ケガの原因になります。



ロボットまたは周辺機器に異常が発生した場合や、点検や給油等をする際など、安全防護柵内に立ち入る場合は、本体の電源スイッチと供給元の電源ブレーカ、どちらも OFF の位置でロックアウト、タグアウトを行い、通電されていない状態で行ってください。

感電、ケガの原因になります。

 警告



補助軸機能を使用してシステムを構築した時に産業用ロボットに該当する場合、産業用ロボットのティーチング、点検、調整、修理等に従事する作業者は、「労働安全衛生法第 59 条および関連省令等」に定める産業用ロボットの「特別教育」の受講が義務付けられています。必ずこの「特別教育」を受講してください。

弊社ではお客様に対し、これらの「特別教育」は行っておりません。中央労働災害防止協会等へご相談ください。

日本国外でご使用になる場合は、ご使用になる国の法令およびガイドライン等に従ってください。



- 運転や作業を行う前に、必ず以下のチェックを行ってください。
- ・ 障害物のチェック : 安全防護柵内に障害物がない、または人がいないこと。
 - ・ 設置上のチェック : ロボットが正しく設置されていること。ロボットや周辺機器に異常がないこと。また、ティーチングペンダントや工具等が所定の位置にあること。
 - ・ 非常停止の機能チェック : I/O-S 回路（インターロック）、非常停止スイッチが正しく機能していること。
- これらのチェックをせずに作業を行うと、危険を引き起こす可能性があります。

[illegible]

	警告
	安全防護柵は十分な強度を有し、ツール・ワークの破片等が作業者に危険を及ぼさないように構築してください。 安全防護柵内での作業には、ヘルメット・保護グローブ・保護ゴーグル・安全靴等の保護具を使用してください。ケガの原因になります。
	ロボットの把持した物が飛来、落下するおそれがある場合には、その物の大きさ、質量、温度、化学的性質を考慮して適切な安全防護措置を取ってください。 ケガ、故障の原因になります。
	安全防護柵の内側で作業を行う場合は、ロボットの最大領域に入らないようにしてください。 ケガの原因になります。
	運転をスタートさせる時は、安全防護柵内に人がいないこと、また運転の妨げになるような障害物がないことを確認してください。 ケガ、故障の原因になります。

安全上のご注意

[illegible]

 危險



引火性、腐食性ガスのある場所で使用しないでください。
 万一ガスが漏れて本体の周囲に溜まると、爆発、火災の原因になります。



本機設置の際はお客様にてリスクアセスメントを実施し、防護措置が必要と判断された場合は、エリアセンサ、囲い等を設置してください。

防護措置のない場合、ロボットの動作中に作業範囲に人が入るとケガの原因になります。



運転や作業を行う際は、非常停止スイッチをすぐに押せる状態にしてください。
非常停止スイッチの周囲に操作の妨げになるものを置くと、非常時に直ちに安全に
ロボットを停止させることができなくなり、危険を引き起こす可能性があります。



取扱説明書『保守編』を参照して、非常停止スイッチの機能チェックを定期的に行ってください。

また I/O-S * のある機種の場合は、I/O-S の機能チェックも定期的に行ってください。このチェックをせずに作業を行うと、非常時に直ちに安全にロボットを停止させることができなくなり、危険を引き起こす可能性があります。

* I/O-S による停止は停止カテゴリ 2 です。

安全上のご注意

[illegible]

設置・保守作業を行う際は、作業者の安全のため、ヘルメット・保護グローブ・保護ゴーグル・安全靴等の保護具をご使用ください。

ケガの原因になります。



質量および使用状態に耐える場所に、確実に設置してください。ロボットは床面からの高さ 600 mm 以上の作業台の中央に設置してください。

また、冷却ファンのある背面側を、壁面から 300 mm 程度離してください。

スイッチボックスは床上 600 mm 以上の操作しやすい高さに設置してください。

設置に不備があると本体の落下、転倒などにより、ケガや故障の原因、または異常温度上昇や火災の原因になります。



電源は、必ず定格銘板の表示内でご使用ください。
感電、火災、故障の原因になります。



専用の電源コードをご使用ください。電源コードは改造しないでください。
感電、故障の原因になります。
改造に起因する不具合に関し、弊社は一切責任を負いませんのでご了承ください。



電源プラグは、確実にコンセントへ差し込んでください。
差し込みが浅いと、電源プラグが加熱し、火災の原因になります。



定格容量を守ってご使用ください。
故障、火災、感電の原因になります。



ヒューズ交換や点検や給油をするときは、必ず電源プラグをコンセントから抜き、本機から電源コードを取り外して、通電されていない状態で行ってください。なお、電源コードを取り外してから5秒以内のインレットの刃には触れないでください。感電、ケガの原因になります。

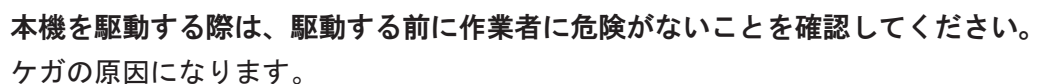
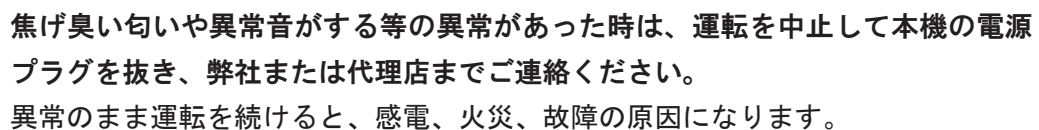


必ず電源コードを通じてアースを接続してください。アースを接続しない状態では使用しないでください。

アースが不完全な場合は、感電、火災、誤動作、故障等の原因になります。



電源プラグを定期的に乾いた布で拭き、ほこり等を取り除いてください。
ほこり等が付着していると絶縁不良となり、火災の原因になります。

[illegible]

安全上のご注意

■■■■■■■■■■ 〈JR3000 シリーズ〉 **■■■■■■■■■■**



下記の条件を満たす環境でご使用ください。

- ・ 周囲温度 0 ～ 40 ℃
- ・ 湿度 20 ～ 90 %、結露がないこと
- ・ 標高 1000 m 以下

誤動作、故障の原因になります。



本機を使用場所に設置しご使用になる前に、アースなどの電気ノイズ対策を行ってください。

本機や周辺機器の誤動作、故障の原因になります。



ツールやカメラ、その他の機器等を取り付けた時は、本機を駆動する前に正しく固定されていることを確認してください。

ケガ、故障の原因になります。



本体の固定ねじ等がゆるんでいないかどうか、定期点検（3 か月毎、または使用時間 750 時間毎）にて点検してください。

ケガ、故障の原因になります。



本体へのコード、ケーブル類の接続は確実に行ってください。

誤動作、故障の原因になります。



ロボットを移動する際は、以下の点に注意してください。

- ・ 移動前に、本機の可動部（Z 機構または ZR 機構）を固定する
- ・ JR3200 シリーズの場合、2 人以上で持つ
- ・ JR3300 ～ JR3600 シリーズの場合、2 人以上で持つか、リフター等を使用する
- ・ ベース左右の底面を持ち、水平に保つ
- ・ 柱や Y 本体、Z 機構または ZR 機構を持たない

ケガ、故障の原因になります。



直射日光の当たらない屋内でご使用ください。

誤動作、故障の原因になります。

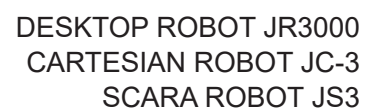


個体情報は同機種のリボットでも個体毎に異なります。バックアップデータを別のリボットへ使用しないでください。正常に動作できなくなります。

[illegible]

[illegible]

〈例〉



[illegible]

停止カテゴリ 0 で安全回路を構築せずに非常停止ボタンを押すと、エラーになります。



ケガの原因になります。



予期せぬ動作や感電、ケガの原因になります。



これらのチェックをせずに作業を行うと、危険を引き起こす可能性があります。

安全上のご注意

[illegible]

 危險



運転や作業を行う際は、非常停止スイッチをすぐに押せる状態にしてください。
非常停止スイッチの周囲に操作の妨げになるものを置くと、非常時に直ちに安全に
ロボットを停止させることができなくなり、危険を引き起こす可能性があります。



非常停止スイッチの機能チェックを定期的に行ってください。
また、EMG OUT 回路の機能チェックも定期的に行ってください。
このチェックをせずに作業を行うと、非常時に直ちに安全にロボットを停止させることができなくなり、危険を引き起こす可能性があります。

 警告



設置・保守作業を行う際は、作業者の安全のため、ヘルメット・保護グローブ・保護ゴーグル・安全靴等の保護具をご使用ください。

ケガの原因になります。



電源は、必ず定格銘板の表示内でご使用ください。
感電、火災、故障の原因になります。



専用の電源コードをご使用ください。電源コードは改造しないでください。
感電、故障の原因になります。
改造に起因する不具合に関し、弊社は一切責任を負いませんのでご了承ください。



電源プラグは、確実にコンセントへ差し込んでください。
差し込みが浅いと、電源プラグが加熱し、火災の原因になります。



電源端子台（DC 48 V 入力）への電源コードには、圧着端子を用いて接続してください。端子台のねじは確実に締め付けてください。

感電、火災、故障の原因になります。



電源を ON にした状態での作業は安全防護柵の外で実施してください。
ケガの原因になります。

安全上のご注意

[illegible]

定格容量を守ってご使用ください。
故障、火災、感電の原因になります。



コントローラは必ず制御盤内に設置し、扉を開けると電源が遮断されるようにしてください。冷却ファンのあるコントローラの上面には 300 mm 程度、通風口のある側面には 100 mm 以上の空間を空けてください。

設置に不備があると、異常温度上昇や火災、感電、ケガの原因になります。



3 軸仕様にてブレーキを解除する場合は、取り付けてあるツール等を取り外すか、落下防止策を行ってください。

電源投入時にティーチングモード、手動運転モード、外部運転モードでブレーキを解除する場合、Z 軸の取付質量によっては軸が落下する場合があります。ケガ、故障の原因になります。



焦げ臭い匂いや異常音がする等の異常があった時は、運転を中止し、通電されていない状態にした上で、弊社または代理店までご連絡ください。

- 必ずコントローラの電源コード(AC)をコンセントから抜いてください。なお、電源コードを取り外してから 5 秒以内のインレットの刃には触れないでください。
- 電源端子台(DC 48 V 入力)への通電を止め、端子台から電源コードを取り外してください。

異常のまま運転を続けると、感電、火災、故障の原因になります。



コントローラの点検や保守を行う際は、通電されていない状態で行ってください。

- 必ずコントローラの電源コード(AC)をコンセントから抜いてください。なお、電源コードを取り外してから5秒以内のインレットの刃には触れないでください。
- 電源端子台(DC 48 V 入力)への通電を止め、端子台から電源コードを取り外してください。

感電、ケガ、データ損失や故障・誤動作の原因になります。

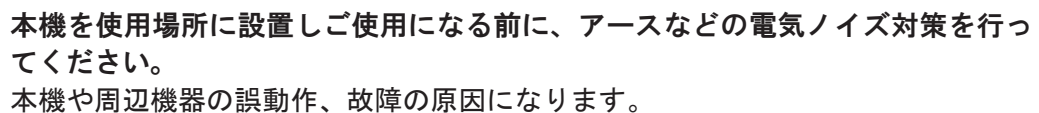


必ず電源コードを通じてアースを接続してください。アースを接続しない状態では使用しないでください。

アースが不完全な場合は、感電、火災、誤動作、故障等の原因になります。



電源プラグを定期的に乾いた布で拭き、ほこり等を取り除いてください。
ほこり等が付着していると絶縁不良となり、火災の原因になります。

[illegible]

安全上のご注意

[illegible]

 注意



ツールやカメラ、その他の機器等を取り付けた時は、本機を駆動する前に正しく固定されていることを確認してください。

ケガ、故障の原因になります。



本体の固定ねじ等がゆるんでいないかどうか、定期点検（3 か月毎、または使用時間 750 時間毎）にて点検してください。

ケガ、故障の原因になります。



本体、コントローラへのコード、ケーブル類の接続は確実に行ってください。
誤動作、故障の原因になります。



外部電源 DC 48 V を遮断後、再び電源を供給するまでの間隔を 1 秒以上あげてください。

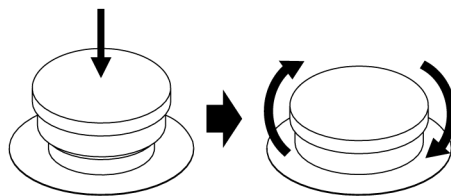
間隔をあけずに電源を再供給すると、故障の原因になります。

- 外部電源 DC 48 V の遮断後、1 秒以上待ってから電源を再投入してください。
- 非常停止スイッチ押下後、1 秒以上待ってから非常停止を解除してください。
- 外部非常停止装置の作動後、1 秒以上待ってから非常停止を解除してください。



非常停止スイッチは、下図のとおり正しく操作してください。
正しく操作しないと故障の原因になります。

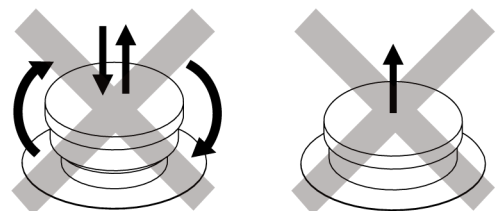
正



真っすぐに押す

回して解除する

誤



解除方向に回した状態で
押し戻ししない

引っ張らない

安全上のご注意

[illegible]

外部電源 DC 48 V の瞬断が発生しないよう、コントローラ側面にある定格銘板に記載された定格電流に対して、余裕を持った電源回路を構築してください。
故障の原因になります。

例：JC-3C-3 の定格電流

INPUT : AC 100-240V~ 50/60Hz 1.0-0.4A
DC 48V --- 6.25 A ← 定格電流



EMG-OUT コネクタ（組）を用いた外部非常停止装置を作動させた際、非常停止の要因を取り除いた後に、非常停止の解除操作を行ってください。

ケガや故障の原因になります。



本機の運転を停止させる際は、次のことにお守りください。
故障の原因になります。

- 非常時以外の運転停止には、スイッチボックスのスタート／ストップスイッチ、または I/O-SYS の運転停止入力をご使用ください。
- 外部非常停止装置は非常時のみ使用し、運転時の停止用として使用しないでください。
- 非常時以外に運転を停止させるのに、外部電源 DC 48 V を遮断しないでください。



移動の際は、移動する前に本機の可動部を固定してください。
ケガ、故障の原因になります。



移動の際は、2人以上で持ち上げてください。
ケガ、故障の原因になります。



直射日光の当たらない屋内でご使用ください。
誤動作、故障の原因になります。



個体情報は同機種 of ロボットでも個体毎に異なります。バックアップデータを別のロボットへ使用しないでください。正常に動作できなくなります。

[illegible]

[illegible]

本体



ロボットの最大領域に入るとケガの原因になります。
必ず以下の点を守った安全防護柵を設置してください。

- 安全防護柵は、簡単に移動できないように設置してください。
- 安全防護柵は、簡単に破損したり倒れたりしないように設置してください。
- 安全防護柵は、万一ロボットが転倒した場合にもぶつからないような作業空間を考慮して設置してください。
- 安全防護柵は、手や頭などが柵内に入らないように設置してください。
- 安全防護柵の出入り口には、ロボットの動作中に開けると非常停止が働くドア等（インターロック装置）を設置し、この出入り口以外からは出入りできないようにしてください。また、インターロック装置は付属の I/O-S コネクタを使用して、コントローラと接続してください。
- 安全防護柵の出入り口の良く見える場所に「立入禁止」「運転禁止」等の警告表示を行ってください。
- 添付の警告表示シール（下図参照）を良く見える場所に貼ってください。

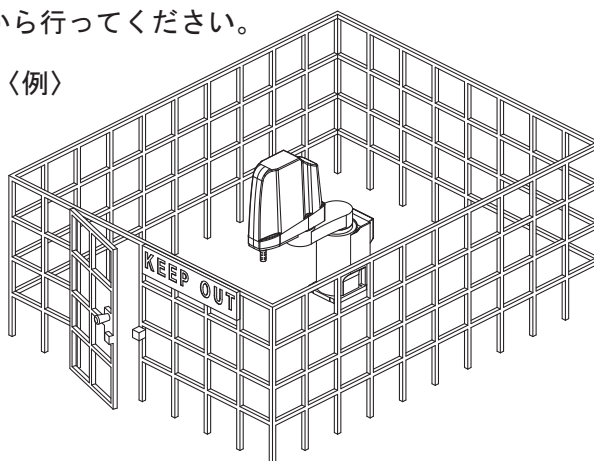
注)

- I/O-S コネクタによる停止は、停止カテゴリ 1 となりますので、使用する際は別途、お客様にてリスクアセスメントを実施願います。
- I/O-S 接続についての詳細は取扱説明書『設置編』をご参照ください。
- 設置後の動作確認は、安全防護柵の外側から行ってください。

〈警告表示シール〉



〈例〉



[illegible]

安全上のご注意

[illegible]

 警告



次のような環境での使用はお避けください。もし、このような環境下でご使用になる場合には、直接ロボットがこのような環境にさらされないようお客様にて十分な予防措置を施してからご使用ください。

- 輸送時 3.5 G 以上の振動や衝撃が加えられるようなところ。
- 動作時 0.5 G 以上の振動や衝撃が加えられるようなところ。

誤動作、故障等の原因になります。



通常の人に対する放射線の許容値を大きく超えるようなところでの使用はお避け
ください。

もし、このような環境下でご使用になる場合には、直接ロボットがこのような環境にさらされないようお客様にて十分な予防措置を施してからご使用ください。

誤動作、故障等の原因になります。

 注意



事故防止のため、ロボットの左右側から持ったり、カバーを持ったりしないでください。

ロボットのカバーを持つとロボットの転倒、カバーの破損や落下など、事故の原因になります。



運搬時には、J2 アームカバーを持たないでください。

カバーを持つと、破損の原因になります。



シャフト部（J3 軸）に力を加えないよう注意してください。

シャフトが損傷し動作時に過負荷エラーが発生する場合があります。



ロボットを横向きにしないでください。

グリスの漏れ出しや故障の原因になります。



設置用架台は鋼製とし、本体質量およびロボット運転時に発生する力に十分耐え得る設計にしてください。

本体の落下、転倒などにより、ケガや故障の原因になります。

安全上のご注意

<JS3 シリーズ>



ロボット本体は、指定された水平な据付け面に固定用ボルトにて確実に固定し、位置ずれを生じないようにしてください。



ロボット据付時、背面にモータ電源接続ケーブルとエンコーダ接続ケーブルの接続、前面にバッテリーの交換に必要なメンテナンススペースを確保してください。また、ロボット本体は、直射日光あるいは照明の熱があたる場所に設置しないでください。ロボット本体の表面温度が上昇し、エラーが発生する場合があります。



本体は定期的に給油を行う必要がありますので、給油が行える状態で設置してください。（取扱説明書『保守編』「4.8 給油」参照）



作業者はエア配管が正しく接続されているか、外れていないか、または外れやすいかを確認し、問題があった場合には正しく接続してください。

ケガ、故障の原因になります。



ロボットに取り付けられたアーム固定板は、必ずロボット据付後に取り外すようにしてください。
故障の原因になります。



本体据付後はトルクレンチにて締め付けトルクを確認するなど、据付ねじにゆるみのないことを、定期点検時（3 か月毎、または使用時間 750 時間毎）に点検し、記録してください。



お客様にて製作したハンドを取り付ける際には、以下の点にご注意ください。

- ハンドとワークの総質量が定格の可搬質量を超えないように注意してください。特に偏芯したハンドを取り付ける場合は、手首部の許容慣性モーメントも満足することを確認してください。
- ハンドなどのツールはワークに対する十分な把持力を備えたものを使ってください。
- ハンドは、ロボットのフランジ部の取付寸法にあわせて指定したサイズのボルトで確実に固定してください。また、機能上必要な部分を除き突起部や鋭利な角がないようにし、必要に応じてカバー等の保護を設けてください。

ケガ、故障の原因になります。

[illegible]

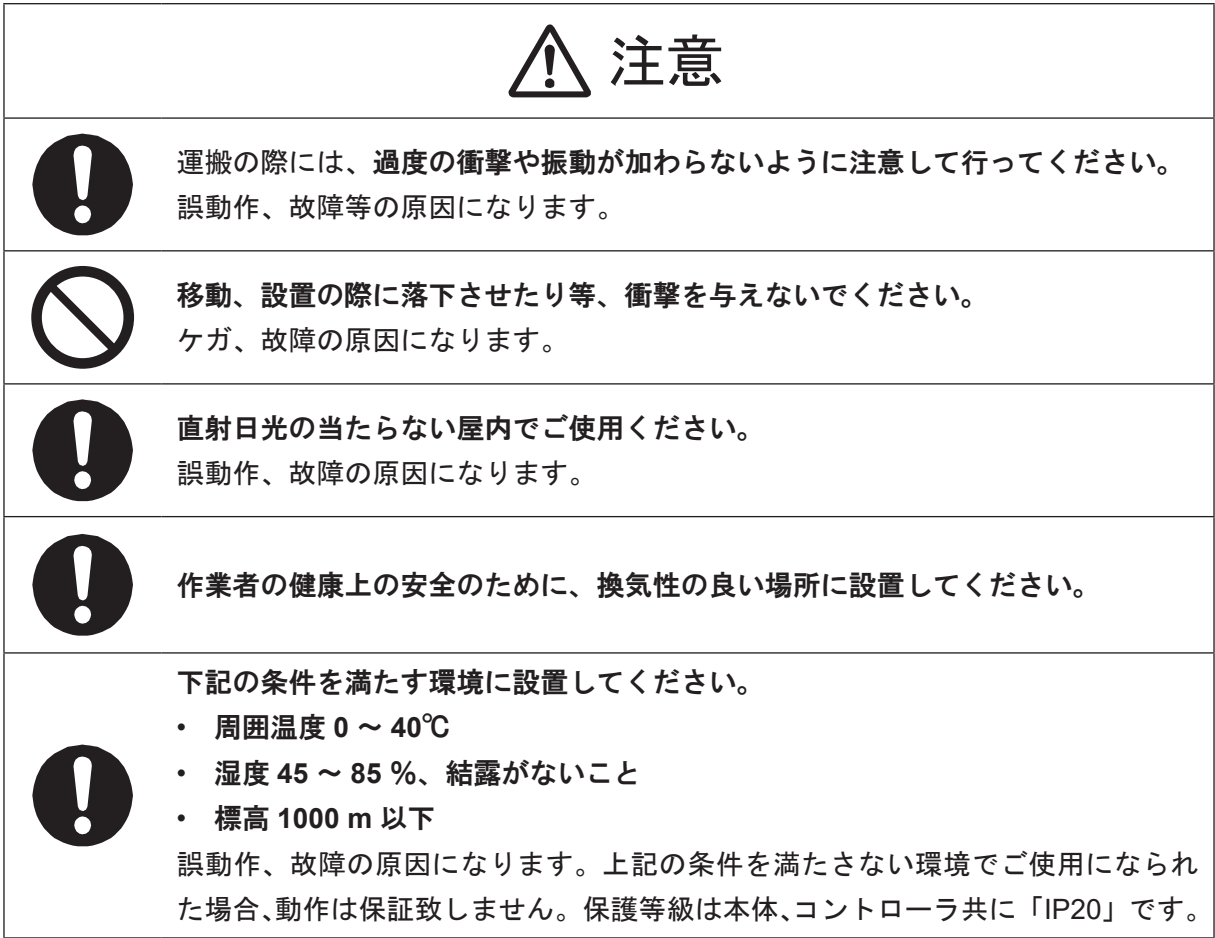
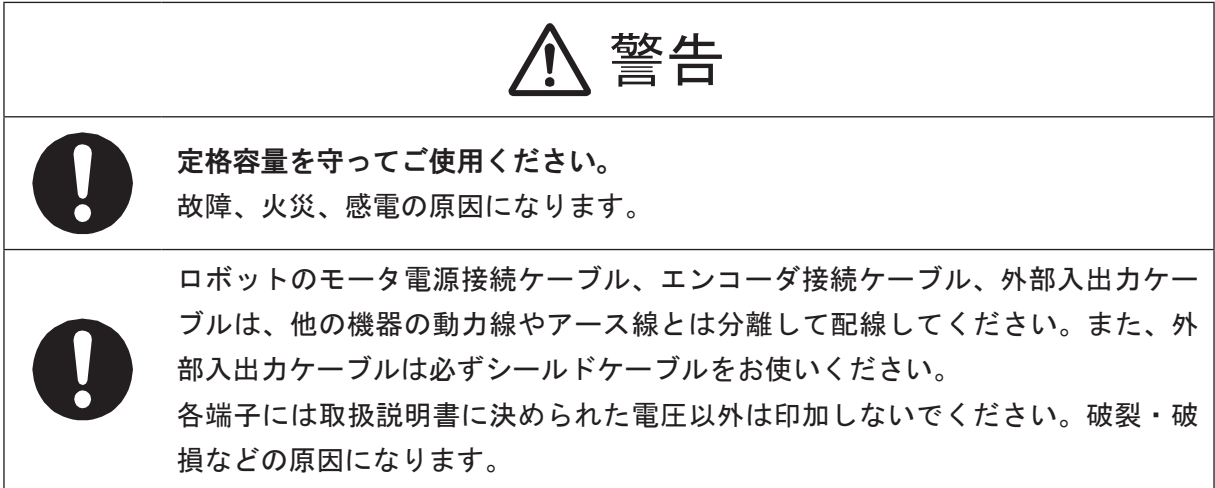
 危險



 **警告**



- DESKTOP ROBOT JR3000
CARTESIAN ROBOT JC-3
SCARA ROBOT JS3

[illegible]

安全上のご注意

[illegible]

 注意



本機を使用場所に設置しご使用になる前に、アースなどの電気ノイズ対策を行ってください。

本機や周辺機器の誤動作、故障の原因になります。



粉塵やオイルミスト、水気のある場所で使用しないでください。
誤動作、故障の原因になります。保護等級は本体、コントローラ共に「IP20」のため、
粉塵やオイルミスト、散水等に関しては無保護です。



ケーブルやコネクタには使用中に無理な力が加わったり、引っ張ったり、踏みつけたり、過度に曲げたりしないでください。必要により、配管やカバーによりケーブルを保護してご使用ください。

ケーブルコネクタの固定ねじや固定機構を確実に締めてください。固定が不十分の場合、運転時に外れるおそれがあります。



通電するとハンド入力ケーブル（オプション）には電圧が供給され、末端がフリーになっている場合、短絡事故につながるため、必ず末端処理をしてから通電してください。

[illegible]

危険

コントローラは安全防護領域の外側に据え付けてください。また、コントローラの操作面は操作する人がロボットに背を向けることなくロボットを監視でき、また容易に操作できる高さに配置してください。

保守作業性から操作面の高さは 600 mm 以上としてください。

フィールドバスを接続する前に動作時の安全を確保してください。

フィールドバスにスタート等の信号が割り付けられている場合、信号を送っている状態が保持されている等の理由で、接続完了直後にロボットが動作するおそれがあります。

ケガ、故障の原因になります。

警告

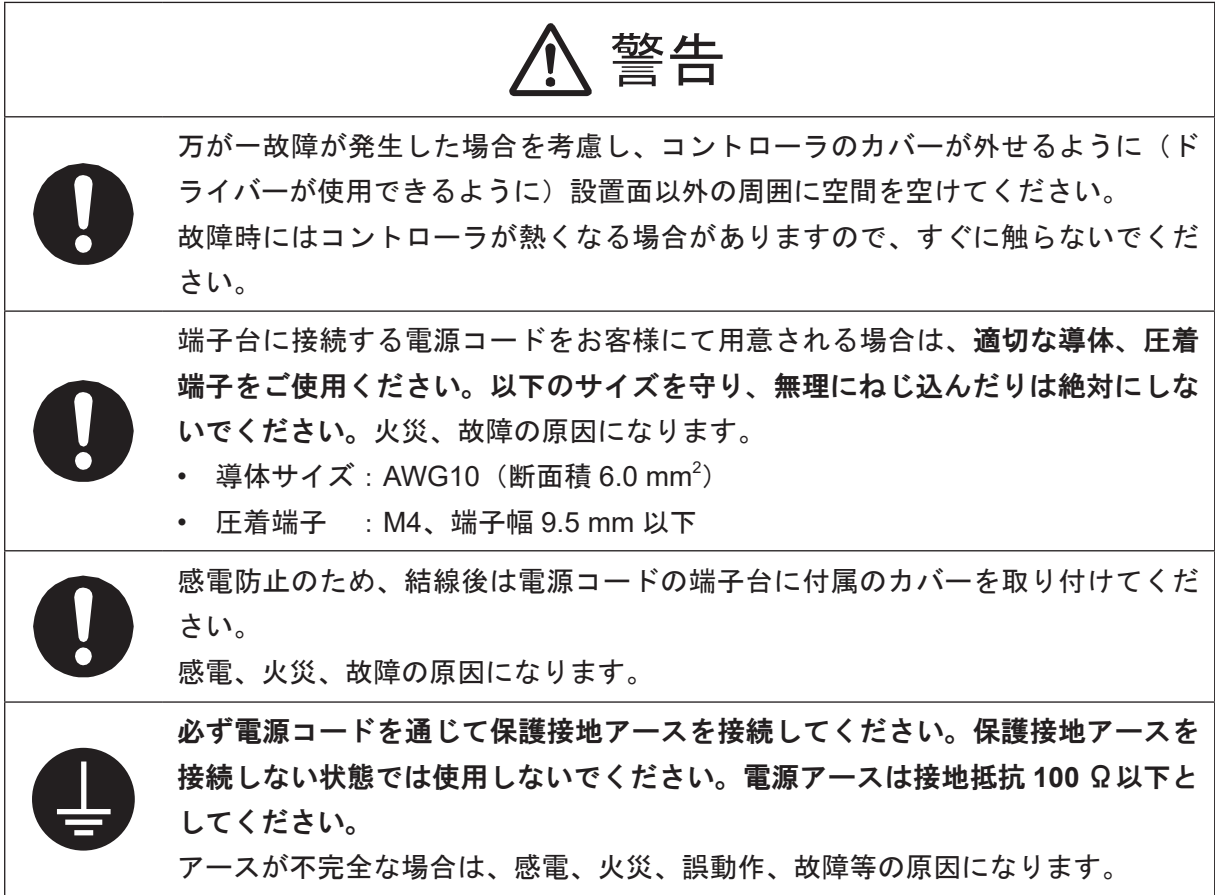
コントローラの移動の際は、リフター等をご使用ください。
ケガ、故障の原因になります。

外部からの振動による落下や移動等を防止するためにコントローラを固定する必要がある場合には、必ず底面部にて行ってください。（ゴム足は取り外さないこと）

質量および使用状態に耐える場所に、確実に設置してください。
設置に不備があると本体の落下、転倒などにより、ケガや故障の原因、または異常温度上昇や火災の原因になります。

コントローラには冷却用ファンや通風口があります。これらを塞がないように、右側面と左側面に 20 mm 以上、背面に 40 mm 以上の隙間を空けてください。
異常温度上昇や故障の原因になります。

コネクタに負荷がかからないよう、また作業等ができるようにコントローラの前面には 200 mm 以上（目安）の空間を空けてください。
誤動作、故障の原因になります。

[illegible]

安全上のご注意

〈JS3 シリーズ〉



スイッチ、端子台、コネクタ等の突起物には力が加わらないように注意して運搬してください。

故障の原因になります。



コントローラ、操作ボックス（オプション）、およびティーチングペンダントを運転中にモニターとして使用する場合は、**非常時に直ちに非常停止スイッチを押せるように、床上 600 mm 以上の操作しやすい高さに設置してください。**

非常停止スイッチに手が届かない状態で作業や運転を行うと、非常時に直ちに安全にロボットを停止させることができず、危険を引き起こす可能性があります。また、低すぎると足が接触する（当たる）等、誤操作の原因にもなります。



メンテナンス時に、メモリーポートへのアクセスができる状態で設置してください。

[illegible]

警告	
	ロボットの把持した物が飛来、落下するおそれがある場合には、その物の大きさ、質量、温度、化学的性質を考慮して適切な安全防護措置を取ってください。 ケガ、故障の原因になります。
	ロボットのアームを外部から手で動かす場合は開口部に手や指を入れないでください。 姿勢によっては手や指をはさまれる場合があります。

注意	
	本機を駆動する際は、駆動する前に作業者に危険がないことを確認してください。 ケガの原因になります。
	ツールやカメラ、その他の機器等を取り付けた時は、本機を駆動する前に正しく 固定されていることを確認してください。 ケガ、故障の原因になります。
	J3 (Z) /J4 (R) 軸に重みが加わっている場合は、本機の電源を OFF すると荷重に より J3/J4 軸が下降する可能性があります。この場合は、J3/J4 軸が下降しないよ う荷重を取り外すか、安全ブロック等を設置してください。
	安全防護柵内で作業を行う場合はリスクアセスメントを実施して、以下の項目を 守った「作業規則」を定め、安全の徹底を図ってください。ケガの原因になります。 - 作業規則項目はロボットの操作手順から作業者間の合図の方法まで、作業内容 に応じた適切なものとしてください。 - 作業規則の作成については、関係作業者および労働安全の専門家等の意見を取り 入れ、定期的に見直し、更新に努めてください。

安全上のご注意

[illegible]

 注意



ロボットのアームを手で動かす場合は、ゆっくりと動かしてください。急速に動かすと、バックラッシュの増大による精度不良や、バックアップデータの破壊を招く場合があります。



姿勢によっては、動作範囲内であっても、シャフト部がベース部と干渉する場合があります。ジョグ操作時には、干渉しないように注意してください。



ロボットの J3 (Z) 軸にはブレーキが付いています。ブレーキが保持されたままの状態
で外部から無理に動かさないでください。
無理に動かすと精度の低下やガタの発生、減速機の損傷につながります。



静電気に帯電したワークを把持する場合は、ハンドおよびロボット本体を通して放電され、ロボットの異常を引き起こす可能性があるため、ハンドとロボット本体は絶縁構造としてください。

また、帯電したワークを置いた際、置かれた方の機器が放電により誤動作する可能性があるため、帯電したワークの電荷を適切な放電ルートで放電させるようシステムを構成してください。

誤動作、故障の原因になります。



低温時にはグリスの性能が十分発揮できないため、暖機運転を行ってください。
位置精度の悪化や誤差過大などのサーボエラーが発生します。



ロボット本体の塗装面にガムテープ等の粘着力の高いテープ、シール類を貼り付けますと、剥がす際に塗装面を傷めるおそれがありますので、ご注意ください。

安全上のご注意

■■■■■■■■■■ 〈JS3 シリーズ〉 **■■■■■■■■■■**

本体とコントローラ

 危険	
	ロボットの電源が ON しているときは、安全防護柵の内側に入ったり、手や頭部等を入れたり等、絶対にしないでください。 ケガの原因になります。
	運転モードに切り替える際、および運転スタートの際は、安全防護柵内に人がいないこと、また運転の妨げになるような障害物がないことを確認してください。 ケガの原因になります。
運転や作業を行う前に、必ず以下のチェックを行ってください。 障害物のチェック 安全防護柵内に障害物がない、または人がいないこと。 設置上のチェック ロボットが正しく設置されていること。ロボットや周辺機器に異常がないこと。 また、ティーチングペンダントや工具等が所定の位置にあること。 非常停止の機能チェック I/O-S 回路（インターロック）、非常停止スイッチが正しく機能していること。 これらのチェックをせずに作業を行うと、危険を引き起こす可能性があります。	
	電源を遮断せずに安全防護領域内（安全防護柵内）に入る場合、必ずティーチングペンダントのセレクトスイッチを「TEACH」（ティーチングモード）にしてください。 「AUTO」（運転モード）になっていると、外部からの指令によりロボットを起動することができ大変危険です。 ケガ、故障の原因になります。
	ティーチング時に一時的に無効にした安全装置があれば、ティーチング終了後にこれを有効にし、本来の機能に復帰させてください。 例 安全防護柵の扉のインターロック装置の有効化等 ケガの原因になります。

安全上のご注意

[illegible]

電源は、必ず定格銘板の表示内でご使用ください。
感電、火災、故障の原因になります。



本体および電源コードに異物が入り込んだり、水、油等がかかったりしないようにしてください。

感電、火災、故障の原因になります。保護等級は「IP20」のため、粉塵やオイルミスト、散水等に関しては無保護です。



ロボット本体・コントローラ内部に異物が混入しないようにしてください。特にねじ・金属片等の導電性異物や油等の可燃性異物が混入した場合は、破裂・破損等の原因になります。



ティーチングペンダントや LAN 等の接続コードやケーブル類は、必ず本体の電源を OFF にしてから抜き差ししてください。

感電、データ消失、故障、誤動作の原因になります。



電源コードは、接続部にほこり等が付着していないことを確認し、確実に接続して固定してください。

確実に接続されていないと接続部が加熱し、火災の原因になります。



長期間ご使用されない場合は、電源コードを供給元の電源から外しておいてください。

ほこり等がたまり、火災の原因になります。



お客様でご用意されたツールを接続する場合は、お客様にてリスクアセスメントを実施し、適切な安全対策を行ってください。
ケガ、故障の原因になります。



焦げ臭い匂いや異常音がする等の異常があった時は、運転を中止して供給元の電源ブレーカを OFF にし、通電されない状態で本機の電源コードを外してください。その後、弊社または代理店までご連絡ください。

異常のまま運転を続けると、感電、火災、故障の原因になります。

[illegible]

 注意	
	本体へのコード、ケーブル類の接続は確実に行ってください。 誤動作、故障の原因になります。
	設定やデータの変更を行った際は、忘れずにデータを保存してください。 本機は、設定やデータの変更を行った後そのまま電源を OFF すると、変更部分が消失し、変更前の状態に戻ってしまいます。
	「故障診断モード」および「機構調整モード」は、メンテナンス作業 * 以外、実行しないでください。

安全上のご注意

[illegible]

コントローラ



通電状態のとき、電源端子台に触らないでください。
感電、ケガの原因になります。



運転や作業を行う際は、非常停止スイッチをすぐに押せる状態にしてください。
非常停止スイッチの周囲に操作の妨げになるものを置くと、非常時に直ちに安全に
ロボットを停止させることができなくなり、危険を引き起こす可能性があります。



個体情報は同機種のロボットでも個体毎に異なります。
バックアップデータを別のロボットへ使用しないでください。
正常に動作できなくなります。



運転中は電源を遮断しないでください。
遮断するとアームの落下や惰走によって周辺装置等と干渉する場合があります。



異常発生時にコントローラの7セグLEDやティーチングペンダントに表示されるエラー番号は、異常の原因を探る重要な手がかりとなります。必ず記録するとともに、取扱説明書『保守編』を参照して対処してください。

安全上のご注意

[illegible]

 危險



原点の再設定時など、安全防護柵内で電源を入れて作業を行う場合には、非常停止スイッチを押してから安全防護柵内に入り、非常停止状態のまま作業を行ってください。

ケガの原因になります。

 警告



保守作業を行う際は、作業者の安全のため、ヘルメット・保護グローブ・保護ゴーグル・安全靴等の保護具を使用してください。

ケガの原因になります。



コントローラの点検や保守を行う際は、通電されていない状態で行ってください。

- 供給元の電源ブレーカを OFF の位置にしてロックアウト／タグアウトを行い、端子台から電源コードを取り外してください。
- 電源コードを取り外してから 5 秒以内は、電源端子台の端子に触れないでください。

感電、ケガ、データ損失や故障の原因になります。



本機を分解する場合は、取扱説明書『保守編』の手順に従い、記載されている方法以外で分解は行わないでください。本機を改造しないでください。
感電、故障の原因になります。
改造に起因する不具合に関し、弊社は一切責任を負いませんのでご了承ください。



安全にご使用いただくために、絶対に本機を改造しないでください。
感電、故障の原因になります。
改造に起因する不具合に関し、弊社は一切責任を負いませんのでご了承ください。



動力遮断後においても、機器に蓄積されたエネルギーが危険を生じる可能性がある場合には、これを徐々に解除する手段を講じてください。

- コントローラ
コンデンサーに溜まった電気を放電させてからコントローラを開けてください。
- ハンド
エアチャックハンドの圧縮空気を電磁弁排気用ホースから排気してください。

[illegible]

注意	
	日常点検および定期点検は必ず実施し、ロボットおよび周辺機器に異常がないことを確認してください。また、その内容を記録して、次期点検時期が判るよう 3 年以上保存してください。
	保守作業中は、「ロボット点検中」等の表示を必要な個所に設置し、第三者が不用意にロボットを操作することがないようにしてください。また、できれば 2 人で作業し、不意の動作に対してロボットの運転を停止させることができるようにしてください。
	円滑な運転をさせ、また長い間ご使用いただけるように、シャフト部は 2,000 km 走行毎に一度を目安として、グリスの補給をしてください。 24 時間稼働の生産ラインで使用される場合には、一定期間における運転時間が長くなるため、グリスの補給期間を上記よりも短縮してください。給油時期は、最高速度での運転の累計値です。
	グリスは弊社指定のグリスをご使用ください。 指定グリス以外のものを使用すると、性能低下や故障の原因になる場合があります。
	関節軸の動作角、または Z 軸の動作距離が微小の軸については、ロボット内部の軸受けの一部だけが摩耗してしまう場合があります。動作させていない軸についても、他の軸の動作による反力またはロボット据付部の振動などによってわずかに動いており、摩耗してしまう場合があります。1 日に 1 回程度、関節軸を 30 度以上、Z 軸は 20 mm 以上動作させることをおすすめします。
	ロボット本体のバッテリーおよびコントローラの電池（組）は充電して再利用したり、焼却処分をしないでください。
	ロボット本体のバッテリーおよびコントローラの電池（組）は産業廃棄物です。このため、処分するときは各自治体の産業廃棄物処理方法に従って正しく処分するようにしてください。

安全上のご注意

[illegible]

コントローラ



ロボットまたは周辺機器に異常が発生した場合や、点検や給油等をする際など、安全装置内に立ち入る場合は、コントローラのブレーカと供給元の電源ブレーカ、どちらも OFF の位置でロックアウト、タグアウトを行い、通電されていない状態で行ってください。

感電、ケガの原因になります。



ヒューズ交換や点検や給油をするときは、供給元の電源を切り、本機から電源コードを取り外して、通電されていない状態で行ってください。

なお、電源コードを取り外してから 5 秒以内は、電源端子台の端子に触れないでください。感電、ケガの原因になります。

1. ポイント作業データの作成手順

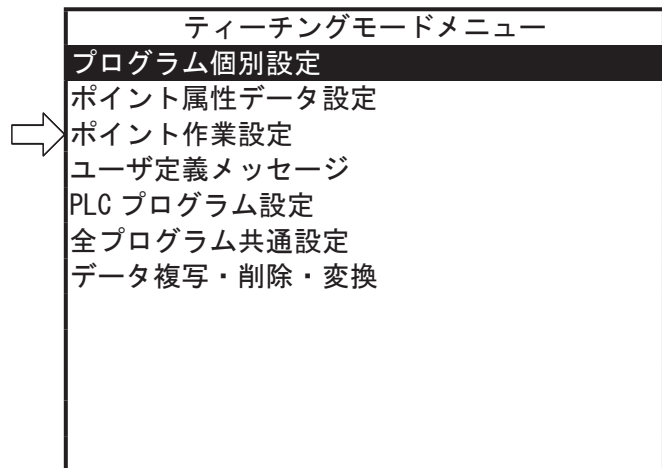
ポイント作業データを作成するにはロボット本体の TP（ティーチングペンダント）を使用する方法と、ロボットと外部機器とをイーサネットで接続する方法があります。

1.1 ロボット本体の TP を使用する

ポイント作業データを作成するには、まず入力画面を表示させる必要があります。ポイント作業データの入力画面を表示させるには、以下の 2 つの手順があります。

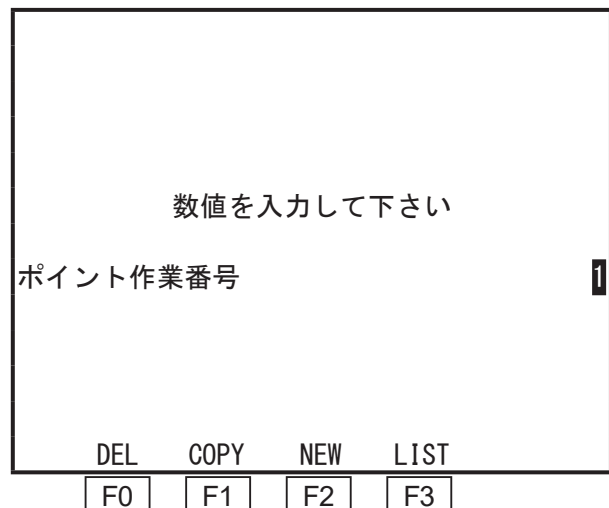
1. ポイント作業データ入力画面を表示させる

MENU キーを押し、ティーチングモードメニューから「ポイント作業設定」を選択してください。



ポイント作業入力画面が表示されます。作成したいポイント作業データの番号を入力し、**ENTR** キーを押してください。ポイント作業の入力画面になります。

番号入力画面で **F2** (NEW) キーを押すと、未入力のポイント作業番号のリストが表示されます。**F3** (LIST) キーを押すと、ティーチング済みのポイント作業番号のリストが表示されます。リストから番号を選択することもできます。



[F4] (VIEW) キーが表示されている画面で数字を入力し、**[ENTR]** キーを押すと、ポイントにポイント作業データの番号が設定されます。ポイント作業の入力画面にはなりません。

また、作成したポイント作業データは、ポイントに番号を設定することで運転時に実行されます。作成したポイント作業データは、ポイントに番号を設定してください。

ポイント作業 1	
001	

〈ポイント作業データ入力画面：新規〉

2. ポイント作業データ入力画面を表示させる

プログラム 1				PTP
X+23	Y+312	Z+25	R+12	
種別			PTP 駆動点	
条件番号			5	
S. MARK E. MARK J. EXEC P. EXEC				

[移動前作業番号] [移動中作業番号]
[移動後作業番号] [CP 駆動中作業番号]
のいずれかを選択してください。

■ ポイント作業データ入力の手順

ポイント作業データの入力画面は、新規の場合は右のように表示されます。[001]は命令番号です。命令番号 [001] を選択してください。

既存のポイント作業データを修正する場合は修正したい命令の行を選択してください。

ポイント作業 1
001

命令番号を選択すると、ポイント作業命令カテゴリ選択画面（右図）が表示されます。

命令カテゴリ選択
ON/OFF 出力制御
条件分岐 (if)、条件待ち (waitCond)
条件演算
ディレイ、データ IN、スタート待ち
パレット操作
実行流れ制御
繰り返し (for, do-loop)
駆動制御
LCD 表示、7 セグ LED
COM 入出力
変数、コメント、システム制御
カメラ、Z 補正

命令カテゴリを選択すると命令選択画面が表示されます。右図は [ON/OFF 出力制御] を選択した例です。入力したい命令を選択してください。

命令を選択すると、必要なパラメータの選択画面が表示されます。例えば [set] 命令を選択した場合は、出力先の選択画面が表示されます。命令によって必要なパラメータが複数ある場合、またはない場合があります。

必要なパラメータを入力／選択すると、命令入力は 1 行終了です。続けて命令を入力する場合は、同様の作業を繰り返してください。

ON/OFF 出力制御
set
reset
pulse
invPulse
delaySet
delayReset
onoffBZ
dataOut
dataOutBCD
onoffGLED
onoffRLED

2. 式の構成

2.1 式

定数、変数、関数（各文字列型、数値型とも）と、演算子を組み合わせたものを式と呼びます。

2.2 定数

数値型定数（例：125、2.0、2e15）と文字列型定数（例："ABC"）があります。

数値型は 8 バイト実数型（double type）、文字列型は 255 バイトとして扱います。

文字列型定数では、「%」を使用することで、文字を 16 進コードで指定することもできます。

例 eoutCOM port2, "%0D%0A" : CR・LF コードの出力

「%」の後に 0～9、A～F、% 以外の文字がある場合は、「%」をそのままの文字として扱います。

例 eoutCOM port2, "%G01" : %G01 と出力

「%」の後に 0～9、A～F の文字があって「%」をそのまま出力したい場合は、「%%」と入力してください。

例 eoutCOM port2, "%%300" : %300 と出力

2.3 変数

変数とは、数値や文字列の値を入れておく入れ物です。

本機では、ロボットの機能として組み込まれている組み込み変数と、自由に定義できるユーザ定義変数が使用できます。

ユーザ定義変数には、ローカル変数（命令「declare」で定義する、定義したポイント作業データ内でのみ有効な変数）、グローバル変数、キープ変数、共通設定変数、条件設定変数、プログラム設定変数といったものがあります。

型：ブール型	1 ビットの変数です。1（真）/0（偽）の値のみ保持します。
数値型	8 バイト実数型（double type）の変数です。
文字列型	255 バイトの変数です。

2.4 関数

数値や文字列の値を与えると、その値を加工して返して来る仕組みを関数と呼びます。
本機では、ロボットの機能として組み込まれている組み込み関数と、自由に定義できるユーザ定義関数が使用できます。
返してくる値の型により、数値型関数、文字列型関数があります。

2.5 演算子

演算子	機能	有効な型 *
+	左項の値に右項の値を加える。	num
-	左項の値から右項の値を引く。	num
*	左項の値に右項の値をかける。	num
/	左項の値を右項の値で割る。	num
&	左項の値に右項の値を接続する。"A" & "B" → "AB"	str
=	左項の値に右項の値を代入する。	num, str
>	左項の値が右項の値より大きい場合、1。同じか小さい場合、0。	num, str
<	左項の値が右項の値より小さい場合、1。同じか大きい場合、0。	num, str
>=, >=	左項の値が右項の値より大きいか等しい場合、1。小さい場合、0。	num, str
<=, <=	左項の値が右項の値より小さいか等しい場合、1。大きい場合、0。	num, str
<>, <>	左項の値と右項の値が等しくない場合、1。等しい場合、0。	num, str
==	左項の値と右項の値が等しい場合、1。等しくない場合、0。	num, str

* 数値型 (num)、文字列型 (str)

■ 演算の優先順位

1. () で囲まれた式
2. 関数、変数
3. 独立した「+」「-」
4. 「*」「/」
5. 「+」「-」「&」
6. 関係演算子 (「>」「>=」「=>」「<」「<=」「<」「<>」「><」)
7. 代入演算子 (「=」)

3. 命令一覧

■で示した命令を含むポイント作業データを CP 連続通過点に設定すると、命令は無視されます。

3.1 ポイント作業

カテゴリ	命令	必要パラメータ	命令内容
ON / OFF 出力制御	set	出力先	ON 出力
	reset	出力先	OFF 出力
	pulse	出力先、時間幅	指定時間幅の ON パルス出力
	invPulse	出力先、時間幅	指定時間幅の OFF パルス出力
	delaySet	出力先、遅延時間	指定した遅延時間後に ON 出力
	delayReset	出力先、遅延時間	指定した遅延時間後に OFF 出力
	onoffBZ	ON 時間、OFF 時間	ブザーを断続的に鳴らす
	onoffGLED	ON 時間、OFF 時間	下記の緑 LED 点滅 - 本体前面（JR3000 シリーズのみ） - スイッチボックス（JR3000/JC-3 シリーズのみ） - コントローラ前面、操作ボックス（JS3 シリーズのみ）
	onoffRLED	ON 時間、OFF 時間	下記の赤 LED 点滅 - 本体前面（JR3000 シリーズのみ） - スイッチボックス（JR3000/JC-3 シリーズのみ） - コントローラ前面、操作ボックス（JS3 シリーズのみ）
	dataOut	出力値、出力先、出力ビット数	数値データ、またはポイントに設定したタグコードを I/O に出力
	dataOutBCD	出力値、出力先、出力ビット数	数値データ、またはポイントに設定したタグコードを I/O に BCD 出力
	motorPowerON	-	ロボットを動力 ON（JS3 シリーズのみ）
	servoON	軸指定	指定軸をサーボ ON（JS3 シリーズのみ）
	servoOFF	軸指定	指定軸をサーボ OFF（JS3 シリーズのみ）

カテゴリ	命令	必要パラメータ	命令内容
条件分岐・条件待ち	if	-	条件分岐
	then	-	真時実行
	else	-	偽時実行
	endif	-	条件分岐終了
	waitCondTime	タイムアウト時間	時間付き条件待ち
	timeUp	-	タイムアップ時実行
	endWait	-	待ち終了
	waitCond	-	条件待ち
条件演算	ld	ブール型変数または式	ON 入力
	ldi	ブール型変数または式	OFF 入力
	and	ブール型変数または式	直列 ON 入力
	ani	ブール型変数または式	直列 OFF 入力
	or	ブール型変数または式	並列 ON 入力
	ori	ブール型変数または式	並列 OFF 入力
	anb	-	ブロックの直列接続
	orb	-	ブロックの並列接続
デイレイ	delay	待ち時間	指定時間だけ停止
	dataIn	代入先変数, 読込先, 読込幅	数値データを I/O から読込
	dataInBCD	代入先変数, 読込先, 読込幅	数値データを I/O から BCD として読込
	waitStart	-	スタート指示を待つ
	waitStartBZ	-	ブザーを断続的に鳴らしながら、スタート指示を待つ
パレット	loopPallet	パレット番号、飛び先ポイント番号	パレット繰返しループ
	resPallet	パレット番号	パレットカウンタリセット
	incPallet	パレット番号	パレットカウンタ加算 (+1)

カテゴリ	命令	必要パラメータ	命令内容
実行流れ制御	callBase	-	ポイント作業番号が設定されたユーザ定義種別ポイントで、種別に定義されたポイント作業を呼び出す
	callJob	ポイント作業番号	指定された番号のポイント作業データをサブルーチンコールする
	callPoints	ポイント列識別子	指定されたポイント列（カスタマイズモードで定義）を実行
	returnJob	-	ポイント作業終了
	returnFunc	式	指定した式の値を戻り値として関数終了（この命令は関数でのみ有効です）
	callProg	プログラム番号	指定された番号のプログラムをサブルーチンコールする
	endProg	-	プログラム終了
	goPoint	駆動条件番号、ポイント番号	指定したポイントへジャンプ
	goRPoint	駆動条件番号、相対ポイント番号	相対指定したポイントへジャンプ
	goCRPoint	PTP 駆動条件番号、移動先選択	CP 駆動の途中から、選択した移動先へジャンプ
	jump	飛び先、ラベル番号	指定ラベルへ飛びます
	Label	ラベル番号	ラベル（飛び先目印）
繰り返し	for	制御変数、初期値、終了値、増分	for から next までの命令を、指定した変数が初期値から終了値になるまで繰り返す
	next	-	
	exitFor	-	for 文から抜け出す
	do	-	do から loop までの命令を繰り返す
	loop	-	
	exitDo	-	do 文から抜け出す
駆動制御	upZ	移動速度、移動距離	Z 上昇
	downZ	移動速度、移動距離	Z 下降
	movetoZ	移動速度、Z 移動位置	Z 移動
	lineMove	速度、各移動（回転）距離	指定した速度で、指定した距離（相対距離）を CP 直線駆動で移動します（相対移動命令） この命令を入力すると、指定した速度と各方向の指定移動距離が以下のように表示されます。 (例) lineMoveSpeed 20 lineMoveX 10 lineMoveY 20 lineMoveZ 0 lineMoveR 0

カテゴリー	命令	必要パラメータ	命令内容
駆動制御	lineMoveStopIf	-	条件により lineMove 命令による移動を途中で止めます。
	endLineMove	-	lineMoveStopIf の条件行終了
	initMec	軸指定	指定した軸をメカ初期化
	checkPos	-	位置ずれ検出 (JR3000 シリーズのみ)
	monoMove	軸指定	指定された 1 軸だけの駆動を行います。ロボットの X (J1), Y (J2), Z (J3), R (J4) 軸および補助軸 MT1, MT2 を指定できます。
	mMovePosition	位置座標値	monoMove 命令で駆動する位置座標値を指定します。 パラメータの単位系は軸ごとに異なります。
	mMoveDistance	距離	monoMove 命令で駆動する距離を指定します。 パラメータの単位系は軸ごとに異なります。
	mMoveSpeed	速度	monoMove 命令で駆動するときの速度を指定します。 パラメータの単位系は軸ごとに異なります。
	mMoveAccelRate	加速度率 [%]	monoMove 命令で駆動するときの加速度を指定します。mMoveAccelRate 命令と mMoveAccelTime 命令のどちらか一方で指定してください。 mMoveAccelRate では、デフォルトの加速度に対する比率を [%] 単位 (百分率) で指定します。 mMoveAccelTime では、mMoveSpeed で指定された速度に到達するまでの時間を [msec] 単位で指定します。
	mMoveAccelTime	加速時間 [msec]	
	monoMoveStopIf	-	monoMove 命令による駆動を条件により停止させます。条件による停止をしない場合は、この命令を入力する必要はありません。
LCD、7SLED	endMonoMove	-	monoMove 命令の終了を示します。
	clrLCD	-	LCD ディスプレイをクリア
	clrLineLCD	行 (1-13)	LCD ディスプレイの指定行をクリア
	outLCD	行 (1-13)、列 (1-40)、文字列	LCD ディスプレイに文字列を表示する
	eoutLCD	行 (1-13)、列 (1-40)、文字列式	LCD ディスプレイに式の結果を表示する
	sys7SLED	-	「out7SLED」で変更した 7 セグ LED の表示を戻す
	out7SLED	表示種類、表示数値	7 セグ LED 出力

カテゴリ	命令	必要パラメータ	命令内容
COM/Ethernet 入出力	outCOM	ポート, 文字列	COM およびイーサネットから文字列出力
	eoutCOM	ポート, 文字列式	COM およびイーサネットから式の結果を出力
	setWTCOM	ポート, 待ち時間	COM およびイーサネット受信待ち時間（タイムアウト時間）を設定
	inCOM	変数名, ポート, 文字数	COM およびイーサネット受信データを指定変数に代入
	cmpCOM	ポート, 文字列	COM およびイーサネット受信データと文字列とを比較 結果はシステムフラグ sysFlag (1) ~ (15) に入る
	ecmpCOM	ポート, 文字列式	COM およびイーサネット受信データと文字列式とを比較 結果はシステムフラグ sysFlag (1) ~ (15) に入る
	clrCOM	ポート	COM およびイーサネット受信バッファをクリア
	shiftCOM	ポート, シフト数	COM およびイーサネット受信データをシフト 先頭からシフト数バイト分のデータを捨てる
	stopPC	-	COM1 およびイーサネットの通信機能を停止 Start PC 命令または電源 OFF まで通信処理を行わない
	startPC	-	COM1 およびイーサネットの通信機能を開始 電源 ON 時には通信可能なので本命令は不要
	connect	ポート, 数値	通信相手に接続する（詳細は「 21.1.2 接続処理 」を参照）
	disconnect	ポート, 数値	通信相手から切断する（詳細は「 21.1.2 接続処理 」を参照）
変数、コメント、システム制御	declare	型, 識別子	ローカル変数の宣言
	let	代入式文字列	右辺の式の計算結果を左辺の変数へ代入 +, -, *, /, =, (,), & 使用可能
	rem	文字列	1 行コメント
	crem		行末コメント（逆インパルス時の表示のみ）
	setProgNum	プログラム番号	プログラム番号の変更 注）運転中にこの命令を実行しないでください 運転中に別のプログラムを運転したい場合は「callProg」命令をご使用ください
	setSeqNum	PLC プログラム番号	「システムデータ」の PLC プログラムの番号を変更します

カテゴリ	命令	必要パラメータ	命令内容
カメラ、Zセンサ	cameraWadj	ワーク補正番号	カメラで画像を撮り込み、得られたデータを「ワーク補正」の設定に従い、補正値を算出します
	wCameraWadj	ワーク補正番号、ショット番目	2度の撮像により、「ワーク補正」の補正値を算出する場合にこの命令を使用します
	multiCamWadj	ワーク補正番号	カウンタ付カメラ補正を実行
	incMultiCamWadj	ワーク補正番号	カウンタ付カメラ補正の補助値カウンタをインクリメント
	clrMultiCamWadj	ワーク補正番号	カウンタ付カメラ補正の補助値をゼロクリア
	cameraTool	切り替えツール番号	カメラで画像を撮り込み、得られたデータから「切り替えツール」の設定に従って「TCP-X」「TCP-Y」を算出します
	cameraPallet	パレット番号	カメラで画像を撮り込み、撮り込んだマークの数とその座標を「パレット」の実行回数と各座標とします
	takeZWadj	ワーク補正番号	距離センサ/タッチセンサにより、得られたデータを「ワーク補正」の設定に従い、Z方向の補正値を算出します
	multiTakeZWadj	ワーク補正番号	カウンタ付CCDカメラ補正設定のZ補正メニューで設定された内容を実行します。実行するごとにワーク補正カウンタ内に補正量を記録します。

注) 起動源が COM1 以外の場合の COM1 の動作は、システムプログラムのバージョンによって異なります。

- ・ システムプログラム Ver.6 より前のバージョン (JR3000/JC-3 シリーズのみ)
起動源が「COM1」以外に設定されている場合、COM1 のすべての通信コマンドの機能が停止となります。
- ・ システムプログラム Ver.6 以降のバージョン
起動源が「COM1」以外に設定されている場合でも、COM1 にて駆動を伴わない通信コマンドが機能します。「inCOM」「outCOM」などの命令を使用して COM1 で任意の通信処理を行う場合は、事前に「stopPC」で通信機能を停止してください。「stopPC」で通信機能を停止しない場合は、ユーザーが任意に定義した文字列の入力に通信エラーの応答を返すため、正常に通信ができなくなります。

注) カメラ、Zセンサの命令に関しては、取扱説明書『カメラ・センサ機能編』をご参照ください。単独駆動の命令に関しては、取扱説明書『補助軸機能編』をご参照ください。なお、単独駆動命令は I/O-MT コネクタ (オプション :JR3200 シリーズでは I/O-1 との選択) を有する場合のみ有効です。

3.2 実行条件

カテゴリ	命令	必要パラメータ	命令内容
条件演算	ld	ブール型変数または式	ON 入力
	ldi	ブール型変数または式	OFF 入力
	and	ブール型変数または式	直列 ON 入力
	ani	ブール型変数または式	直列 OFF 入力
	or	ブール型変数または式	並列 ON 入力
	ori	ブール型変数または式	並列 OFF 入力
	anb	-	ブロックの直列接続
	orb	-	ブロックの並列接続

3.3 PLC プログラム

カテゴリ	命令	必要パラメータ	命令内容
接点扱い	ld	ブール型変数	ON 入力
	ldi	ブール型変数	OFF 入力
	and	ブール型変数	直列 ON 入力
	ani	ブール型変数	直列 OFF 入力
	or	ブール型変数	並列 ON 入力
	ori	ブール型変数	並列 OFF 入力
コイル扱い	out	出力先	コイル駆動
	set	出力先	動作保持出力
	reset	出力先	動作保持解除
	pls	出力先	立ち上がりパルス出力
	plf	出力先	立ち下がりパルス出力
接続命令	anb	-	並列回路ブロック直列接続
	orb	-	直列回路ブロック並列接続
	mps	-	演算途中結果の記憶
	mrd	-	演算途中結果の読み出し
	mpp	-	演算途中結果の読み出しとリセット
その他	nop	-	無処理

4. 変数一覧

本機では、ロボットの機能として組み込まれている組み込み変数と、自由に定義できるユーザ定義変数が使用できます。

■ ユーザ定義変数

ローカル変数（命令「declare」で定義する、定義したポイント作業データ内でのみ有効な変数）、グローバル変数、キープ変数、共通設定変数、条件設定変数、プログラム設定変数といったものがあります。

■ 組み込み変数

文字、式入力画面で LCD 最下行（F3 キーの上）に「BVar」と表示されている場合に、F3 キーを押すと、組み込み変数のリストが表示されます。

組み込み変数一覧を下表に示します。

「型」は変数の型を表します。

boo ブール型。1 ビットの変数です。1（真）と 0（偽）のみの変数です。

num 数値型（numeric 型）です。8 バイトの実数値の変数です。

str 文字列型（string 型）です。最大 255 バイトです。

「アクセス」は読み書きの制限を表します。

R 読み込みのみ許可されています。（リードオンリー）

W 書き込みのみ許可されています。（ライトオンリー）

R/W 読み込みと書き込みが可能です。

• 自由変数

型	識別子	概要	アクセス	備考
boo	#mv (1 ~ 99)	ブール型変数	R/W	
boo	#mkv (1 ~ 99)	ブール型変数（キープ変数）	R/W	*
num	#nv (1 ~ 99)	数値型変数	R/W	
num	#nkV (1 ~ 99)	数値型変数（キープ変数）	R/W	*
str	#sv (1 ~ 99)	文字列型変数	R/W	
str	#skv (1 ~ 99)	文字列型変数（キープ変数）	R/W	*

* キープ変数は、ロボットに搭載されている電池によって、ロボットの電源を OFF しても値が保持されます。

• 入力変数

型	識別子	概要	アクセス	備考
boo	#sysIn1 ~ #sysIn16	I/O-SYS 入力 (JR3000/JC-3 シリーズ)	R	
boo	#sysIn1 ~ #sysIn15	I/O-SYS 入力 (JS3 シリーズ)	R	
boo	#genIn1 ~ #genIn8	I/O-1 入力 (JR3000/JC-3 シリーズ)	R	
boo	#genIn1 ~ #genIn18	I/O-1 入力 (JS3 シリーズ)	R	
boo	#handIn1 ~ #handIn8	I/O-H 入力 (JS3 シリーズのみ)	R	
num	#fbIn (a, b) a=I/O アドレス (0x1000 ~ 0x17FF) b= ビット幅 (1 ~ 32)	フィールドバス I/O 入力	R	*

* フィールドバス I/O 変数について

- ビット幅 1 でアクセスする場合はブール型、2 以上のビット幅でアクセスする場合は数値型となります。
- I/O アドレスは、0x に続けて 4 桁の 16 進数で指定してください。
- 指定できるビット幅は、1 ~ 32 ビットで、最大 2 ワード (4 バイト) 指定することができます。ただし、フィールドバスの設定により、フィールドバス入出力エリア範囲外になった場合、範囲外のビットは切り捨てになります。

• 出力変数

型	識別子	概要	アクセス	備考
boo	#sysOut1 ~ #sysOut16	I/O-SYS 出力 (JR3000/JC-3 シリーズ)	W	
boo	#sysOut1 ~ #sysOut14	I/O-SYS 出力 (JS3 シリーズ)	W	
boo	#genOut1 ~ #genOut8	I/O-1 出力 (JR3000/JC-3 シリーズ)	W	
boo	#genOut1 ~ #genOut22	I/O-1 出力 (JS3 シリーズ)	W	
boo	#handOut1 ~ #handOut8	I/O-H 出力 (JS3 シリーズのみ)	W	
num	#fbOut (a, b) a=I/O アドレス (0x1800 ~ 0x1FFF) b= ビット幅 (1 ~ 32)	フィールドバス I/O 出力	W	*

* フィールドバス I/O 変数について

- ビット幅 1 でアクセスする場合はブール型、2 以上のビット幅でアクセスする場合は数値型となります。
- I/O アドレスは、0x に続けて 4 桁の 16 進数で指定してください。
- 指定できるビット幅は、1 ~ 32 ビットで、最大 2 ワード (4 バイト) 指定することができます。ただし、フィールドバスの設定により、フィールドバス入出力エリア範囲外になった場合、範囲外のビットは切り捨てになります。

• システムフラグ

型	識別子	概要	アクセス	備考
boo	#sysFlag (1 ~ 999)	システムフラグ 「 6. システムフラグ一覧 」をご参照ください。	R	

• ハードウェア

型	識別子	概要	アクセス	備考
boo	#optionLED (1 ~ 3)	オプション LED	R/W	

• 特殊変数

型	識別子	概要	アクセス	備考
num	#downTimer1 ~ #downTimer10	ダウンタイマー msec 単位	R/W	
num	#jobStartHight	移動後作業開始高さ mm 単位	R/W	
num	#jobStartX	移動後作業開始 X (J1) 軸 mm 単位	R/W	
num	#jobStartY	移動後作業開始 Y (J2) 軸 mm 単位	R/W	
num	#jobStartR	移動後作業開始 R (J4) 軸 deg 単位	R/W	
num	#jobStartMT1	移動後作業開始 MT1 軸 任意単位	R/W	
num	#jobStartMT2	移動後作業開始 MT2 軸 任意単位	R/W	
num	#priorityPTPCondNum	優先 PTP 駆動条件番号	R/W	

• パレット

型	識別子	概要	アクセス	備考
boo	#palletFlag (1 ~ 100)	パレットフラグ	R	*
num	#palletCount (1 ~ 100)	パレットカウンタ	R/W	*

* JS3 シリーズのピック & プレイス仕様をお使いの場合、パレットフラグ、パレットカウンタの 41 ~ 100 は予約されておりますので、使用しないでください。

• PLC プログラム

型	識別子	概要	アクセス	備考
boo	#seqT (1 ~ 99)	PLC タイマ #setTCount が指定した値以上になったら 1 (真)	R/W	
num	#seqTCount (1 ~ 99) 1 ~ 50 : 積算タイマ 51 ~ 99 : 非積算タイマ	PLC タイマカウント値	R/W	
boo	#seqC (1 ~ 99)	PLC カウンタ #seqCCount が指定した値以上になったら 1 (真)	R/W	
num	#seqCCount (1 ~ 99)	PLC カウンタカウント値	R/W	

• ワーク補正（ポイント属性データ）

型	識別子	概要	アクセス	備考
num	#workAdj_X (1 ~ 3000)	X 補正量 mm 単位	R/W	
num	#workAdj_Y (1 ~ 3000)	Y 補正量 mm 単位	R/W	
num	#workAdj_Z (1 ~ 3000)	Z 補正量 mm 単位	R/W	
num	#workAdj_R (1 ~ 3000)	R 補正量 deg 単位	R/W	
num	#workAdj_Rotation (1 ~ 3000)	回転補正量 deg 単位	R/W	

• カウンタ付き CCD カメラワーク補正

型	識別子	概要	アクセス	備考
num	#mulWorkAdj_Wrt_Cam	カメラ補正量書込カウンタ	R	
num	#mulWorkAdj_Wrt_Zadj	Z 補正量書込みカウンタ	R	
num	#mulWorkAdj_Read	補正值読出しカウンタ	R/W	
num	#mulWorkAdj_Num	カウンタワーク補正番号	R/W	
num	#mulWorkAdj_Result	カウンタデータ結果 0= 失敗、1= 成功	R/W	

• 現在のプログラムの実行中のポイント

型	識別子	概要	アクセス	備考
num	#point_X	X (J1) 軸座標 mm 単位	R/W	
num	#point_Y	Y (J2) 軸座標 mm 単位	R/W	
num	#point_Z	Z (J3) 軸座標 mm 単位	R/W	
num	#point_R	R (J4) 軸座標 deg 単位	R/W	
num	#point_MT1	MT1 軸座標 任意単位	R/W	*
num	#point_MT2	MT2 軸座標 任意単位	R/W	*
num	#point_LineSpeed	線速度 mm/s 単位	R/W	
num	#point_CondNum	条件番号	R/W	
num	#point_MoveBeforeNum	移動前作業番号	R/W	
num	#point_MovingNum	移動中作業番号	R/W	
num	#point_MoveAfterNum	移動後作業番号	R/W	
num	#point_CPWorkNum	CP 駆動中作業番号	R/W	
num	#point_PTPCondNum	PTP 駆動条件番号	R/W	
num	#point_CPCondNum	CP 駆動条件番号	R/W	
num	#point_ToolNum	切り替えツール番号	R/W	
num	#point_PalletNum	パレット番号	R/W	
num	#point_WorkAdjNum	ワーク補正番号	R/W	
num	#point_RunCondNum	実行条件番号	R/W	
num	#point_TagCode	タグコード	R/W	

* MT1, MT2 の位置座標の単位系は、別途、補助軸コンフィグレーションで設定された単位系です。

- 現在のプログラムの任意のポイント

a= ポイント番号 (0 ~最終ポイント数)

型	識別子	概要	アクセス	備考
num	#P_X (a)	X (J1) 軸座標 mm 単位	R/W	*2
num	#P_Y (a)	Y (J2) 軸座標 mm 単位	R/W	*2
num	#P_Z (a)	Z (J3) 軸座標 mm 単位	R/W	*2
num	#P_R (a)	R (J4) 軸座標 deg 単位	R/W	*2
num	#P_MT1 (a)	MT1 軸座標 任意単位	R/W	*1 *2
num	#P_MT2 (a)	MT2 軸座標 任意単位	R/W	*1 *2
num	#P_LineSpeed (a)	線速度 mm/s 単位	R/W	*2
num	#P_CondNum (a)	条件番号	R/W	*2
num	#P_MoveBeforeNum (a)	移動前作業番号	R/W	*2
num	#P_MovingNum (a)	移動中作業番号	R/W	*2
num	#P_MoveAfterNum (a)	移動後作業番号	R/W	*2
num	#P_CPWorkNum (a)	CP 駆動中作業番号	R/W	*2
num	#P_PTPCondNum (a)	PTP 駆動条件番号	R/W	*2
num	#P_CPCondNum (a)	CP 駆動条件番号	R/W	*2
num	#P_ToolNum (a)	切り替えツール番号	R/W	*2
num	#P_PalletNum (a)	パレット番号	R/W	*2
num	#P_WorkAdjNum (a)	ワーク補正番号	R/W	*2
num	#P_RunCondNum (a)	実行条件番号	R/W	*2
num	#P_TagCode (a)	タグコード	R/W	*2

* 1: MT1, MT2 の位置座標の単位系は、別途、補助軸コンフィグレーションで設定された単位系です。

* 2: ポイント番号に 0 を指定すると作業原点のポイントデータにアクセスすることができます。アクセス先は、プログラム設定の選択状況（全プログラム共通設定／プログラム個別設定）に従います。個別作業原点はリード、ライト可能です。共通作業原点はリードオンリーであり、ライトはできません。

• 任意のプログラム番号の任意のポイント

a= プログラム番号 (1 ~ 999) , b= ポイント番号 (0 ~ 最終ポイント数)

型	識別子	概要	アクセス	備考
num	#prog_P_X (a, b)	X (J1) 軸座標 mm 単位	R/W	*2
num	#prog_P_Y (a, b)	Y (J2) 軸座標 mm 単位	R/W	*2
num	#prog_P_Z (a, b)	Z (J3) 軸座標 mm 単位	R/W	*2
num	#prog_P_R (a, b)	R (J4) 軸座標 deg 単位	R/W	*2
num	#prog_P_MT1 (a, b)	MT1 軸座標 任意単位	R/W	*1 *2
num	#prog_P_MT2 (a, b)	MT2 軸座標 任意単位	R/W	*1 *2
num	#prog_P_LineSpeed (a, b)	線速度 mm/s 単位	R/W	*2
num	#prog_P_CondNum (a, b)	条件番号	R/W	*2
num	#prog_P_MoveBeforeNum (a, b)	移動前作業番号	R/W	*2
num	#prog_P_MovingNum (a, b)	移動中作業番号	R/W	*2
num	#prog_P_MoveAfterNum (a, b)	移動後作業番号	R/W	*2
num	#prog_P_CPWorkNum (a, b)	CP 駆動中作業番号	R/W	*2
num	#prog_P_PTPCondNum (a, b)	PTP 駆動条件番号	R/W	*2
num	#prog_P_CPCondNum (a, b)	CP 駆動条件番号	R/W	*2
num	#prog_P_ToolNum (a, b)	切り替えツール番号	R/W	*2
num	#prog_P_PalletNum (a, b)	パレット番号	R/W	*2
num	#prog_P_WorkAdjNum (a, b)	ワーク補正番号	R/W	*2
num	#prog_P_RunCondNum (a, b)	実行条件番号	R/W	*2
num	#prog_P_TagCode (a, b)	タグコード	R/W	*2

* 1: MT1, MT2 の位置座標の単位系は、別途、補助軸コンフィグレーションで設定された単位系です。

* 2: ポイント番号に 0 を指定すると作業原点のポイントデータにアクセスすることができます。アクセス先は、プログラム設定の選択状況 (全プログラム共通設定 / プログラム個別設定) に従います。個別作業原点はリード、ライト可能です。共通作業原点はリードオンリーであり、ライトはできません。

• 全プログラム共通設定 ツールデータ

型	識別子	概要	アクセス	備考
num	#comm_ToolData_Mass	ツール質量 (質量番号の選択)	R/W	*
num	#comm_ToolData_X	TCP-X mm 単位	R/W	
num	#comm_ToolData_Y	TCP-Y mm 単位	R/W	
num	#comm_ToolData_DeltaZ	TCP- Δ Z mm 単位	R/W	

* ツール質量の番号と kg 単位の質量との対応関係は機種ごとに異なります。下表の通りです。

	JR3200	JR3300 ~ JR3600	JR3303F JR3403F	JC-3 3 軸片持ち	JC-3 3 軸両持ち	JC-3 4 軸両持ち	JS3
0	1 kg	1 kg	1 kg	4 kg	8 kg	3 kg	1 kg
1	3.5 kg	4 kg	5 kg	-	-	-	3 kg
2	-	7 kg	10 kg	-	-	-	6 kg
3	-	-	15 kg	-	-	-	-

• 全プログラム共通設定 PTP 駆動条件

型	識別子	概要	アクセス	備考
num	#comm_PTPData_Speed	PTP 速度 % 単位	R/W	
num	#comm_PTPData_R_Speed	R 軸回転速度 % 単位	R/W	
num	#comm_PTPData_R_Acc	R 軸回転加速度 % 単位	R/W	
num	#comm_PTPData_Archmotion	アーチモーション 0=Z 移動相対距離指定 1=Z 移動絶対位置指定	R/W	
num	#comm_PTPData_Z_Height	Z 移動高さ mm 単位	R/W	
num	#comm_PTPData_Z_Up_Dis	Z のみ上昇距離 mm 単位	R/W	
num	#comm_PTPData_Z_Down_Dis	Z のみ下降距離 mm 単位	R/W	
num	#comm_PTPData_Move_Dis_Pos	水平移動位置 mm 単位	R/W	
num	#comm_PTPData_Move_Start_Pos	水平開始位置 mm 単位	R/W	
num	#comm_PTPData_Down_Start_Pos	下降開始位置 mm 単位	R/W	

• 全プログラム共通設定 CP 駆動条件

型	識別子	概要	アクセス	備考
num	#comm_CPData_Acc	CP 加速度 % 単位	R/W	
num	#comm_CPData_R_Speed	R 軸回転速度 % 単位	R/W	
num	#comm_CPData_R_Acc	R 軸回転加速度 % 単位	R/W	

• プログラム個別設定 ツールデータ

型	識別子	概要	アクセス	備考
num	#prog_ToolData_EachCommon (1 ~ 999)	プログラム設定の選択 0= 全プログラム共通設定 1= プログラム個別設定	R/W	
num	#prog_ToolData_Mass (1 ~ 999)	ツール質量 (質量番号の選択)	R/W	*
num	#prog_ToolData_X (1 ~ 999)	TCP-X mm 単位	R/W	
num	#prog_ToolData_Y (1 ~ 999)	TCP-Y mm 単位	R/W	
num	#prog_ToolData_DeltaZ (1 ~ 999)	TCP- Δ Z mm 単位	R/W	

* MT1, MT2 の位置座標の単位系は、別途、補助軸コンフィグレーションで設定された単位系です。

• プログラム個別設定 PTP 駆動条件

型	識別子	概要	アクセス	備考
num	#prog_PTPData_EachCommon (1 ~ 999)	プログラム設定の選択 0= 全プログラム共通設定 1= プログラム個別設定	R/W	
num	#prog_PTPData_Speed (1 ~ 999)	PTP 速度 % 単位	R/W	
num	#prog_PTPData_R_Speed (1 ~ 999)	R 軸回転速度 % 単位	R/W	
num	#prog_PTPData_R_Acc (1 ~ 999)	R 軸回転加速度 % 単位	R/W	
num	#prog_PTPData_Archmotion (1 ~ 999)	アーチモーション 0=Z 移動相対距離指定 1=Z 移動絶対位置指定	R/W	
num	#prog_PTPData_Z_Height (1 ~ 999)	Z 移動高さ mm 単位	R/W	
num	#prog_PTPData_Z_Up_Dis (1 ~ 999)	Z のみ上昇距離 mm 単位	R/W	
num	#prog_PTPData_Z_Down_Dis (1 ~ 999)	Z のみ下降距離 mm 単位	R/W	
num	#prog_PTPData_Move_Dis_Pos (1 ~ 999)	水平移動位置 mm 単位	R/W	
num	#prog_PTPData_Move_Start_Pos (1 ~ 999)	水平開始位置 mm 単位	R/W	
num	#prog_PTPData_Down_Start_Pos (1 ~ 999)	下降開始位置 mm 単位	R/W	

• プログラム個別設定 CP 駆動条件

型	識別子	概要	アクセス	備考
num	#prog_CPData_EachCommon (1 ~ 999)	プログラム設定の選択 0= 全プログラム共通設定 1= プログラム個別設定	R/W	
num	#prog_CPData_Acc (1 ~ 999)	CP 加速度 % 単位	R/W	
num	#prog_CPData_R_Speed (1 ~ 999)	R 軸回転速度 % 単位	R/W	
num	#prog_CPData_R_Acc (1 ~ 999)	R 軸回転加速度 % 単位	R/W	

• ポイント属性データ 切り替えツール

型	識別子	概要	アクセス	備考
num	#tool_Mass (1 ~ 100)	ツール質量 (質量番号の選択)	R/W	*
num	#tool_X (1 ~ 100)	TCP-X mm 単位	R/W	
num	#tool_Y (1 ~ 100)	TCP-Y mm 単位	R/W	
num	#tool_Z (1 ~ 100)	TCP- Δ Z mm 単位	R/W	
num	#tool_R (1 ~ 100)	R 軸回転量 deg 単位 (カメラ設定 TCP の場合)	R/W	

* ツール質量の番号と kg 単位の質量との対応関係は機種ごとに異なります。下表の通りです。

	JR3200	JR3300 ~ JR3600	JR3303F JR3403F	JC-3 3 軸片持ち	JC-3 3 軸両持ち	JC-3 4 軸両持ち	JS3
0	1 kg	1 kg	1 kg	4 kg	8 kg	3 kg	1 kg
1	3.5 kg	4 kg	5 kg	-	-	-	3 kg
2	-	7 kg	10 kg	-	-	-	6 kg
3	-	-	15 kg	-	-	-	-

• ポイント属性データ PTP 駆動条件

型	識別子	概要	アクセス	備考
num	#ptp_Speed (1 ~ 100)	PTP 速度 % 単位	R/W	
num	#ptp_R_Speed (1 ~ 100)	R 軸回転速度 % 単位	R/W	
num	#ptp_R_Acc (1 ~ 100)	R 軸回転加速度 % 単位	R/W	
num	#ptp_Archmotion (1 ~ 100)	アーチモーション 0=Z 移動相対距離指定 1=Z 移動絶対位置指定	R/W	
num	#ptp_Z_Height (1 ~ 100)	Z 移動高さ mm 単位	R/W	
num	#ptp_Z_Up_Dis (1 ~ 100)	Z のみ上昇距離 mm 単位	R/W	
num	#ptp_Z_Down_Dis (1 ~ 100)	Z のみ下降距離 mm 単位	R/W	
num	#ptp_Move_Dis_Pos (1 ~ 100)	水平移動位置 mm 単位	R/W	
num	#ptp_Move_Start_Pos (1 ~ 100)	水平開始位置 mm 単位	R/W	
num	#ptp_Down_Start_Pos (1 ~ 100)	下降開始位置 mm 単位	R/W	

• ポイント属性データ CP 駆動条件

型	識別子	概要	アクセス	備考
num	#cp_Acc (1 ~ 100)	CP 加速度 % 単位	R/W	
num	#cp_R_Speed (1 ~ 100)	R 軸回転速度 % 単位	R/W	
num	#cp_R_Acc (1 ~ 100)	R 軸回転加速度 % 単位	R/W	

• 機種情報等

型	識別子	概要	アクセス	備考
num	#Model_Series	シリーズ情報 必ず 3 が返されます。	R	
num	#Model_AxisNum	ロボットの機構軸数 2=2 軸仕様 (XY) 3=3 軸仕様 (XYZ) 4=4 軸仕様 (XYZR (J1J2J3J4))	R	
num	#Model_Info	機種情報 0= 卓上ロボット JR3000 1= スカラロボット JS3 2= 直交ロボット JC-3	R	
num	#Model_AuxAxis	補助軸 I/O-MT の有無 0= 無し 1= 有り	R	
num	#Model_Language	現在選択中の言語 0= 英語, 1= 日本語 2= ドイツ語, 3= イタリア語 4= スペイン語, 5= フランス語 6= 韓国語, 7= 中国語 (簡体字) 8= チェコ語, 9= ベトナム語 10= 中国語 (繁体字)	R	

5. 関数一覧

本機では、ロボットの機能として組み込まれている組み込み関数と、自由に定義できるユーザ定義関数が使用できます。

ユーザ定義関数 : ティーチングモード、またはカスタマイズモードで定義します（カスタマイズモードに関しては、取扱説明書『機能編 IV（カスタマイズ）』をご参照ください）。

組み込み関数 : 文字、式入力画面で LCD 最下行（**F2** キーの上）に「BFunc」と表示されている場合に、**F2** キーを押すと、組み込み関数のリストが表示されます。

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識 別 子	概 要
ロボット系	num	currentMainProgNumber()	現在実行中のメインプログラム番号
	num	currentSubProgNumber()	現在実行中のサブプログラム番号
	num	currentPointNumber()	現在実行中のポイント番号
	num	currentArmX()	現在の X 座標、[mm] 単位
	num	currentArmY()	現在の Y 座標、[mm] 単位
	num	currentArmZ()	現在の Z 座標、[mm] 単位
	num	currentArmR()	現在の R 座標、[度] 単位
	num	currentArmH()	現在の座標の手系（1: 右手系 -1: 左手系） JR3000/JC-3 シリーズでは、1（右手系）に固定されています。
	num	currentCmdArmX()	現在の指令 X 座標、[mm] 単位
	num	currentCmdArmY()	現在の指令 Y 座標、[mm] 単位
	num	currentCmdArmZ()	現在の指令 Z 座標、[mm] 単位
	num	currentCmdArmR()	現在の指令 R 座標、[度] 単位
	num	numCOM(COM ポート番号)	COM 受信ポートのデータバイト数
	num	moveAPTP (num a, num b, num X, num Y, num Z, num R)	絶対位置指定 PTP 駆動関数。指定した位置まで PTP 駆動で移動します。
	num	moveAPTP6 (num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)	

種類	型	識 別 子	概 要
ロボット系	num	moveRPTP (num a, num b, num X, num Y, num Z, num R)	相対位置指定 PTP 駆動関数。現在の位置から指定した距離だけ離れた位置まで PTP 駆動で移動します。
	num	moveRPTP6 (num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)	
	num	isConditionData(n)	条件データ有無（1: 有り、0: 無し） 指定した番号の条件データがあるかどうか
	str	strCenterLCD(a)	ティーチングペンダント LCD 表示でセンタリング（中央揃え）になるように文字列を調整します。

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識 別 子	概 要
ロボット系	str	strRightLCD(a)	ティーチングペンダント LCD 表示で右寄せになるように文字列を調整します。
	str	strPlusRLCD(a, b)	ティーチングペンダント LCD 表示用、右優先文字列の左右振り分け結合
	str	strPlusLLCD(a, b)	ティーチングペンダント LCD 表示用、左優先文字列の左右振り分け結合
	num	getSystemPTPmoveTime()	移動中作業でのみ有効 現在の PTP 駆動に必要な時間、[秒] 単位
	num	getSystemPTPrestTime()	移動中作業でのみ有効 現在の PTP 駆動の残り時間（到達までの時間）、[秒] 単位
	num	Pause(X)	一時停止。 ただし、駆動途中での一時停止はできません。カッコ内の引数は値参照実行時の pause 停止番号
	str	getUserMessage(num x)	x で指定された番号のユーザ定義メッセージを取得します。
	num	addPointSkip()	ポイント指定の欠品飛ばしを登録します。
	num	delPointSkip()	ポイント指定の欠品飛ばしを解除します。
	num	clearPointSkip()	ポイント指定の欠品飛ばしを全解除します。
	num	addPalletSkip()	パレット指定の欠品飛ばしを登録します。
	num	delPalletSkip()	パレット指定の欠品飛ばしを解除します。
	num	clearPalletSkip()	パレット指定の欠品飛ばしを全解除します。
	num	qCameraWadj(num X, num Y)	4 点の撮影によりワーク補正の補正値を算出します。

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識別子	概要
算術系	num	abs(x)	絶対値
	num	max(x, y)	最大値
	num	min(x, y)	最小値
	num	degrad(x)	度からラジアンへ変換 ($x * \pi / 180$)
	num	raddeg(x)	ラジアンから度へ変換 ($x * 180 / \pi$)
	num	sqrt(x)	平方根
	num	sin(x)	正弦
	num	cos(x)	余弦
	num	tan(x)	正接
	num	atan(x)	逆正接
	num	atan2(x, y)	y を x で割った値 (y/x) の逆正接
	num	int(x)	x を超えない最大の整数 例 : int(1.3) → 1, int(-1.3) → -2
	num	ip(x)	x の整数部。sgn(x) * int(abs(x)) (sgn(x) は、x が負の数の場合 -1、正の数の場合 +1) 例 : ip(1.3) → 1, ip(-1.3) → -1
	num	fp(x)	x の小数部。x-ip(x) 例 : fp(1.3) → 0.3, fp(-1.3) → -0.3
	num	mod(x, y)	y を法とする x の値 $x - y * \text{int}(x/y)$
	num	remainder(x, y)	x を y で割った余り $x - y * \text{ip}(x/y)$
	num	pow(x, y)	x の y 乗

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識別子	概要
文字列系	str	chr(x)	与えられた文字コードを持つ文字列（1文字）を返します。
	num	ord(a)	先頭文字コード値を返します。先頭以外は無視します。aで与えられた文字列の数が0文字の場合は0を返します。
	num	len(a)	文字列の長さ（バイト長）を返す 多バイトコードには対応していません
	num	strPos(a, b)	aの中でbと一致する最初の部分文字列位置を返します。
	str	strMid(a, n, m)	与えられた文字列aの先頭から数えてn文字目からm文字分の文字列を返します。
	str	str(x)	数値を10進数文字列に変換します。
	str	strBin(n, m)	数値を2進数文字列に変換します。 mは2進文字列桁数
	str	strHex(n, m)	数値を16進数文字列に変換します。 mは16進文字列桁数
	str	str1SI(x)	数値を1バイト符号付整数に丸め、1バイト文字列に変換します。（1 byte Signed Integer）
	str	str2SIBE(x)	数値を2バイト符号付整数に丸め、上位、下位順の2バイト文字列に変換します。（2 byte Signed Integer Big Endian）
	str	str2SILE(x)	数値を2バイト符号付整数に丸め、下位、上位順の2バイト文字列に変換します。（2 byte Signed Integer Little Endian）
	str	str4SIBE(x)	数値を4バイト符号付整数に丸め、上位、下位順の4バイト文字列に変換します。（4 byte Signed Integer Big Endian）
	str	str4SILE(x)	数値を4バイト符号付整数に丸め、下位、上位順の4バイト文字列に変換します。（4 byte Signed Integer Little Endian）
	str	str4FBE(x)	数値を浮動小数として、上位、下位順の4バイト文字列に変換します。（4 byte Float Big Endian）
	str	str4FLE(x)	数値を浮動小数として、下位、上位順の4バイト文字列に変換します。（4 byte Float Little Endian）
	str	str8DBE(x)	数値を倍精度浮動小数として、上位、下位順の8バイト文字列に変換します。（8 byte Double Big Endian）
	str	str8DLE(x)	数値を倍精度浮動小数として、下位、上位順の8バイト文字列に変換します。（8 byte Double Little Endian）
	num	val(a)	文字列を10進文字列と見なして数値（符号無しの整数型）に変換します。 文字列の先頭が負符号の場合は0を返します。

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識 別 子	概 要
文字列系	num	valBin(a)	文字列を2進文字列('0', '1'の並び)と見なして数値に変換します。
	num	valHex(a)	文字列を16進文字列('0' ~ '9', 'A' ~ 'F' または 'a' ~ 'f'の並び)と見なして数値に変換します
	num	val1SI(a)	先頭1文字を1バイト符号付整数として変換します。 (1 byte Signed Integer)
	num	val2SIBE(a)	先頭2文字を上位、下位順の2バイト符号付整数とみなして変換します。(2 byte Signed Integer Big Endian)
	num	val2SILE(a)	先頭2文字を下位、上位順の2バイト符号付整数とみなして変換します。(2 byte Signed Integer Little Endian)
	num	val4SIBE(a)	先頭4文字を上位、下位順の4バイト符号付整数とみなして変換します。(4 byte Signed Integer Big Endian)
	num	val4SILE(a)	先頭4文字を下位、上位順の4バイト符号付整数とみなして変換します。(4 byte Signed Integer Little Endian)
	num	val4FBE(a)	先頭4文字を上位、下位順の浮動小数とみなして変換します。(4 byte Float Big Endian)
	num	val4FLE(a)	先頭4文字を下位、上位順の浮動小数とみなして変換します。(4 byte Float Little Endian)
	num	val8DBE(a)	先頭8文字を上位、下位順の倍精度浮動小数とみなして変換します。(8 byte Double Big Endian)
	num	val8DLE(a)	先頭8文字を下位、上位順の倍精度浮動小数とみなして変換します。(8 byte Double Little Endian)
	num	valSum(a)	文字列コードを先頭から末尾まで加算した値を返します。
	num	valCRC(a)	文字列をビット列とみなしてこれを、 生成多項式 $X^{16}+X^{12}+X^5+1$ で割った余り
	str	bitNot(a)	ビット反転
	str	bitAnd(a, b)	ビット論理積
	str	bitOr(a, b)	ビット論理和
	str	bitXor(a, b)	ビット排他的論理和

6. システムフラグ一覧

本機では、ブール型の変数として「システムフラグ」を使用できます。システムフラグは条件を満たした場合に「1（真）」、満たしていない場合に「0（偽）」の値が自動的に代入され、いつでも参照することができます。

6.1 JR3000/JC-3 シリーズ

No.	識別子	内容	1（真）の条件
01	#FisCOM1	COM1 受信データの有無	有り
02	#FItCOM1	COM1 受信データ比較命令（cmpCOM）結果	定数 > 受信データ
03	#FeqCOM1	COM1 受信データ比較命令（cmpCOM）結果	定数 = 受信データ
04	#FgtCOM1	COM1 受信データ比較命令（cmpCOM）結果	定数 < 受信データ
05	#FtimeOutCOM1	COM1 受信データ比較命令（cmpCOM）タイムアウト	タイムアウト発生
06	#FisCOM2	COM2 受信データの有無	有り
07	#FItCOM2	COM2 受信データ比較命令（cmpCOM）結果	定数 > 受信データ
08	#FeqCOM2	COM2 受信データ比較命令（cmpCOM）結果	定数 = 受信データ
09	#FgtCOM2	COM2 受信データ比較命令（cmpCOM）結果	定数 < 受信データ
10	#FtimeOutCOM2	COM2 受信データ比較命令（cmpCOM）タイムアウト	タイムアウト発生
11	#FisCOM3	COM3 受信データの有無	有り
12	#FItCOM3	COM3 受信データ比較命令（cmpCOM）結果	定数 > 受信データ
13	#FeqCOM3	COM3 受信データ比較命令（cmpCOM）結果	定数 = 受信データ
14	#FgtCOM3	COM3 受信データ比較命令（cmpCOM）結果	定数 < 受信データ
15	#FtimeOutCOM3	COM3 受信データ比較命令（cmpCOM）タイムアウト	タイムアウト発生
30	#FinitMecError	メカ初期化命令エラーの状態	メカ初期化エラー
31	#FcameraError	カメラ補正の取り込みエラーの状態	エラー発生
32	#FtakeZError	Z 高さ補正（takeZWadj）取り込みエラーの状態	エラー発生
33	#FIMoveOutOfRange	相対移動命令の範囲状態	範囲外
34	#FIMoveStop	相対移動命令の条件停止状態	停止条件により停止
35	#FcheckPosError	位置ずれ検出命令結果	位置ずれエラー発生
36	#FdataInBCDError	dataInBCD 命令のエラー状態	エラー発生
37	#FmultiWadjValError	読出しカウンタで示されたワーク補正量取得時のエラー有無	エラー発生
60	#FstartSW	スタート/ストップスイッチ	ON（押されている）
61	#FincSW	プログラム選択キー（+）	ON（押されている）

No.	識別子	内容	1（真）の条件
62	#FdecSW	プログラム選択キー（-）	ON（押されている）
63	#FemgSW	EMG 直接入力	ON（非常停止ボタンが押されている）
64	#Fios	IOS 直接入力	回路オープン（切れる）
71	#Fsensor1	X 初期位置検出センサ	遮蔽
72	#Fsensor2	Y 初期位置検出センサ	遮蔽
73	#Fsensor3	Z 初期位置検出センサ	遮蔽
74	#Fsensor4	R 初期位置検出センサ	遮蔽
76	#Fdrvoz1	X ドライバゼロ相	ON
77	#Fdrvoz2	Y ドライバゼロ相	ON
78	#Fdrvoz3	Z ドライバゼロ相	ON
79	#Fdrvoz4	R ドライバゼロ相	ON
80	#FpurgeSW	捨て打ちスイッチ	ON（押されている）
81	#FdspRunning	塗布実行中	塗布実行中
82	#FdspDevRespError	塗布装置応答信号エラー発生状態	エラー発生
91	#FenableSW	イネーブルスイッチ	ON（押されている）
94	#FmotorPower	モータ動力状態	ON
95	#Finitialize1*	X 軸のメカ初期化状態	完了している
96	#Finitialize2*	Y 軸のメカ初期化状態	完了している
97	#Finitialize3*	Z 軸のメカ初期化状態	完了している
98	#Finitialize4*	R 軸のメカ初期化状態	完了している
111	#FoptionSW1	オプションスイッチ 1	ON（押されている）
112	#FoptionSW2	オプションスイッチ 2	ON（押されている）
113	#FoptionSW3	オプションスイッチ 3	ON（押されている）
300	#FisEther1	Ether1 受信データ有無	データ有り
301	#FltEther1	Ether1 受信データ比較結果	定数 > 受信データ
302	#FeqEther1	Ether1 受信データ比較結果	定数 = 受信データ
303	#FgtEther1	Ether1 受信データ比較結果	定数 < 受信データ
304	#FtimeOutEther1	Ether1 受信データ比較結果	タイムアウト発生
305	#FconnectEther1	Ether1 接続状態	接続中
306	#FisEther2	Ether2 受信データ有無	データ有り
307	#FltEther2	Ether2 受信データ比較結果	定数 > 受信データ
308	#FeqEther2	Ether2 受信データ比較結果	定数 = 受信データ
309	#FgtEther2	Ether2 受信データ比較結果	定数 < 受信データ
310	#FtimeOutEther2	Ether2 受信データ比較結果	タイムアウト発生
311	#FconnectEther2	Ether2 接続状態	接続中
312	#FisEther3	Ether3 受信データ有無	データ有り
313	#FltEther3	Ether3 受信データ比較結果	定数 > 受信データ
314	#FeqEther3	Ether3 受信データ比較結果	定数 = 受信データ

No.	識別子	内容	1（真）の条件
315	#FgtEther3	Ether3 受信データ比較結果	定数 < 受信データ
316	#FtimeOutEther3	Ether3 受信データ比較結果	タイムアウト発生
317	#FconnectEther3	Ether3 接続状態	接続中

* #Finitialize1 ～ 4 の仕様と注意点

- 電源投入直後は全軸のフラグが偽（0）です。
- 非常停止が発生すると、全軸のフラグが偽（0）になります。
- すでにメカ初期化を実行済かつフラグが真（1）の状態で、再度メカ初期化を実行した場合は、メカ初期化の動作が始まる直前にフラグが偽（0）になります。
- Z 軸と R 軸は各軸の初期化動作が完了した時点で、各軸のフラグが真（1）になります。
- X 軸と Y 軸の注意
 - [メカ初期化順] の設定が [XY 同時] の場合、X 軸と Y 軸の両方の初期化動作が完了した時点で 95 と 96 が同時に真（1）になります。例えば、X 軸の初期化動作が先に完了し、まだ Y 軸が初期化動作を行っている最中の場合、96 だけでなく 95 も偽（0）です。
 - [メカ初期化順] が [X 先 Y 後] または [Y 先 X 後] の場合、X 軸と Y 軸のそれぞれの初期化動作が完了した時点で、それぞれのシステムフラグが真（1）になります。
 - ポイント作業命令 initMec で 1 軸のメカ初期化を行った場合は、その軸のフラグのみが変化します。例えば、X 軸のみのメカ初期化の場合、メカ初期化順に関わらず X 軸のフラグのみが変化し Y 軸のフラグは変化しません。
- メカ初期化の手段・状況に関わらず、フラグの挙動は同じ仕様です。
 - 手段：スイッチボックス、I/O-SYS、フィールドバス、通信コマンド、ポイント作業命令、…等
 - 状況：電源投入後のメカ初期化、運転待機中のメカ初期化、ティーチングモード、…等

6.2 JS3 シリーズ

No	識別子	内容	1 (真) の条件
01	#FisCOM1	COM1 受信データの有無	有り
02	#FltCOM1	COM1 受信データ比較命令 (cmpCOM) 結果	定数 > 受信データ
03	#FeqCOM1	COM1 受信データ比較命令 (cmpCOM) 結果	定数 = 受信データ
04	#FgtCOM1	COM1 受信データ比較命令 (cmpCOM) 結果	定数 < 受信データ
05	#FtimeOutCOM1	COM1 受信データ比較命令 (cmpCOM) タイムアウト	タイムアウト発生
06	#FisCOM2	COM2 受信データの有無	有り
07	#FltCOM2	COM2 受信データ比較命令 (cmpCOM) 結果	定数 > 受信データ
08	#FeqCOM2	COM2 受信データ比較命令 (cmpCOM) 結果	定数 = 受信データ
09	#FgtCOM2	COM2 受信データ比較命令 (cmpCOM) 結果	定数 < 受信データ
10	#FtimeOutCOM2	COM2 受信データ比較命令 (cmpCOM) タイムアウト	タイムアウト発生
31	#FcameraError	カメラ補正の取り込みエラーの状態	エラー発生
32	#FtakeZError	Z 高さ補正 (takeZWadj) 取り込みエラーの状態	エラー発生
33	#FIMoveOutOfRange	相対移動命令の範囲状態	範囲外
34	#FIMoveStop	相対移動命令の条件停止状態	停止条件により停止
36	#FdataInBCDError	dataInBCD 命令のエラー状態	エラー発生
37	#FmultiWadjValError	読みしカウンタで示されたワーク補正量取得時のエラー有無	エラー発生
63	#FemgSW	EMG 直接入力	ON (非常停止ボタンが押されている)
66	#FmponSW	動力 ON スイッチ	ON (押されている)
68	#FmdSW1	セレクトスイッチ	ON
91	#FenableSW	イネーブルスイッチ	ON (押されている)
94	#FmotorPower	モータ動力状態	ON
121	#FsvReady1	J1/X サーボレディ	サーボ準備完了している
122	#FsvReady2	J2/Y サーボレディ	サーボ準備完了している
123	#FsvReady3	J3/Z サーボレディ	サーボ準備完了している
124	#FsvReady4	J4/R サーボレディ	サーボ準備完了している
126	#FsvAlarm1	J1/X サーボアラーム	サーボドライバ異常
127	#FsvAlarm2	J2/Y サーボアラーム	サーボドライバ異常
128	#FsvAlarm3	J3/Z サーボアラーム	サーボドライバ異常
129	#FsvAlarm4	J4/R サーボアラーム	サーボドライバ異常
131	#FsvPos1	J1/X サーボ位置決め完了	位置決め完了している
132	#FsvPos2	J2/Y サーボ位置決め完了	位置決め完了している
133	#FsvPos3	J3/Z サーボ位置決め完了	位置決め完了している
134	#FsvPos4	J4/R サーボ位置決め完了	位置決め完了している
136	#FencOz1	J1/X エンコーダゼロ相	遮蔽

No	識別子	内容	1（真）の条件
137	#FencOz2	J2/Y エンコーダゼロ相	遮蔽
138	#FencOz3	J3/Z エンコーダゼロ相	遮蔽
139	#FencOz4	J4/R エンコーダゼロ相	遮蔽
300	#FisEther1	Ether1 受信データ有無	データ有り
301	#FltEther1	Ether1 受信データ比較結果	定数 > 受信データ
302	#FeqEther1	Ether1 受信データ比較結果	定数 = 受信データ
303	#FgtEther1	Ether1 受信データ比較結果	定数 < 受信データ
304	#FtimeOutEther1	Ether1 受信データ比較結果	タイムアウト発生
305	#FconnectEther1	Ether1 接続状態	接続中
306	#FisEther2	Ether2 受信データ有無	データ有り
307	#FltEther2	Ether2 受信データ比較結果	定数 > 受信データ
308	#FeqEther2	Ether2 受信データ比較結果	定数 = 受信データ
309	#FgtEther2	Ether2 受信データ比較結果	定数 < 受信データ
310	#FtimeOutEther2	Ether2 受信データ比較結果	タイムアウト発生
311	#FconnectEther2	Ether2 接続状態	接続中
312	#FisEther3	Ether3 受信データ有無	データ有り
313	#FltEther3	Ether3 受信データ比較結果	定数 > 受信データ
314	#FeqEther3	Ether3 受信データ比較結果	定数 = 受信データ
315	#FgtEther3	Ether3 受信データ比較結果	定数 < 受信データ
316	#FtimeOutEther3	Ether3 受信データ比較結果	タイムアウト発生
317	#FconnectEther3	Ether3 接続状態	接続中
400	#FwarningBattery	ロボットメンテナンス機能：バッテリーワーニング発生有無	ワーニング発生有り
401	#FwarningGrease1	ロボットメンテナンス機能：J1 軸グリースワーニング発生有無	ワーニング発生有り
402	#FwarningGrease2	ロボットメンテナンス機能：J2 軸グリースワーニング発生有無	ワーニング発生有り
403	#FwarningGrease3	ロボットメンテナンス機能：J3 軸グリースワーニング発生有無	ワーニング発生有り
404	#FwarningGrease4	ロボットメンテナンス機能：J4 軸グリースワーニング発生有無	ワーニング発生有り
405	#FwarningBelt1	ロボットメンテナンス機能：J1 軸ベルトワーニング発生有無	ワーニング発生有り
406	#FwarningBelt2	ロボットメンテナンス機能：J2 軸ベルトワーニング発生有無	ワーニング発生有り
407	#FwarningBelt3	ロボットメンテナンス機能：J3 軸ベルトワーニング発生有無	ワーニング発生有り
408	#FwarningBelt4	ロボットメンテナンス機能：J4 軸ベルトワーニング発生有無	ワーニング発生有り

7. 変数

7.1 組み込み変数

7.1.1 自由変数

■ #mv, #mkv, #nv, #nkv, #sv, #skv

変数とは、値を入れておく入れ物です。本機では自由に使える変数として、以下の変数が用意されています。以下の変数を使用する場合には、変数宣言は必要ありません。

	識別子	
自由変数	#mv (1 ~ 99)	ブール型変数
	#mkv (1 ~ 99)	ブール型変数 (キープ変数)
	#nv (1 ~ 99)	数値型変数
	#nkv (1 ~ 99)	数値型変数 (キープ変数)
	#sv (1 ~ 99)	文字列型変数
	#skv (1 ~ 99)	文字列型変数 (キープ変数)

注) キープ変数は、ロボットの電源を OFF しても値を保持する変数です。

■ #mv (1 ~ 99) , #mkv (1 ~ 99)

ブール型変数です。ブール型変数とは、1 ビットの 0/1 の値を格納できる変数です。条件演算式 (Id,Idi)、代入式 (let) のパラメータとして使用できます。

注) ブール型の自由変数(#mv (1 ~ 99), #mkv (1 ~ 99))は PLC プログラムでも使用可能です。

■ #nv (1 ~ 99) , #nkv (1 ~ 99)

数値型変数です。8 バイト実数型 (double type) で、代入式 (let) のパラメータとして使用できます。

■ #sv (1 ~ 99) , #skv (1 ~ 99)

文字列型変数です。255 バイトまで格納できます。代入式 (let) のパラメータとして使用する場合は、「=」で代入、「&」で結合することができます。

7.1.2 入力変数

■ #sysIn1 ～ , #genIn1 ～ , #handIn1 ～ , #fbIn1 ～

入力変数は、参照のみ可能な変数です。値を書き込むことはできません。

I/O-SYS, I/O-1, I/O-H, フィールドバスの各入力ピンに対応しており、ON 信号を受信している場合、「1 (真)」となります。

種類	識別子 (JR3000/JC-3 シリーズ)	識別子 (JS3 シリーズ)	コネクタ
入力変数	#sysIn1 ～ 16, #sysIn (1 ～ 16)	#sysIn1 ～ 15, #sysIn (1 ～ 15)	I/O-SYS
	#genIn1 ～ 8, #genIn (1 ～ 8)	#genIn1 ～ 18, #genIn (1 ～ 18)	I/O-1
	-	#handIn1 ～ 8, #handIn (1 ～ 8)	I/O-H
	#fbIn (0x1000 ～ 0x17FF, 1 ～ 32)	#fbIn (0x1000 ～ 0x17FF, 1 ～ 32)	フィールドバス

#sysIn1 ～ 16 または #sysIn1 ～ 15 (I/O-SYS) には、あらかじめ機能が割り付けられているものもあります。

例 #sysIn1: スタート信号 (ON 信号受信時、運転スタート)

#sysIn1 ～ 16 または #sysIn1 ～ 15 (I/O-SYS) を割り付けられている機能以外で使いたい場合は、全プログラム共通設定の設定をフリーに切り替えてください (全プログラム共通設定 → I/O 設定 → I/O-SYS 機能割付)。

#fbIn1000 ～ 17FF (フィールドバス) を割り付けられている機能以外で使いたい場合は、全プログラム共通設定の設定をフリーに切り替えてください (全プログラム共通設定 → I/O 設定 → フィールドバス機能割付)。

注)

- #handIn1 ～ 8 は JS3 シリーズのみ有効です。JR3000/JC-3 シリーズには、I/O-H はありません。
- I/O-SYS およびフィールドバスのあらかじめ割り付けられている機能についての詳細は、取扱説明書『外部制御編 (I/O/ フィールドバス, 入出力)』をご参照ください。

フィールドバスの #fbIn 変数の記述方法は、以下の通りとなります。

- #fbIn (I/O アドレス, ビット幅)

注) I/O アドレスは、0x に続けて 4 桁の 16 進数で指定してください。

指定できるビット幅は、1 ～ 32 ビットで、最大 2 ワード (4 バイト) 指定することができます。ただし、フィールドバスの設定により、フィールドバス入力エリア範囲外になった場合、範囲外のビットは切り捨てになります。

7.1.3 出力変数

■ #sysOut1 ～ , #genOut1 ～ , #handOut1 ～ , #fbOut1 ～

出力変数はブール型変数（フィールドバスは数値型として使用も可）です。

I/O-SYS, I/O-1, I/O-H, フィールドバスの各出力ピンに対応しており、ON 信号を出力している場合、「1（真）」となります。

種類	識別子 (JR3000/JC-3 シリーズ)	識別子 (JS3 シリーズ)	コネクタ
出力変数	#sysOut1 ～ 16, #sysOut (1 ～ 16)	#sysOut1 ～ 14, #sysOut (1 ～ 14)	I/O-SYS
	#genOut1 ～ 8, #genOut (1 ～ 8)	#genOut1 ～ 18, #genOut (1 ～ 18)	I/O-1
	-	#handOut1 ～ 8, #handOut (1 ～ 8)	I/O-H
	#fbOut (0x1800 ～ 0x1FFF, 1 ～ 32)	#fbOut (0x1800 ～ 0x1FFF, 1 ～ 32)	フィールドバス

#sysOut1 ～ 16 または #sysOut1 ～ 15 (I/O-SYS) には、あらかじめ機能が割り付けられているものもあります。

例 #sysOut1: スタート準備完了 (ON 信号時、運転スタート可能)

#sysOut1 ～ 16 または #sysOut1 ～ 15 (I/O-SYS) を割り付けられている機能以外で使用したい場合は、全プログラム共通設定の設定をフリーに切り替えてください (全プログラム共通設定 → I/O 設定 → I/O-SYS 機能割付)。

#fbOut1800 ～ 1FFF (フィールドバス) を割り付けられている機能以外で使用したい場合は、全プログラム共通設定の設定をフリーに切り替えてください (全プログラム共通設定 → I/O 設定 → フィールドバス機能割付)。

注)

- JR3000/JC-3 シリーズには、I/O-H はありません。#handOut1 ～ 8 は JS3 シリーズのみ有効です。
- I/O-SYS およびフィールドバスのあらかじめ割り付けられている機能についての詳細は、取扱説明書『外部制御編 (I/O/ フィールドバス, 入出力)』をご参照ください。

フィールドバスの #fbOut 変数の記述方法は、以下の通りとなります。

- #fbOut (I/O アドレス, ビット幅)

注) I/O アドレスは、0x に続けて 4 桁の 16 進数で指定してください。

指定できるビット幅は 1 ～ 32 ビットで、最大 2 ワード (4 バイト) 指定することができます。ただしフィールドバスの設定によりフィールドバス出力エリア範囲外になった場合、範囲外のビットは切り捨てになります。

7.1.4 ダウンタイマー

■ #downTimer1 ～ 10

数値型変数です。値を代入 (let) すると、自動的に msec 単位でダウンカウントを開始します。ダウンカウントの途中で値を代入することもできます。代入できる値の最大値は 2, 147, 483, 647 (msec) です。

種類	識別子	概要
特殊変数	#downTimer1 ～ 10	値を代入すると自動的にダウンカウント開始 (msec 単位)

例 以下のポイント作業データを作成し、CP 駆動中作業としてポイントに設定すると、CP 駆動中 0.5 秒毎に COM2 へ "CR" を出力します。

<pre>if Id #downTimer1 == 0 then eoutCOM port2, "%0D" #downTimer1 = 500</pre>	もし #downTimer1 = 0 であれば COM2 から "CR" の 16 進コードを出力し、 #downTimer1 に 500 (0.5 秒) を代入する。
---	--

注) ただし、この例の場合は事前に (ポイント作業等で) #downTimer1 に値を代入しておく必要があります。

7.1.5 移動後作業開始高さ

■ #jobStartHight

PTP 駆動の到着時の Z (J3) 軸位置を変更する変数です。

ポイントデータに設定された Z (J3) 座標に対して、実際に PTP 駆動を行ったときの到着時の Z (J3) 軸位置を変更することができます。Z (J3) 軸位置を高い位置に変更すれば、PTP 駆動後の移動後作業の実行タイミングを早めることができます。

移動前作業において let 命令で「#jobStartHight」に値を代入すると、その後に行われる PTP 駆動の到着時の Z 位置が変更されます。

移動後作業、移動中作業で「#jobStartHight」に値を代入しても無効であり、駆動動作には反映されません。

「#jobStartHight」に代入した値が正の値の場合、ポイントデータの Z 座標より高い位置となります。負の値の場合、ポイントデータの Z 座標より低い位置となります。

変更された Z (J3) 軸位置に PTP 駆動で到着した時点で移動後作業が実行され、次のポイントに進みます。

種類	識別子	概要
特殊変数	#jobStartHight	PTP 駆動の到着時の Z (J3) 軸位置を変更する変数

■ 例

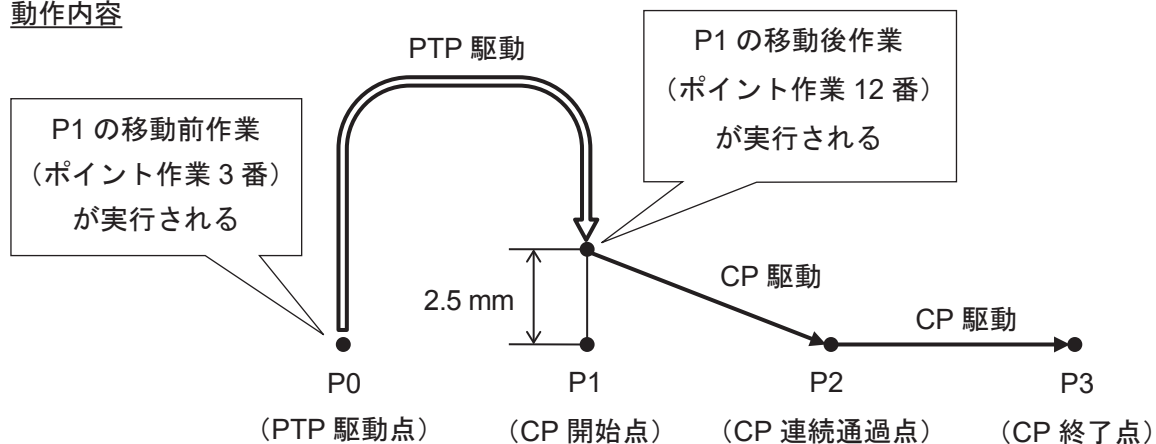
P1 への PTP 駆動の到着位置を、P1 に設定されている Z 座標より 2.5 mm だけ高い位置に変更する例です。

ティーチング例

	P0（作業原点）	P1	P2	P3
種別	PTP 駆動点	CP 開始点	CP 連続通過点	CP 終了点
移動前作業番号	-	3	-	-
移動後作業番号	-	12	-	-

ポイント作業 3 番	ポイント作業 12 番
let #jobStartHight = 2.5	任意の作業内容

動作内容



P1 に設定されている Z 座標より 2.5 mm 高い位置で P1 の移動後作業が実行され、その位置から P2 に進みます。

P1 に設定されている本来の Z 座標まで Z（J3）軸を下降させたい場合は、P1 の移動後作業にて downZ 命令等の駆動命令を利用してください。

7.1.6 パレット

■ #palletFlag (1 ~ 100) , #palletCount (1 ~ 100)

#palletCount (1 ~ 100) は数値型変数、#palletFlag (1 ~ 100) はブール型変数です。

ポイント属性データ [パレット] の、パレットカウンタの値、パレットフラグの値（パレットカウンタが最大の場合、1（真））を保持しています。

この変数を使用すれば、パレットの途中で次ポイント等へ移動したり、指定番目のパレットを飛ばしたりすることができます。

種類	識別子	概要
パレット	#palletFlag (1 ~ 100)	パレットフラグ（パレット 1 ~ 100 に対応）
	#palletCount (1 ~ 100)	パレットカウンタ（パレット 1 ~ 100 に対応）

注)

- ・ JS3 シリーズのピック & プレイス仕様をお使いの場合、パレットフラグ、パレットカウンタの 41 ~ 100 は予約されておりますので、使用しないでください。
- ・ #palletCount (1 ~ 100) に、カウントが最大になる値を代入しても、#palletFlag (1 ~ 100) が「1（真）」になることはありません。

例 指定した番目を抜かしてパレット作業をする。（JS3 シリーズの場合）

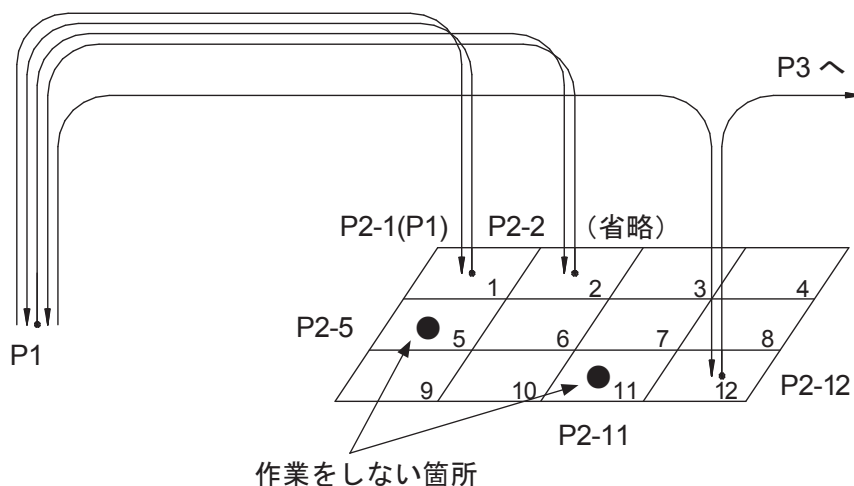
パレット種類 : 平面パレット
 カウント制御種類 : ポイント作業制御
 パレット行数 : 4
 パレット列数 : 3
 P0、Pa、Pb : 任意の値

P1 で荷物を掴み、パレット(P2 に設定)に置いて行き、パレットが最大になったら次ポイント(P3)へ進むものとします。ただし、パレット中の 2 点(P2-5, P2-11)には荷物を置かないものとします。

パレット番号は「3」とし、ツールは、以下の設定で接続されているものとします。

「荷物を掴む」（ピックアップ）: #handOut1=ON

「荷物を置く」（プレイス）: #handOut1=OFF



P1 に設定するポイント作業データ

```
set #handOut1
```

荷物を掴みます。

P2 に設定するポイント作業データ

```
if
  Id #palletCount (3) == 5
  or #palletCount (3) == 11
then
  loopPallet 3,2
else
  reset #handOut1
  loopPallet 3,1
endif
```

#palletCount (3) が 5 (P2-5)、11 (P2-11) の場合、パレット 3 のカウンタに 1 加え、P2 の次の作業箇所へ移動します。
それ以外の場合、荷物を置き（離し）ます。
パレット 3 のカウンタに 1 加え、カウンタが最大になったら、次の命令（この場合、次の命令がないので、ポイント作業終了）へ進みます。
最大値に満たなければ P1 へ移動します。

7.1.7 ワーク補正

- #workAdj_X, #workAdj_Y, #workAdj_Z, #workAdj_R, #workAdj_Rotation, #mulWorkAdj_Wrt_Cam, #mulWorkAdj_Wrt_Zadj, #mulWorkAdj_Read, #mulWorkAdj_Num

数値型変数です。ポイント属性データ「ワーク補正」の各軸方向の補正量、および回転補正量を保持しています。

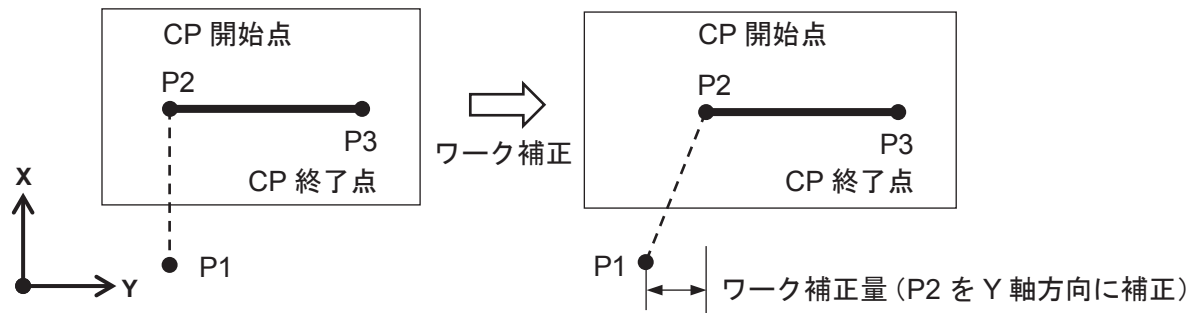
種類	識別子	概要
ワーク補正	#workAdj_X (1 ~ 3000)	ワーク補正の X 方向補正量(ワーク補正 1 ~ 3000 に対応)
	#workAdj_Y (1 ~ 3000)	ワーク補正の Y 方向補正量(ワーク補正 1 ~ 3000 に対応)
	#workAdj_Z (1 ~ 3000)	ワーク補正の Z 方向補正量(ワーク補正 1 ~ 3000 に対応)
	#workAdj_R (1 ~ 3000)	ワーク補正の R 方向補正量(ワーク補正 1 ~ 3000 に対応)
	#workAdj_Rotation (1 ~ 3000)	ワーク補正の回転角度補正量(ワーク補正 1 ~ 3000 に対応)
ワーク補正 (カウンタ付 CCD カメラ補正)	#mulWorkAdj_Wrt_Cam	カウンタ付 CCD カメラ補正の、ワーク補正カウンタの書き込みカウンタ値
	#mulWorkAdj_Wrt_Zadj	カウンタ付 CCD カメラ補正の、Z 補正カウンタの書き込みカウンタ値
	#mulWorkAdj_Read	カウンタ付 CCD カメラ補正の、ワーク補正カウンタの読み出しカウンタ値
	#mulWorkAdj_Num	カウンタ付 CCD カメラ補正の、ワーク補正カウンタのワーク補正番号

例 P2-P3 間を線塗布するものとします。

P1 で、COM に接続されたセンサからワーク補正量（ワークのズレた値）を受信するものとします。

ワーク補正番号は「6」とし、ツールは、以下の設定で接続されているものとします。

「塗布開始」：#genOut1=ON「塗布終了」：#genOut1=OFF



P1 に設定するポイント作業データ

```
declare string hosei
setWTCOM port1,1000
inCOM hosei,port1,10
let #workAdj_Y (6) = val (hosei)
```

文字列型ローカル変数「hosei」の宣言。
 受信待ち時間 1.0 sec（省略時は 0.1 sec 待機）
 COM1 からワーク補正量を「hosei」に受信。
 「hosei」の値を数値型に変換して #workAdj_Y (6) へ代入。
 (#workAdj_Y (6) = ワーク補正 6 の Y 方向補正量)

P2（[ワーク補正] 設定ポイント）に設定するポイント作業データ

```
set #genOut1
```

塗布開始

P3 に設定するポイント作業データ

```
reset #genOut1
```

塗布終了

注）[ワーク補正] は CP 開始点に設定すると、CP 終了点に到着するまで有効です。

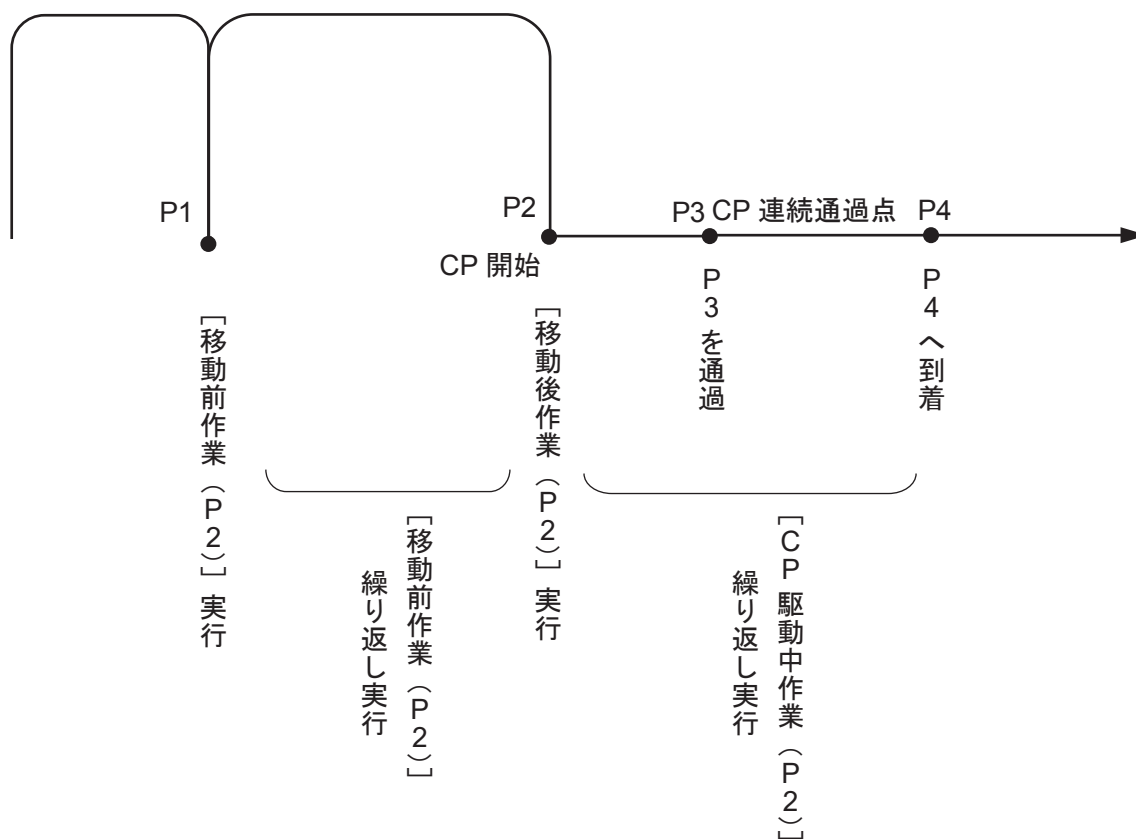
7.1.8 ポイント座標

■ #point_X, #point_Y, #point_Z, #point_R, #point_TagCode

実行中のポイントの各座標値および、タグコード値を保持する変数です。実行中のポイントとは、この変数を含むポイント作業データが設定されているポイントを指します。移動前作業、移動中作業、CP 駆動中作業にこの変数を含むポイント作業データが設定されている場合、ツールセンターポイントの現在位置とこの変数の値とは異なります。

下図の場合、P2 に設定されている移動前作業は、P1 のポイントで実行しますが、移動前作業に設定されているポイント作業データにこの変数が含まれる場合、保持する値は P2 の座標値です。

また、この変数はポイントの持つもともとの座標値を保持します。[ワーク補正] や「#jobStartHight」を使用している場合でも、値が変更されることはありません。



種類	識別子	概要
現在 ポイント 座標	#point_X	実行中のポイントの X 座標値
	#point_Y	実行中のポイントの Y 座標値
	#point_Z	実行中のポイントの Z 座標値
	#point_R	実行中のポイントの R 座標値
	#point_TagCode	実行中のポイントのタグコード値

7.1.9 指定ポイント座標

■ #P_X, #P_Y, #P_Z, #P_R, #P_TagCode

現在プログラム内の指定ポイントの各座標値および、タグコード値を保持する変数です。
この変数はポイントの持つもともとの座標値を保持します。[ワーク補正]や「#jobStartHight」
を使用している場合でも、値が変更されることはありません。

種類	識別子	概要
指定 ポイント 座標	#P_X (1 ~ 最終ポイント数)	現在プログラム内の指定ポイントの X 座標値
	#P_Y (1 ~ 最終ポイント数)	現在プログラム内の指定ポイントの Y 座標値
	#P_Z (1 ~ 最終ポイント数)	現在プログラム内の指定ポイントの Z 座標値
	#P_R (1 ~ 最終ポイント数)	現在プログラム内の指定ポイントの R 座標値
	#P_TagCode (1 ~ 最終ポイント数)	現在プログラム内の指定ポイントのタグコード値

7.1.10 指定プログラム指定ポイント座標

■ #prog_P_X, #prog_P_Y, #prog_P_Z, #prog_P_R, #prog_P_TagCode

指定プログラムの指定ポイントの各座標値および、タグコード値を保持する変数です。
この変数はポイントの持つもともとの座標値を保持します。[ワーク補正]や「#jobStartHight」
を使用している場合でも、値が変更されることはありません。

種類	識別子	概要
指定 プログラム 指定 ポイント 座標	#prog_P_X (1 ~ 999, 1 ~ 最終ポイント数)	指定プログラム指定ポイントの X 座標値
	#prog_P_Y (1 ~ 999, 1 ~ 最終ポイント数)	指定プログラム指定ポイントの Y 座標値
	#prog_P_Z (1 ~ 999, 1 ~ 最終ポイント数)	指定プログラム指定ポイントの Z 座標値
	#prog_P_R (1 ~ 999, 1 ~ 最終ポイント数)	指定プログラム指定ポイントの R 座標値
	#prog_P_TagCode (1 ~ 999, 1 ~ 最終ポイント数)	指定プログラム指定ポイントのタグコード値

7.1.11 指定プログラムツールデータ

- #prog_ToolData_X, #prog_ToolData_Y, #prog_ToolData_DeltaZ, #prog_ToolData_EachCommon,
指定プログラムのツールデータの各設定距離および、指定したツールデータのプログラム設定状態を保持する変数です。() 内の引数を 0 に指定した場合、全プログラム共通設定のツールデータが指定されます。

種類	識別子	概要
指定 プログラム ツール データ	#prog_ToolData_X (0 ~ 999)	指定プログラムのツールデータの TCP-X
	#prog_ToolData_Y (0 ~ 999)	指定プログラムのツールデータの TCP-Y
	#prog_ToolData_DeltaZ (0 ~ 999)	指定プログラムのツールデータの TCP- ΔZ
	#prog_ToolData_EachCommon (1) ~ (999)	指定したツールデータのプログラム設定状態

7.1.12 ポイント属性データ番号

- #point_WorkAdjNum, #point_ToolNum, #point_PalletNum
実行中のポイントに設定されたワーク補正番号、切り替えツール番号、パレット番号を保持する変数です。

種類	識別子	概要
ポイント 属性 データ番号	#point_WorkAdjNum	実行中ポイントに設定されたワーク補正番号
	#point_ToolNum	実行中ポイントに設定された切り替えツール番号
	#point_PalletNum	実行中ポイントに設定されたパレット番号

7.2 ユーザ定義変数

7.2.1 グローバル変数・キープ変数

数値型、文字列型の選択ができ、3次元までの配列にすることが可能です。

ローカル変数と反対に、どのプログラム、どのポイントからでも参照できる変数をグローバル変数と呼びます。ポイント作業データ内で宣言して使用するローカル変数以外の変数は、すべてグローバル変数です。

キープ変数は電源を OFF しても値を保持している変数です。ここで定義するキープ変数は、値を保持できるグローバル変数です。

TP 所有者ありの場合は、カスタマイズモードでアカウントにログインしてください。

[変数定義]

[グローバル変数定義]

[キープ変数定義]

所有者無しの場合は、ティーチングモードから選択してください。

[変数・関数・エイリアス設定]

[グローバル変数定義]

[キープ変数定義]

PC [データ] → [グローバル変数]
→ [キープ変数]

設定項目	タイプ	内容
識別子	識別子 文字列	ポイント作業データの式内で、ここで規定した識別子を使用します。 新規作成時に指定。変更できません。
保護モード	選択	この定義の扱いに関する設定 (1) 制限無し (2) 公開 (3) 保護 (4) 私的
変数型	選択	変数の型として2つから選択設定します。 (1) 数値型 (8バイト実数型 (double type)) (2) 文字列型 (255バイトまで格納できます)
次元	選択	変数は最大3次元の配列にすることができます。変数の次元を次の4つから選択設定します。 (1) 単体 (2) 1次元配列 (3) 2次元配列 (4) 3次元配列
1次元要素数	数値	変数が1次元以上の配列の場合、1次元の配列要素数。
2次元要素数	数値	変数が2次元以上の配列の場合、2次元の配列要素数。
3次元要素数	数値	変数が3次元の配列の場合、3次元の配列要素数。

8. 関数

本機では、ロボットの機能として組み込まれている組み込み関数と、自由に定義できるユーザ定義関数が使用できます。

8.1 組み込み関数

8.1.1 ロボット系関数

ロボット系の組み込み関数として、以下が使用できます。

型	識別子	概 要
num	currentMainProgNumber()	現在実行中のメインプログラム番号
num	currentSubProgNumber()	現在実行中のサブプログラム番号
num	currentPointNumber()	現在実行中のポイント番号
num	currentArmX()	現在の X 座標、[mm] 単位
num	currentArmY()	現在の Y 座標、[mm] 単位
num	currentArmZ()	現在の Z 座標、[mm] 単位
num	currentArmR()	現在の R 座標、[度] 単位
num	currentArmH()	現在の座標の手系（1: 右手系 -1: 左手系） 注）JR3000/JC-3 シリーズでは、1（右手系）に固定されています。
num	currentCmdArmX()	現在の指令 X 座標、[mm] 単位
num	currentCmdArmY()	現在の指令 Y 座標、[mm] 単位
num	currentCmdArmZ()	現在の指令 Z 座標、[mm] 単位
num	currentCmdArmR()	現在の指令 R 座標、[度] 単位
num	numCOM(port#)	COM 受信ポートのデータバイト数
num	moveAPTP (num a, num b, num X, num Y, num Z, num R)	絶対位置指定 PTP 駆動関数。指定した位置まで PTP 駆動で移動します。
num	moveAPTP6 (num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)	
num	moveRPTP (num a, num b, num X, num Y, num Z, num R)	相対位置指定 PTP 駆動関数。現在の位置から指定した距離だけ離れた位置まで PTP 駆動で移動します。
num	moveRPTP6 (num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)	
num	isConditionData(num n)	条件データ有無（1: 有り、0: 無し） 指定した番号の条件データがあるかどうか

型	識別子	概 要
str	strCenterLCD(string s)	ティーチングペンダント LCD 表示でセンタリング（中央揃え）になるように文字列を調整します。
str	strRightLCD(string s)	ティーチングペンダント LCD 表示で右寄せになるように文字列を調整します。
str	strPlusRLCD(string a, string b)	ティーチングペンダント LCD 表示用、右優先文字列の左右振り分け結合
str	strPlusLLCD(string a, string b)	ティーチングペンダント LCD 表示用、左優先文字列の左右振り分け結合
num	getSystemPTPmoveTime()	移動中作業でのみ有効 現在の PTP 駆動に必要な時間、[秒] 単位
num	getSystemPTPrestTime()	移動中作業でのみ有効 現在の PTP 駆動の残り時間（到達までの時間）、[秒] 単位
num	Pause()	一時停止 ただし、JR C-Points II により、Pause 停止切替を「有効」に設定する必要あり カッコ内の引数は値参照実行時の pause 停止番号
str	getUserMessage(num x)	x で指定された番号のユーザ定義メッセージを取得します。
num	addPointSkip()	ポイント指定の欠品飛ばしを登録します。
num	delPointSkip()	ポイント指定の欠品飛ばしを解除します。
num	clearPointSkip()	ポイント指定の欠品飛ばしを全解除します。
num	addPalletSkip()	パレット指定の欠品飛ばしを登録します。
num	delPalletSkip()	パレット指定の欠品飛ばしを解除します。
num	clearPalletSkip()	パレット指定の欠品飛ばしを全解除します。
num	qCameraWadj(num X, num Y)	4 点の撮影によりワーク補正の補正値を算出します。

- currentMainProgNumber()
現在実行中のメインプログラム番号を保持しています。
- currentSubProgNumber()
現在実行中のサブプログラム番号を保持しています。サブプログラムを運転していない場合は、現在実行中のメインプログラム番号を保持します。
- currentPointNumber()
現在実行中のポイント番号を保持しています。作業原点の場合は「0」、
- currentArmX(), currentArmY(), currentArmZ()
現在の座標位置を保持しています。（絶対座標、[mm] 単位）

- `currentArmR()`
現在の R (J4) 軸の回転度数 (R 軸座標) を保持しています。(絶対座標、[度] 単位)
- `currentCmdArmX()`, `currentCmdArmY()`, `currentCmdArmZ()`
現在の座標指令位置を保持しています。(絶対座標、[mm] 単位)
- `currentCmdArmR()`
現在の R (J4) 軸の指令回転度数 (R 軸座標) を保持しています。(絶対座標、[度] 単位)
- `moveAFTP(num a, num b, num X, num Y, num Z, num R)`
- `moveAFTP6(num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)`
指定した位置まで PTP 駆動で移動します。
ただし、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

`num a` : PTP 駆動条件番号

0 の場合、プログラムデータの PTP 駆動条件に従います。

プログラム実行中でない場合、全プログラム共通設定に従います。

`num b` : 1 = 右手系、-1 = 左手系

JR3000/JC-3 シリーズでは、必ず 1 を指定してください。

`num X` : X (J1) 軸位置 (絶対座標、単位 [mm])

`num Y` : Y (J2) 軸位置 (絶対座標、単位 [mm])

`num Z` : Z (J3) 軸位置 (絶対座標、単位 [mm])

`num R` : R (J4) 軸位置 (絶対座標、単位 [度])

`num MT1` : MT1 軸位置 (絶対座標、任意単位)

`num MT2` : MT1 軸位置 (絶対座標、任意単位)

戻り値 : 0 = 移動が成功した

-1 = 移動が失敗した

戻り値 = -1 (移動が失敗) となる原因は次の通りです。

- a で指定された番号の PTP 駆動条件が存在しない
- CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでの実行
- 移動中作業または CP 駆動中作業での実行

例えば、`moveAFTP(3, 1, 100, 110, 50, 90)` の場合、PTP 駆動条件番号 3 の条件に従い、(JS3 シリーズの場合は右手系で) X=100 mm, Y=110 mm, Z=50 mm, R=90° の位置へ移動します。

- moveRPTP(num a, num b, num X, num Y, num Z, num R)
 - moveRPTP6(num a, num b, num X, num Y, num Z, num R, num MT1, num MT2)
- 現在の位置から指定した距離だけ離れた位置まで PTP 駆動で移動します。
ただし、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

num a : PTP 駆動条件番号

0 の場合、プログラムデータの PTP 駆動条件に従います。

プログラム実行中でない場合、全プログラム共通設定に従います。

num b : 1 = 右手系、-1 = 左手系

JR3000/JC-3 シリーズでは、必ず 1 を指定してください。

num X : X (J1) 軸距離 (相対距離、単位 [mm])

num Y : Y (J2) 軸距離 (相対距離、単位 [mm])

num Z : Z (J3) 軸距離 (相対距離、単位 [mm])

num R : R (J4) 軸距離 (相対距離、単位 [度])

num MT1 : MT1 軸位置 (相対距離、任意単位)

num MT2 : MT2 軸位置 (相対距離、任意単位)

戻り値 : 0 = 移動が成功した

-1 = 移動が失敗した

戻り値 = -1 (移動が失敗) となる原因は次の通りです。

- a で指定された番号の PTP 駆動条件が存在しない
- CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでの実行
- 移動中作業または CP 駆動中作業での実行

例えば、現在位置が X=50 mm, Y=50 mm, Z=10 mm, R=30° だった場合、moveRPTP(3, 1, 100, 110, 50, 90) を実行すると、X=150 mm, Y=160 mm, Z=60 mm, R=120° の位置へ移動します。

- numCOM(port#)

COM 受信ポートのデータバイト数。

- isConditionData(num n)

指定した番号の条件データがあるかどうか (1: 有り、0: 無し)。

- strCenterLCD(string s)

ティーチングペンダント LCD 表示で、センタリング (中央揃え) になるように文字列を調整します。

- strRightLCD(string s)

ティーチングペンダント LCD 表示で、右揃えになるように文字列を調整します (通常は左揃え)。

- strPlusRLCD(string a, string b)
ティーチングペンダント LCD 表示用で、結合させた文字列を左右に振り分けた場合に、文字数が 1 行を超える時、左側文字列の表示を消して右側文字列を表示させます。
- strPlusLLCD(string a, string b)
ティーチングペンダント LCD 表示用で、結合させた文字列を左右に振り分けた場合に、文字数が 1 行を超える時、右側文字列の表示を消して左側文字列を表示させます。
- getSystemPTPmoveTime()
現在の PTP 駆動に掛かる所要時間。移動中作業でのみ有効、[秒] 単位。
- getSystemPTPrestTime()
現在の PTP 駆動に掛かる所要時間の残り時間(到達までの時間)。移動中作業でのみ有効、[秒] 単位。
- Pause()
一時停止。指定した引数 (0 ~ 100) が、「値参照」実行時に「pause 停止番号」として使用されます。引数がない場合は式評価エラーとなります。
ただし、JR C-Points II により、Pause 停止切替を「有効」に設定する必要があります。
また、ロボットが駆動中の場合は「有効」に設定されていても無効となります。例えば CP 駆動中作業で「Pause」を使用した場合は、一時停止しません。一時停止を解除するには、JR C-Points II での操作が必要です。
- getUserMessage(num x)
x で指定された番号のユーザ定義メッセージを取得します。
取得されるメッセージ文字列は、ティーチング環境設定の表示言語設定と連動しています。
あらかじめ言語ごとに翻訳したメッセージ文字列をユーザ定義メッセージに登録しておく必要があります。
num x: ユーザ定義メッセージの番号
戻り値: メッセージ文字列
- addPointSkip(num a, num b)
ポイント指定の欠品飛ばしを登録します。
num a: プログラム番号を指定します。(1 ~ 999)
num b: ポイント番号を指定します。(1 ~ プログラムの最大数)

- `delPointSkip(num a, num b)`
ポイント指定の欠品飛ばしを解除します。
num a: プログラム番号を指定します。(1 ~ 999)
num b: ポイント番号を指定します。(1 ~ プログラムの最大数)
-1 を指定することで、num a で指定したプログラム番号をもつ全データを一括して解除することができます。
- `clearPointSkip()`
ポイント指定の欠品飛ばしの登録情報をすべて解除します。
- `addPalletSkip(num a, num b)`
パレット指定の欠品飛ばしを登録します。
num a: パレット番号を指定します。(1 ~ 100)
num b: パレットカウンタを指定します。(0 ~)
- `delPalletSkip(num a, num b)`
パレット指定の欠品飛ばしを解除します。
num a: パレット番号を指定します。(1 ~ 100)
num b: パレットカウンタを指定します。(0 ~)
-1 を指定することで、num a で指定したパレット番号をもつ全データを一括して解除することができます。
- `clearPalletSkip()`
パレット指定の欠品飛ばしの登録情報をすべて解除します。
- `qCameraWadj(num X, num Y)`
4 点の撮影によりワーク補正の補正値を算出する場合に、この関数を使用します。
4 点の撮影や補正の方法については取扱説明書『カメラ・センサ機能編』をご参照ください。
num X: ワーク補正番号（連番の先頭となる番号）
num Y: ショット番号（1 ~ 4）
戻り値: 0= 正常
-1= 失敗（引数が誤っている）
-2= 失敗（カメラ関連の設定が誤っている）

8.1.2 算術関数

算術系の組み込み関数として、以下が使用できます。

識別子	num abs(num x)
説明	絶対値を求めます。
引数	x: 数値
戻り値	x の絶対値

識別子	num max(num x, num y)
説明	最大値を求めます。 与えられた数値 x と数値 y を比較し、大きい方の数値を返します。
引数	x: 比較対象の数値 y: 比較対象の数値
戻り値	最大値

識別子	num min(num x, num y)
説明	最小値を求めます。 与えられた数値 x と数値 y を比較し、小さい方の値を返します。
引数	x: 比較対象の数値 y: 比較対象の数値
戻り値	最小値

識別子	num degrad(num x)
説明	度（deg）からラジアン（rad）に変換します。 下式によって求めます。 $x * \pi / 180$
引数	x: ラジアン（rad）に変換しようとする数値
戻り値	ラジアン値

識別子	num raddeg(num x)
説明	ラジアン（rad）から度（deg）に変換します。 下式によって求めます。 $x * 180 / \pi$
引数	x: 度（deg）に変換しようとする数値
戻り値	度（deg）単位の数値

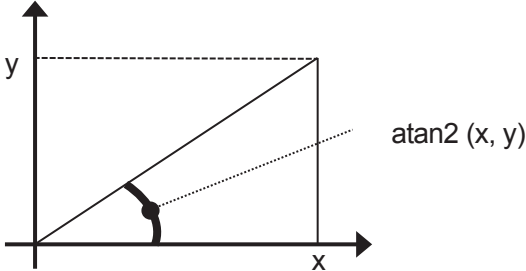
識別子	num sqrt(num x)
説明	平方根の値を求めます。
引数	x: 平方根を求める数値 x < 0 を指定しないでください。
戻り値	x の平方根の値

識別子	num sin(num x)
説明	正弦値を求めます。 ラジアン値で指定してください。
引数	x: 正弦を求めるラジアン値
戻り値	x の正弦値

識別子	num cos(num x)
説明	余弦値を求めます。 ラジアン値で指定してください。
引数	x: 余弦を求めるラジアン値
戻り値	x の余弦値

識別子	num tan(num x)
説明	正接値を求めます。 ラジアン値で指定してください。
引数	x: 正接を求めるラジアン値
戻り値	x の正接値

識別子	num atan(num x)
説明	逆正接を求めます。
引数	x: 逆正接を求める数値
戻り値	x の逆正接値 - $\pi/2 \sim \pi/2$ の範囲のラジアン値です。

識別子	num atan2(num x, num y)
説明	逆正接を求めます。 y を x で割った数値 (y / x) の逆正接です。  $x = 0$ かつ $y > 0$ の場合、戻り値は $\pi / 2$ となります。 $x = 0$ かつ $y < 0$ の場合、戻り値は $-\pi / 2$ となります。 $x > 0$ かつ $y = 0$ の場合、戻り値は 0 となります。 $x < 0$ かつ $y = 0$ の場合、戻り値は π となります。
引数	x: 除数 (割る数) y: 被除数 (割られる数) $x = 0$ かつ $y = 0$ を指定しないでください。
戻り値	x の逆正接値 $-\pi \sim \pi$ の範囲のラジアン値です。

識別子	num int(num x)
説明	x を超えない最大の整数を求めます。
引数	x : 数値
戻り値	x を超えない最大の整数
例	実行結果の例を示します。 例 1 : int(1.3) は 1 となります。 例 2 : int(-1.3) は -2 となります。

識別子	num ip(num x)
説明	x の整数部を返します。 下式と等価です。 $\text{sgn}(x) * \text{int}(\text{abs}(x))$ ($\text{sgn}(x)$ は、x が負の数の場合 -1、正の数の場合 +1 を表します。)
引数	x : 数値
戻り値	x の整数部
例	実行結果の例を示します。 例 1 : ip(1.3) は 1 となります。 例 2 : ip(-1.3) は -1 となります。

識別子	num fp(num x)
説明	x の小数部を返します。 下式と等価です。 $x - \text{ip}(x)$
引数	x: 数値
戻り値	x の小数部
例	実行結果の例を示します。 例 1 : fp(1.3) は 0.3 となります。 例 2 : fp(-1.3) は -0.3 となります。

識別子	num mod(num x, num y)
説明	y を法とする x の値を求めます。 下式と等価です。 $x - y * \text{int}(x / y)$
引数	x: 被除数（割られる数） y: 除数（割る数） y=0 を指定しないでください。
戻り値	y を法とする x の値

識別子	num remainder(num x, num y)
説明	x を y で割った余りを求めます。 下式と等価です。 $x - y * \text{ip}(x / y)$
引数	x: 被除数（割られる数） y: 除数（割る数） y=0 を指定しないでください。
戻り値	x を y で割った余り。

識別子	num pow(num x, num y)
説明	x の y 乗を返します。
引数	x: べき乗される値 y: べき乗する値 x = 0 かつ y = 0 を指定しないでください。 x < 0 かつ y が整数でない値を指定しないでください。
戻り値	x の y 乗の値

8.1.3 文字列関数

文字列系の組み込み関数として、以下が使用できます。

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識別子	概要
文字列系	str	chr(x)	与えられた文字コードを持つ文字列（1文字）を返す
	num	ord(a)	先頭文字コード値を返します。先頭以外は無視します。aで与えられた文字列の文字数が0文字の場合は0を返します。
	num	len(a)	文字列の長さ（バイト長）を返す 多バイトコードには対応していません
	num	strPos(a, b)	aの中でbと一致する最初の部分文字列位置を返す
	str	strMid(a, n, m)	与えられた文字列aの先頭から数えてn文字目からm文字分の文字列を返す
	str	str(x)	数値を10進数文字列に変換
	str	strBin(n, m)	数値を2進数文字列に変換。mは2進文字列桁数
	str	strHex(n, m)	数値を16進数文字列に変換。mは16進文字列桁数
	str	str1SI(x)	数値を1バイト符号付整数に丸め、1バイト文字列に変換 (1 byte Signed Integer)
	str	str2SIBE(x)	数値を2バイト符号付整数に丸め、上位、下位順の2バイト文字列に変換 (2 byte Signed Integer Big Endian)
	str	str2SILE(x)	数値を2バイト符号付整数に丸め、下位、上位順の2バイト文字列に変換 (2 byte Signed Integer Little Endian)
	str	str4SIBE(x)	数値を4バイト符号付整数に丸め、上位、下位順の4バイト文字列に変換 (4 byte Signed Integer Big Endian)
	str	str4SILE(x)	数値を4バイト符号付整数に丸め、下位、上位順の4バイト文字列に変換 (4 byte Signed Integer Little Endian)
	str	str4FBE(x)	数値を浮動小数として、上位、下位順の4バイト文字列に変換 (4 byte Float Big Endian)
	str	str4FLE(x)	数値を浮動小数として、下位、上位順の4バイト文字列に変換 (4 byte Float Little Endian)
	str	str8DBE(x)	数値を倍精度浮動小数として、上位、下位順の8バイト文字列に変換 (8 byte Double Big Endian)
	str	str8DLE(x)	数値を倍精度浮動小数として、下位、上位順の8バイト文字列に変換 (8 byte Double Little Endian)
	num	val(a)	文字列を10進文字列と見なして数値（符号無しの整数型）に変換 文字列の先頭が負符号の場合は0を返す
	num	valBin(a)	文字列を2進文字列（'0', '1'の並び）と見なして数値に変換

x, y : 数値、または数値型変数

n, m : 四捨五入、または切り捨て切り上げにより、ある桁以上にした数値

a, b : 文字列、または文字列型変数

種類	型	識別子	概要
文字列系	num	valHex(a)	文字列を 16 進文字列（'0' ~ '9', 'A' ~ 'F' または 'a' ~ 'f' の並び）と見なして数値に変換
	num	val1SI(a)	先頭 1 文字を 1 バイト符号付整数として変換 (1 byte Signed Integer)
	num	val2SIBE(a)	先頭 2 文字を上位、下位順の 2 バイト符号付整数とみなして変換 (2 byte Signed Integer Big Endian)
	num	val2SILE(a)	先頭 2 文字を下位、上位順の 2 バイト符号付整数とみなして変換 (2 byte Signed Integer Little Endian)
	num	val4SIBE(a)	先頭 4 文字を上位、下位順の 4 バイト符号付整数とみなして変換 (4 byte Signed Integer Big Endian)
	num	val4SILE(a)	先頭 4 文字を下位、上位順の 4 バイト符号付整数とみなして変換 (4 byte Signed Integer Little Endian)
	num	val4FBE(a)	先頭 4 文字を上位、下位順の浮動小数とみなして変換 (4 byte Float Big Endian)
	num	val4FLE(a)	先頭 4 文字を下位、上位順の浮動小数とみなして変換 (4 byte Float Little Endian)
	num	val8DBE(a)	先頭 8 文字を上位、下位順の倍精度浮動小数とみなして変換 (8 byte Double Big Endian)
	num	val8DLE(a)	先頭 8 文字を下位、上位順の倍精度浮動小数とみなして変換 (8 byte Double Little Endian)
	num	valSum(a)	文字列コードを先頭から末尾まで加算した値を返す
	num	valCRC(a)	文字列をビット列とみなしてこれを、 生成多項式 $X^{16}+X^{12}+X^5+1$ で割った余り
	str	bitNot(a)	ビット反転
	str	bitAnd(a, b)	ビット論理積
	str	bitOr(a, b)	ビット論理和
	str	bitXor(a, b)	ビット排他的論理和

8.2 ユーザ定義関数

定番でご使用になる命令列は関数として定義しておくくと便利です。数値型、文字列型の選択ができ、引数を3次元までの配列にすることが可能です。

ユーザ関数には引数を設定することが出来ます。引数に設定できる変数型は「文字列型」と「数値型」です。

ユーザ関数をポイント作業から実行する場合には、指定する引数を変数名で指定する場合、定義されている変数型に合わせる必要があります。

定義例)

UserFunction (“文字列型” , “数値型”)

実行例)

```
let Result = UserFunction(str(UserStr), int(SelectValue))
```

引数の変数型に合わせるために、

文字列型は組み込み関数 :str() を使って文字列型に変換

数値型は組み込み関数 :int() を使って数値型（整数）に変換

数値型の引数に「選択型」変数名を指定したい場合には上記のような、変数の変換操作が必要です。変数型を変換せずに実行すると式評価エラーになります。

TP 所有者ありの場合は、カスタマイズモードでアカウントにログインしてください。
[ユーザ関数定義]

所有者無しの場合は、ティーチングモードから以下を選択してください。

[変数・関数・エイリアス設定]

[ユーザ関数定義]

PC [データ] → [ユーザ関数]

設定項目	タイプ	内容
識別子	識別子 文字列	ポイント作業データの式内で、ここで規定した識別子を使用します。 この識別子を括弧付きで書くことにより関数を呼び出します。 新規作成時に指定し、変更はできません。
保護モード	選択	この定義の扱いに関する設定 (1) 制限無し (2) 公開 (3) 保護 (4) 私的
関数型	選択	関数値の型として2つから選択設定します。 (1) 数値型 (2) 文字列型
引数識別子	識別子 文字列	仮引数の識別子。関数本体内ではこの識別子を使って関数引数値の値を参照したり代入したりします。

設定項目	タイプ	内容
引数渡し型	選択	(1) 値渡し : 関数の評価開始前に実引数が評価され、仮引数に値が渡されます。実引数が変数であってもこの値が変わることはありません。 (2) 参照渡し : 実引数は変数に限られ、仮引数の参照は実引数変数の参照、仮引数へ代入は実引数変数への代入として評価されます。
引数型	選択	引数の型として 2 つから選択設定します。 (1) 数値型 (2) 文字列型
引数次元	選択	(1) 単体 (2) 1 次元配列 (3) 2 次元配列 (4) 3 次元配列
引数各要素数	数値	引数が配列の場合 (1 次元以上の場合) 各次元の配列要素数。
関数本体	命令列	呼び出されたとき実行される命令列

9. エイリアス定義

9.1 I/O エイリアス

I/O エイリアスとは I/O 入出力に別名を与える機能です。I/O-SYS, I/O-1 の入出力を、入出力幅を含めて定義します。set, reset, dataIn, dataOut 命令の入力元・出力先として選択できます（ポイント作業命令ティーチング時の入力元・出力先選択時に「I/O エイリアス」を選択）。

例えば #sysOut16 に塗布装置が接続されている場合、エイリアス設定で #sysOut16 を「DISPENSE」としておきます。その状態でポイント作業命令を「set DISPENSE」とティーチングすれば、見て解りやすいものになる上、後から塗布装置の接続先を変更した場合に、エイリアス設定を変更するだけで、ポイント作業命令を書き換えずに済みます。

（ポイント作業命令を「set DISPENSE」とするには、set 命令の出力先選択時に「Alias」を選択）
対象の I/O をエイリアスとして定義した場合は、全プログラム共通設定でその信号が「フリー」に設定されている必要があります。

例 set #sysOut16 → set DISPENSE
 (I/O エイリアス設定なし) (#sysOut16 を「DISPENSE」に設定)

TP 所有者ありの場合は、カスタマイズモードでアカウントにログインしてください。

 [エイリアス定義]

 [I/O エイリアス定義]

所有者無しの場合は、ティーチングモードから以下を選択してください。

 [変数・関数・エイリアス設定]

 [エイリアス定義]

 [I/O エイリアス定義]

PC [データ] → [エイリアス]

設定項目	タイプ	内容
識別子	識別子 文字列	ポイント作業命令あるいは PLC プログラム命令のパラメータとして、ここで規定した識別子を使用します。一種の変数識別子。ただし、変数と異なり、式の要素としては使えません。 新規作成時に指定します。変更はできません。
保護モード	選択	この定義の扱いに関する設定 (1) 制限無し (2) 公開 (3) 保護 (4) 私的

設定項目	タイプ	内容
キャプション	多言語 文字列	ティーチングモードでの項目名として表示される文字列。
既定値	IOM	既定値。変更前の値。また、全データ削除やシステム既定値に戻す処理をしたときに設定される値。 種類（sysIn / genIn / fbIn / mt1In / mt1Sen / mt2In / mt2Sen / sysOut / genOut / fbOut / mt1Out / mt2Out / mv / mkv）と番号を指定して設定します。
データ幅 既定値	数値	既定値。幅を数値で設定。データ幅が意味を持つのは dataOut, dataIn 命令に使用する場合です。Id, set, reset 等の命令では幅は必要ありません。
入出力	選択	ティーチングモードでの種類選択時の選択肢を切り換えます。出力を選択すると、入力しかできないものは選択肢として挙げないように制限します。 (1) 入力：sysIn / genIn / fbIn / mt1In / mt1Sen / mt2In / mt2Sen / sysOut / genOut / fbOut / mt1Out / mt2Out / mv / mkv (2) 出力：sysOut / genOut / fbOut / mt1Out / mt2Out / mv / mkv
データ幅設定	選択	ティーチングモード設定時に「幅」入力をするか、しないか。幅の意味が無い（1に固定）もの、あるいは幅を変更されたくないものには「無し」と設定します。 (1) 有り：ティーチングモード設定時に「幅」の入力をする（変更可） (2) 無し：ティーチングモード設定時に「幅」の入力をしない

9.2 COM エイリアス

COM エイリアスとは COM に別名を与える機能です。COM ポートを識別子で定義します。ティーチング時、COM/Ethernet 入出力命令（outCOM など）の COM ポート指定として使用できます（ポイント作業命令ティーチング時の入力元・出力先選択時に「COM エイリアス」を選択）。

例えば COM1 に PC が接続されている場合、エイリアス設定で COM1 を「PC」としておきます。その状態でポイント作業命令を「eoutCOM PC, “ERROR”」とティーチングすれば、見て解りやすいものになる上、後から PC の接続先を変更してもポイント作業命令を書き換えずに済みます。（ポイント作業命令を「eoutCOM PC, “ERROR”」とするには、eoutCOM 命令の出力先選択時に「Alias」を選択）

例 eoutCOM port1, “ERROR” → eoutCOM PC, “ERROR”
 (COM エイリアス設定なし) (COM1 を「PC」に設定)

TP 所有者ありの場合は、カスタマイズモードでアカウントにログインしてください。

[エイリアス定義]

[COM エイリアス定義]

所有者無しの場合は、ティーチングモードで以下を選択してください。

[変数・関数・エイリアス設定]

[エイリアス定義]

[COM エイリアス定義]

PC [データ] → [エイリアス]

設定項目	タイプ	内容
識別子	識別子 文字列	ポイント作業命令のパラメータとして、ここで規定した識別子を使用します。一種の変数識別子。ただし、変数と異なり式の要素としては使えません。 新規作成時に指定。変更できません。
保護モード	選択	この定義の扱いに関する設定 (1) 制限無し (2) 公開 (3) 保護 (4) 私的
キャプション	多言語 文字列	ティーチングモード設定時の項目名として表示される文字列。
既定値	選択	既定値。変更前の値。また、全データ削除やシステム既定値に戻す処理をしたときに設定される値。 (1) COM1 (2) COM2 (3) COM3 (JR3000/JC-3 シリーズのみ)

10. ON/OFF 出力制御

10.1 I/O へ出力

■ set, reset, pulse, invPulse

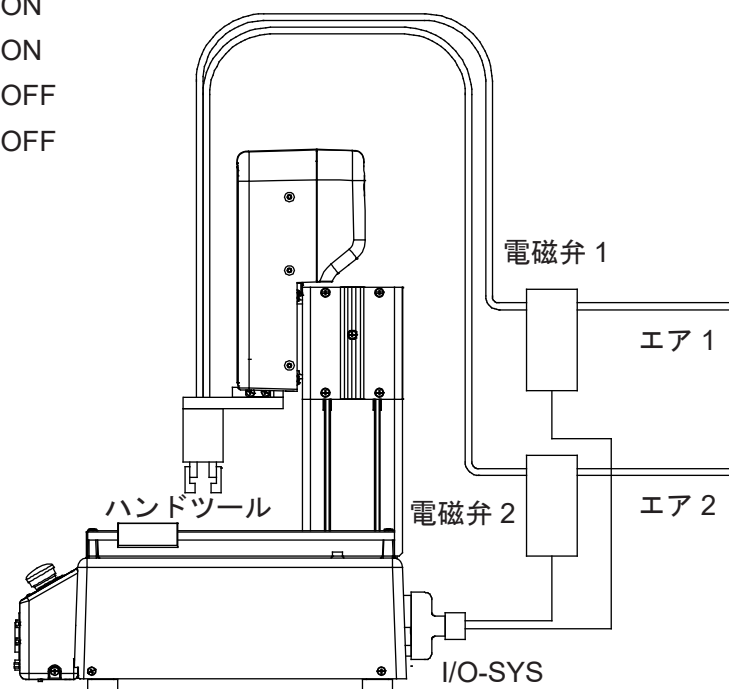
ツールへ出力する（I/O への出力）命令を説明します。命令の種類（カテゴリ）は、[ON / OFF 出力制御] に属します。

命令カテゴリ	命令	必要パラメータ		作業
ON / OFF 出力制御	set	出力先		指定出力先へ ON 出力
	reset	出力先		指定出力先へ OFF 出力
	pulse	出力先	パルス幅	指定出力先へ、指定幅の ON パルス出力
	invPulse	出力先	パルス幅	指定出力先へ、指定幅の OFF パルス（反転パルス）出力

例 下図のように、ロボットにハンドツールが接続されているものとします。

- ハンドツール開 ← エアー 1 閉・エアー 2 開
- ハンドツール閉 ← エアー 1 開・エアー 2 閉
- エアー 1 開 ← 電磁弁 1 ON
- エアー 2 開 ← 電磁弁 2 ON
- エアー 1 閉 ← 電磁弁 1 OFF
- エアー 2 閉 ← 電磁弁 2 OFF
- 電磁弁 1 ON ← #sysOut15 ON
- 電磁弁 2 ON ← #sysOut16 ON
- 電磁弁 1 OFF ← #sysOut15 OFF
- 電磁弁 2 OFF ← #sysOut16 OFF

〈図は JR3203N-AC の例です〉



従って、

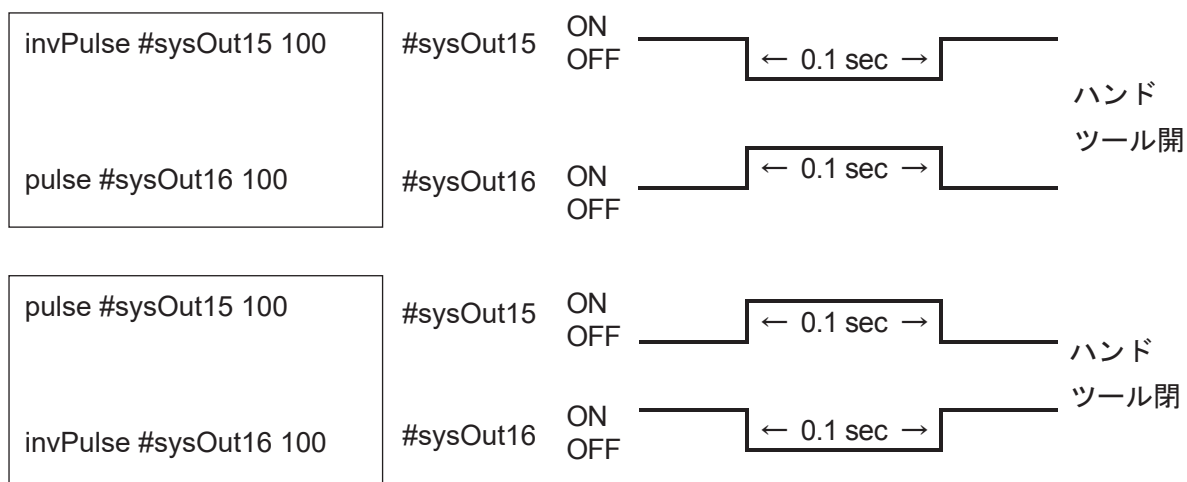
- ハンドツール開 ← #sysOut15 OFF・#sysOut16 ON
- ハンドツール閉 ← #sysOut15 ON・#sysOut16 OFF

■ ハンドツールの開閉を行う出力命令は以下の通りになります。

reset #sysOut15 set #sysOut16	#sysOut15 OFF 出力 #sysOut16 ON 出力	> ハンドツール開
set #sysOut15 reset #sysOut16	#sysOut15 ON 出力 #sysOut16 OFF 出力	> ハンドツール閉

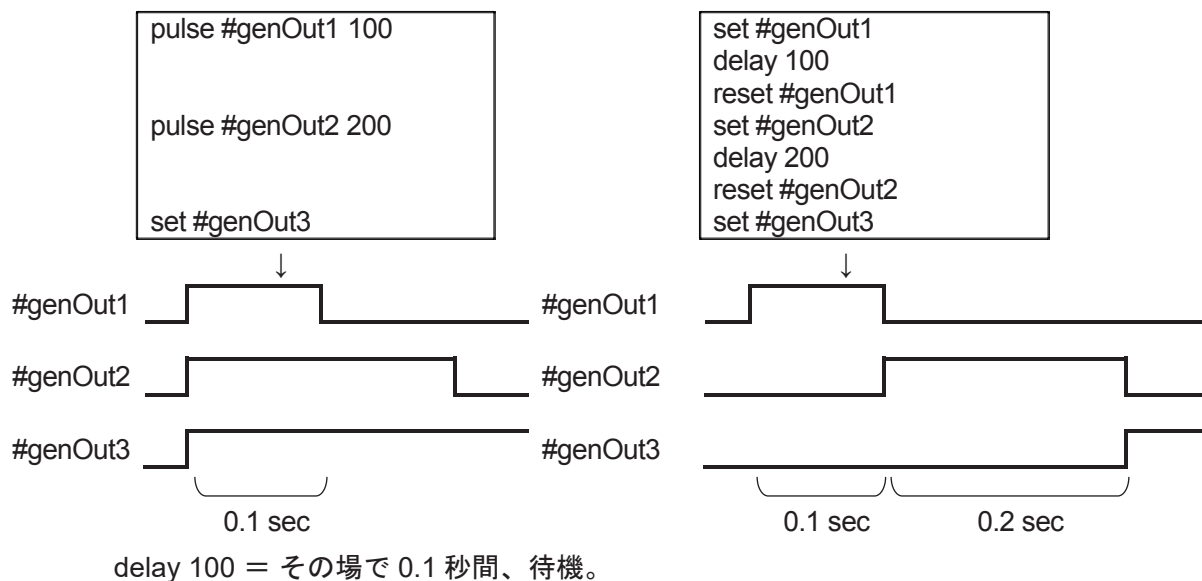
注)「set」命令は、「reset」命令が無い限り、ON 出力し続けます。

■ ハンドツールの開閉をパルス出力で行う場合の命令は以下の通りになります。



注) 「pulse」「invPulse」 命令は、出力し終わるまで待たずに次の命令を実行します。

例えば、以下の 2 つのポイント作業データは同一の結果にはなりません。



注) 「pulse」「invPulse」 命令では、パルス幅を変数や式で指定することもできます。

10.2 X 秒後に出力

■ delaySet, delayReset

「delaySet」「delayReset」 命令を使用すれば、指定した時間後に指定した出力先へ ON / OFF 出力ができます。

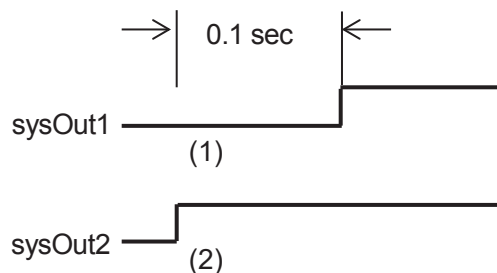
指定できる遅延時間は、1 msec ~ 999999999 msec です。

命令カテゴリ	命令	必要パラメータ		作業
ON / OFF 出力制御	delaySet	出力先	遅延時間	遅延時間後に ON 出力
	delayReset	出力先	遅延時間	遅延時間後に OFF 出力

「delaySet」「delayReset」命令は、この命令の出力を待たずに次の命令を実行します。「delay」で時間待ちしてから set/reset で出力する場合とは、次の命令を実行するタイミングが異なります。

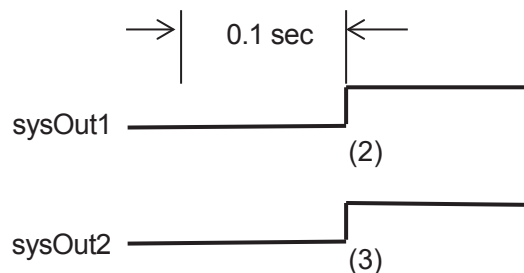
delaySet の例

- (1) delaySet #sysOut1 100
- (2) set #sysOut2
- (3)



Delay/set の例

- (1) delay 100
- (2) set #sysOut1
- (3) set #sysOut2
- (4)



注) 遅延時間を変数や式で指定することもできます。

10.3 ブザーを鳴らす

■ onoffBZ

ポイント作業でブザーを鳴らすことができます。

命令カテゴリ	命令	必要パラメータ	作業
ON / OFF 出力制御	set	出力先 (BZ)	ブザーを鳴らす
	reset	出力先 (BZ)	ブザーを止める
	onoffBZ	ON 時間、OFF 時間	ブザーを断続的に鳴らす

「ブザーを鳴らす／断続的に鳴らす」命令 (set/onoffBZ) を実行した場合、「ブザーを止める」命令 (reset) を実行するまでブザーを鳴らし続けます。

注) 「onoffBZ」命令では、ON 時間、OFF 時間を変数や式で指定することもできます。

10.4 緑 LED を点滅する

■ onoffGLED

ポイント作業で本体前面またはスイッチボックス／操作ボックスの LED を点灯／点滅させることができます。

命令カテゴリ	命令	必要パラメータ	作業
ON / OFF 出力制御	set	出力先 (GLED)	緑 LED を点灯する
	reset	出力先 (GLED)	緑 LED を消灯する
	onoffGLED	ON 時間、OFF 時間	緑 LED を点滅する

「緑 LED を点灯する／点滅する」命令 (set/onoffGLED) を実行した場合は、「緑 LED を消灯する」命令 (reset) を実行するまで、緑 LED を点灯／点滅し続けます。

注) 「onoffGLED」命令では、ON 時間、OFF 時間を変数や式で指定することもできます。

10.5 赤 LED を点滅する

■ onoffRLED

ポイント作業で本体前面またはスイッチボックス／操作ボックスの LED を点灯／点滅させることができます。

命令カテゴリ	命令	必要パラメータ	作業
ON / OFF 出力制御	set	出力先 (RLED)	赤 LED を点灯する
	reset	出力先 (RLED)	赤 LED を消灯する
	onoffRLED	ON 時間、OFF 時間	赤 LED を点滅する

「赤 LED を点灯する／点滅する」命令 (set/onoffRLED) を実行した場合は、「赤 LED を消灯する」命令 (reset) を実行するまで赤 LED を点灯／点滅し続けます。

注) 「onoffRLED」命令では、ON 時間、OFF 時間を変数や式で指定することもできます。

10.6 I/O から数値を出力

■ dataOut, dataOutBCD

0 ～ 999,999,999 までの任意の数値、またはタグコードを、I/O、またはブール型自由変数 (#mv (1 ～ 99) , #mkv (1 ～ 99)) へ出力することができます。

命令カテゴリ	命令	必要パラメータ			作業
ON / OFF	dataOut	出力数値	出力先	出力ビット数	I/O から数値出力
出力制御	dataOutBCD	出力数値	出力先	出力ビット数	I/O から BCD として数値出力

注)

- ・ タグコード出力の場合、複数のポイントに異なる数値をタグコードとして設定しておけば、同じポイント作業データを使用して、異なる数値を出力できます。
- ・ 出力数値、出力ビット数を、変数や式で指定することもできます。

「dataOut」「dataOutBCD」命令には出力数値の他に、パラメータとして「出力ビット数（出力に使用する I/O の本数）」、「出力先（出力に使用する I/O の内、1 番小さい番号）例：#genOut8 ～ 10 を使用する場合は [8]」が必要です。

注) 「dataOut」「dataOutBCD」命令では、出力番号から出力ビット数分の I/O が連番で使われます。連続していない番号の I/O を使用することはできません。

例：

〈指定〉	〈命令表示〉	〈出力〉 6 = 110 (2 進数)
出力数値 : 6		#genOut8 : 0 (OFF)
出力ビット数 : 3	dataOut 6, #genOut8, 3	#genOut9 : 1 (ON)
出力先 : #genOut8		#genOut10 : 1 (ON)

注) 出力数値が出力ビット数に納まらない場合、はみ出した上位の桁は切り捨てになります。

例：

〈指定〉	〈命令表示〉	〈出力〉 14 = 1110 (2 進数)
出力数値 : 14		#genOut8 : 0 (OFF)
出力ビット数 : 3	dataOut 14, #genOut8, 3	#genOut9 : 1 (ON)
出力先 : #genOut8		#genOut10 : 1 (ON)
		: 1 (切り捨て)

注) 出力ビット数は最大「32」まで設定できます。ただし、異なる I/O にまたがって読込幅を設定することはできません。

出力先にフィールドバスを使用する場合は、以下のようになります。

例：

〈指定〉	〈命令表示〉	〈出力〉
出力数値 : 100000		100000=186A0 (16 進数)
出力ビット数 : 32	dataOut 100000, #fbOut (1820) ,32	82h : 86A0h (16 進数)
出力先 : #fbOut (1820)		183h : 0001h (16 進数)

注)

- 出力数値が出力ビット数に納まらない場合、はみ出した上位の桁は切り捨てになります。
- 出力ビット数は、最大「32」まで設定できます。ただし、フィールドバスの設定により、フィールドバス出力エリア範囲外になった場合、範囲外の桁は切り捨てになります。

10.7 動力 ON、サーボ ON/OFF

■ motorPowerON, servoON, servoOFF

以下の命令は JS3 シリーズのみ有効です。JR3000/JC-3 シリーズには以下の命令はありません。

命令によりロボットを動力 ON したり、指定した軸をサーボ ON / OFF の状態にすることができます。サーボ OFF の状態にした軸は、ロボットによって制御することはできません。また、サーボ OFF 状態の X (J1), Y (J2), R (J4) 軸は、手動で動かすことができます。

注) 命令により、動力 OFF することはできません。

命令カテゴリ	命令	必要パラメータ	作業
ON / OFF 出力制御	motorPowerON	-	ロボットを動力 ON します
	servoON	軸指定	指定軸をサーボ ON します
	servoOFF	軸指定	指定軸をサーボ OFF します

11. 条件分岐・条件待ち

11.1 条件分岐

■ if, then, else, endif

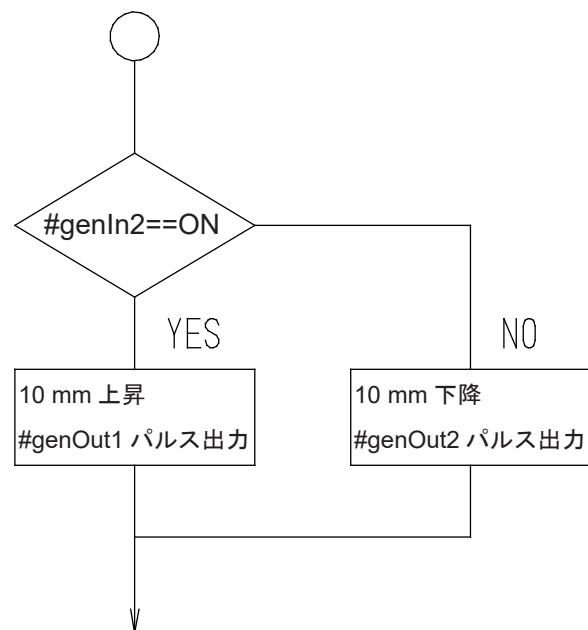
条件によって、違う作業をするポイント作業データ命令を説明します。命令の種類（カテゴリ）は、[条件分岐・条件待ち]に属します。

命令カテゴリ	命令	必要パラメータ	作業
条件分岐・条件待ち	if	-	条件分岐
	then	-	条件が真なら以下の命令を実行
	else	-	条件が偽なら以下の命令を実行
	endif	-	条件分岐終了

注)「if」の後には必ず条件演算命令を置きます。

■ if ~ then ~ else ~ endif 例

例 1 : #genIn2 が ON の場合、Z (J3) 軸を 10 mm 上昇し、#genOut1 にパルス出力。
#genIn2 が ON でなかったら、Z (J3) 軸を 10 mm 下降し、#genOut2 にパルス出力。



例 1 を命令にすると、以下のようになります。

```
if
  Id #genIn2
then
  upZ 20,10
  pulse #genOut1,200
else
  downZ 20,10
  pulse #genOut2,200
endif
```

以下の条件が真なら then へ、偽なら else へ。

#genIn2==ON (条件)

条件が真の場合、以下の命令を実行。

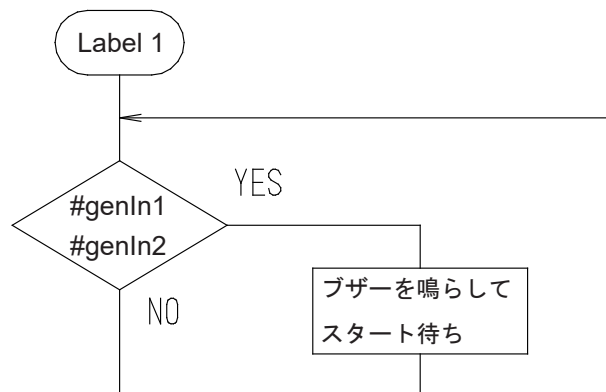
Z (J3) 軸を速度 20 mm/sec で 10 mm 上昇させ、
#genOut1 に ON パルス出力 (パルス幅 0.2 秒)。

条件が偽の場合、以下の命令を実行。

Z (J3) 軸を速度 20 mm/sec で 10 mm 下降させ、
#genOut2 に ON パルス出力 (パルス幅 0.2 秒)。

条件分岐命令終了。

例 2：#genIn1 と #genIn2 が共に ON の場合、ブザーを鳴らしてスタート指示があるまで待機。#genIn1 と #genIn2 が共に ON でなかったら、そのまま次の作業へ。



例 2 を命令にすると、以下のようになります。

```

Label 1
if
  ld #genIn1
  and #genIn2
then
  waitStartBZ
  jump L1
endif
  
```

(飛び先目印)

以下の条件が真なら then へ、偽なら endif の次へ。

#genIn1 = ON (条件 1) で、
かつ、#genIn2 = ON (条件 2)

条件が真の場合、以下の命令を実行。

ブザーを鳴らし、スタート指示があるまでその場で待機。

[Label 1] へ (スタート指示があったら)。

条件分岐命令終了。

注)

- 「then ~」と「else ~」は、必ずしも両方ある必要はありません。ただし、「endif」が無い場合はエラーになります。
- 「waitCondTime」「timeUp」～「endWait」、「if」～「endif」を使用すると、命令行がインデントされます。(下図参照)

```

waitCondTime 200
  ld #genIn2
timeUp
  set #genOut2
  if
    ld #genIn1
  then
    downZ 20,20
    waitCondTime 200
    ld #genIn4
    timeUp
    waitStartBZ
  endWait
endif
endWait
  
```

インデントは、9 段階を超えないようにご注意ください。

9 段階を超えたインデントを含むポイント作業データを運転すると、「指定ポイント作業が異常です」と表示され、エラーとなります。

「timeUp」「endWait」が「waitCondTime」より先にある場合も「指定ポイント作業が異常です」と表示され、エラーとなります。

「if」より先に「then」「else」「endif」がある場合も同様です。

3 段階
2 段階
1 段階

11.2 条件待ち

■ waitCond, waitCondTime, timeUp, endWait

センサ（#genIn2 に接続されていると想定）が ON になるのを待つポイント作業データ命令を説明します。命令の種類（カテゴリ）は、[条件待ち] に属します。

[条件待ち] には、以下の命令があります。

命令カテゴリ	命令	必要パラメータ	作業
条件待ち	waitCondTime	タイムアウト時間	時間付き条件待ち
	timeUp	-	タイムアップ時実行
	endWait	-	待ち終了
	waitCond	-	条件待ち

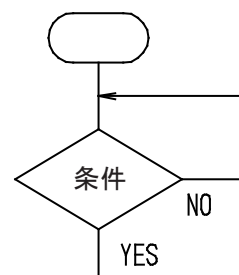
注)

- 条件待ち命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。
- 「waitCond」「waitCondTime」の後には、必ず条件演算命令を置きます。

■ waitCond ～ endWait : 条件が満たされるまで待つ

例 作業物が有る → センサ（#genIn2）ON

作業物が無い → センサ（#genIn2）OFF



```
waitCond
  Id #genIn2
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn2 = ON（条件）

条件の記載行終了。

■ waitCondtime ～ timeUp ～ endWait : 条件が満たされるのを指定時間待つ

例 作業物が 3 秒待っても来ない場合、エラーとし、外部ランプ（#genOut2 に接続されている想定）を ON にして、スタート指示があるまでその場で待機。作業者が問題を取り除き、スタートスイッチを押すと、運転を再開する。

```
waitCondTime 3000
  Id #genIn2
timeUp
  set #genOut2
  waitStartBZ
  reset #genOut2
endWait
```

以下の条件が満たされるのを 3 秒間待つ。

#genIn2 = ON（条件）

3 秒たっても条件が満たされない場合、

#genOut2 に ON 出力し、

スタート指示があるまで、その場で待機。

スタート指示があったら #genOut2 に OFF 出力。

3 秒たっても、条件が満たされない場合の処理命令記載行終了。

注)

- ・「endWait」「timeUp」は、単独では使用できません。
- ・「waitCondTime」は、タイムアウト時間を変数や式で指定することもできます。

例

```
declare numeric wtime
if
  Id #genIn3
then
  let wtime = 3000
else
  let wtime = 1000
endif
waitCondTime wtime
  Id #genIn2
timeUp
  set #genOut2
  waitStartBZ
  reset #genOut2
endWait
```

ローカル変数「wtime」の宣言。

もし

#genIn3 = ON

であれば

「wtime」に 3000 を代入。

でなければ

「wtime」に 1000 を代入。

以下の条件が満たされるのを 3 秒／1 秒間待つ。

#genIn2 = ON (条件)

3 秒／1 秒たっても条件が満たされない場合、

#genOut2 に ON 出力し、

スタート指示があるまで、その場で待機。

スタート指示があったら、#genOut2 に OFF 出力。

3 秒／1 秒たっても、条件が満たされない場合の処理命令
記載行終了。

12. 条件演算

12.1 条件設定

■ Id, ldi, and, ani, or, ori, anb, orb

条件分岐・条件待ち (if, waitCond, waitCondTime) 命令の次に置く条件演算命令を説明します。命令の種類 (カテゴリ) は、[条件演算] に属します。

命令カテゴリ	命令	必要 パラメータ	作業
条件演算	Id	ブール型変数または式	ON 入力
	ldi	ブール型変数または式	OFF 入力
	and	ブール型変数または式	直列 ON 入力
	ani	ブール型変数または式	直列 OFF 入力
	or	ブール型変数または式	並列 ON 入力
	ori	ブール型変数または式	並列 OFF 入力
	anb	-	ブロックの直列接続
	orb	-	ブロックの並列接続

注) ブール型は次の値とする ON : (真) 0 以外、 OFF : (偽) 0

命令のパラメータとして I/O-SYS 入力 (#sysIn)、I/O-1 入力 (#genIn)、I/O-H 入力 (#handIn)、I/O-FB 入力 (#fbIn) の他に、I/O-SYS 出力 (#sysOut)、I/O-1 出力 (#genOut)、I/O-H 出力 (#handOut)、I/O-FB 出力 (#fbOut)、システムフラグ (#sysFlag)、補助リレー (#mv)、キープリレー (#mkv)、パレットフラグや、I/O エイリアス、PLC タイマ (#seqT)、PLC カウンタ (#seqC) を指定することができます。

また、比較演算式を使用することもできます。比較演算式には、上記のパラメータ以外の変数や関数も使用できます。

比較演算式	意味	比較演算式	意味
○ == □	□は○と等しい	○ <= □	□は○より大きいまたは等しい
○ < □	□は○より大きい	○ >= □	□は○より小さいまたは等しい
○ > □	□は○より小さい	○ <> □	等しくない
		○ >< □	

条件演算は必ず Id または ldi 命令の行から始まります。単独の ON (真) / OFF (偽) 条件だけなら 1 行の命令ですが、and, or 等で複数の条件を繋ぐ場合には複数行の命令列になります。

また、条件演算で式を使うこともできます。この場合には式の結果を 0（偽）／0 以外（真）として判断します。

■ Id : ON 入力

```
waitCond
  Id #genIn2
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn2 = ON（条件）

条件の記載行終了。

■ Idi : OFF 入力

```
waitCond
  Idi #genIn2
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn2 = OFF（条件）

条件の記載行終了。

■ and : 直列 ON 入力

```
waitCond
  Id #genIn1
  and count>=10
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn1 が ON（条件 1）で、

かつ、count の値が 10 以上（条件 2）の場合。

条件の記載行終了。

■ ani : 直列 OFF 入力

```
waitCond
  Idi #genIn1
  ani count>=10
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn1 が OFF（条件 1）で、

かつ、count の値が 10 以下（条件 2）の場合。

条件の記載行終了。

■ or : 並列 ON 入力

```
waitCond
  Id #genIn1
  or #genIn2
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn1 が ON（条件 1）か、

あるいは、#genIn2 が ON（条件 2）の場合。

条件の記載行終了。

■ ori : 並列 OFF 入力

```
waitCond
  Idi #genIn1
  ori #genIn2
endWait
```

以下の条件が満たされるまで、その場で待機。

#genIn1 が OFF（条件 1）か、

あるいは、#genIn2 が OFF（条件 2）の場合。

条件の記載行終了。

■ anb : ブロックの直列接続

```
waitCond
  ld count>=10
  or flag
  ldi #genIn1
  ani #genIn2
  anb
endWait
```

以下の条件が満たされるまで、その場で待機。

count が 10 以上か、	}	条件 1
あるいは、flag が ON の場合。		
#genIn1 が OFF で、	}	条件 2
かつ、#genIn2 も OFF の場合		

条件 1 が真で、かつ、条件 2 も真の場合。

条件の記載行終了。

■ orb : ブロックの並列接続

```
waitCond
  ld count>=10
  or flag
  ldi #genIn1
  ani #genIn2
  orb
endWait
```

以下の条件が満たされるまで、その場で待機。

count が 10 以上か、	}	条件 1
あるいは、flag が ON の場合。		
#genIn1 が OFF で、	}	条件 2
かつ、#genIn2 も OFF の場合		

条件 1 が真か、あるいは、条件 2 が真の場合。

条件の記載行終了。

注)

- 「anb」「orb」があつて、それに相当する「ld」または「ldi」がない場合は、「ポイント作業が異常です」と表示され、エラーとなります。
- ここまでの設定例では「count」「flag」は任意の変数です。
- 変数を使用した代入式を設定した場合、宣言が設定されていないとエラーが出ます。詳細は「[20. 変数、コメント、システム制御](#)」をご参照ください。

13. ディレイ、データ IN、スタート待ち

13.1 時間遅延

■ delay

時間遅延のポイント作業データ命令を説明します。

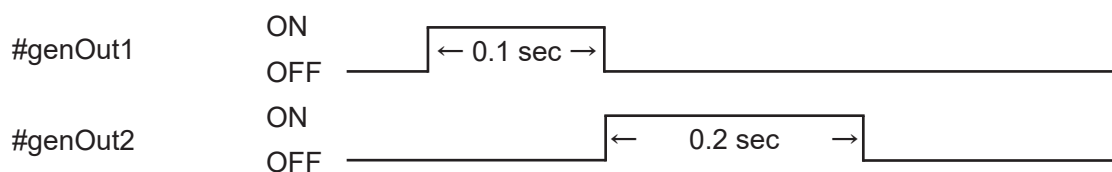
命令カテゴリ	命令	必要 パラメータ	作業
ディレイ、データ IN、 スタート待ち	delay	遅延時間	指定した遅延時間をその場で待機します

注)「delay」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

■ delay : 指定時間遅延 例

```
set #genOut1
delay 100
reset #genOut1
set #genOut2
delay 200
reset #genOut2
```

#genOut1 に ON 出力して、
0.1 sec 遅延する。
#genOut1 に OFF 出力。
#genOut2 に ON 出力して、
0.2 sec 遅延する。
#genOut2 に OFF 出力。



遅延時間を数値でなく変数や式で指定することもできます。

例

```
declare numeric wtime
if
  ld #genIn1
then
  let wtime = 100
else
  let wtime = 200
endif
set #genOut1
delay wtime
reset #genOut1
```

ローカル変数「wtime」の宣言。

もし

#genIn1 = ON

であれば

「wtime」に 100 を代入。

でなければ

「wtime」に 200 を代入。

#genOut1 に ON 出力して、
0.1 sec / 0.2 sec 遅延する。

#genOut1 に OFF 出力。

13.2 スタート指示を待つ

■ waitStart, waitStartBZ

スタート指示があるまで運転を停止するポイント作業データ命令を説明します。

命令カテゴリ	命令	必要 パラメータ	作業
ディレイ、データ IN、 スタート待ち	waitStart	-	スタート指示があるまで、その場で待機
	waitStartBZ	-	スタート指示があるまで、ブザーを鳴らしながらその場で待機

注) 「waitStart」「waitStartBZ」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

■ waitStart : スタート待ち

例

```
set #genOut1
waitStart
reset #genOut1
```

#genOut1 に ON 出力し、

スタート指示があるまで、その場で待機。

#genOut1 に OFF 出力。(スタート指示があったら)

■ waitStartBZ : スタート待ち（ブザー）

例 2秒待っても #genIn1 が ON にならない場合は、エラー発生として #genOut2（外部アラーム／異常ランプ等に接続している想定）を ON し、「ブザーを鳴らして、スタート待ち」にする。作業者が問題を取り除き、再スタートの指示を出したら、#genOut2 に OFF 出力し、ポイント 05 から運転再開。

```
waitCondTime 2000
  Id #genIn1
timeUp
  upZ 20, 50
  set #genOut2
  waitStartBZ
  reset #genOut2
  goPoint PTP3, 5
endWait
```

#genIn1 が ON になるのを 2 秒待つ。

2 秒待つ、#genIn1 が ON にならない場合、Z（J3）軸を 50 mm 上昇（速度 20 mm/sec で）させ、#genOut2 に ON 出力をしてブザーを鳴らし、スタート指示があるまで、その場で待機。#genOut2 に OFF 出力し（スタート指示があったら）、ポイント 05 へ移動。
2 秒待つ、#genIn1 が ON にならない場合の処理命令終わり。

注）「waitCondTime」「timeUp」～「endWait」、「if」～「endif」を使用すると、命令行がインデントされます。（下図参照）

```
waitCondTime 200
  Id #genIn2
timeUp
  set #genOut2
  if
    Id #genIn1
  then
    downZ 20,20
    waitCondTime 200
    Id #genIn4
    timeUp
    waitStartBZ
  endWait
  endif
endWait
```

インデントは、9 段階を超えないようにご注意ください。

9 段階を超えたインデントを含むポイント作業データを運転すると、「指定ポイント作業が異常です」と表示され、エラーとなります。

「timeUp」「endWait」が「waitCondTime」より先にある場合も「指定ポイント作業が異常です」と表示され、エラーとなります。

「if」より先に「then」「else」「endif」がある場合も同様です。

3 段階
2 段階
1 段階

13.3 I/O から入力

■ dataIn, dataInBCD

I/O、またはブール型変数（#mv（1 ～ 99）、#mkv（1 ～ 99））から数値を読み込み、指定した変数へ代入します。

命令カテゴリ	命令	必要パラメータ			作業
ディレイ、データ IN、スタート待ち	dataIn	代入先変数	読込元	読込幅	数値データを I/O から読込
	dataInBCD	代入先変数	読込元	読込幅	数値データを I/O から BCD として読込

BCD = 2 進法 10 進数

読込幅を、変数や式で指定することもできます。

「dataIn」「dataInBCD」命令には読み込んだ数値を代入する変数の他に、パラメータとして「読込幅（入力に使用する I/O の本数）」、「読込元（入力に使用する I/O の内、1 番小さい番号（例：#genIn3 ～ 10 を使用する場合は [3]））」が必要です。

注）読込元から読込幅分の I/O が連番で使用されます。連続していない番号の I/O を使用することはできません。

例

```
declare numeric code
dataIn code, #genIn1, 8
```

「code」のローカル変数宣言
#genIn1（I/O-1）～ #genIn8 までを、数値として読み込み、
「code」へ代入する。

```
declare numeric code
dataInBCD code, #genIn1, 8
```

「code」のローカル変数宣言
#genIn1（I/O-1）～ #genIn8 までを、BCD として読み込み、
「code」へ代入する。

I/O-1 の状態

#genIn8	#genIn7	#genIn6	#genIn5	#genIn4	#genIn3	#genIn2	#genIn1
OFF	OFF	OFF	ON	OFF	OFF	ON	OFF

読込幅 8 →

読み込んだ状態が上記のように 00010010 となる場合、次のような結果になります。

「dataIn」命令の場合、「code」の値は 18。

「dataInBCD」命令の場合、「code」の値は 12。

注）読込幅は最大「32」まで設定できます。ただし、異なる I/O にまたがって読込幅を設定することはできません。

読込元にフィールドバスを使用する場合は、以下のようになります。

例

```
declare numeric code  
dataIn code, #fbIn (1020), 32
```

「code」のローカル変数宣言
#fbIn (1020) ～ #fbIn (103F) までを、数値として読み込み、「code」へ代入する。

```
declare numeric code  
dataInBCD code, #fbIn (1020), 32
```

「code」のローカル変数宣言
#fbIn (1020) ～ #fbIn (103F) までを、BCD として読み込み、「code」へ代入する。

注) フィールドバス入力エリア 102h ～ 103h に外部機器 (PLC) から下記の通り設定しておきます。

102h : 0000h (16 進数)

103h : 0010h (16 進数)

読み込んだ状態が上記の場合、次のような結果になります。

「dataIn」命令の場合、「code」の値は 1048576。

「dataInBCD」命令の場合、「code」の値は 100000。

14. パレット操作

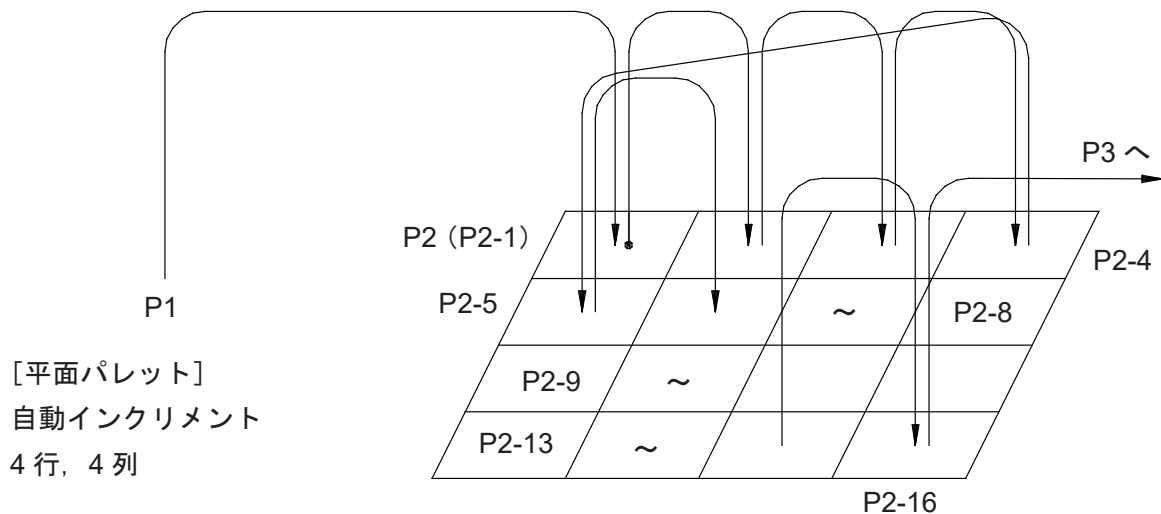
14.1 パレット命令

■ loopPallet, resPallet, incPallet

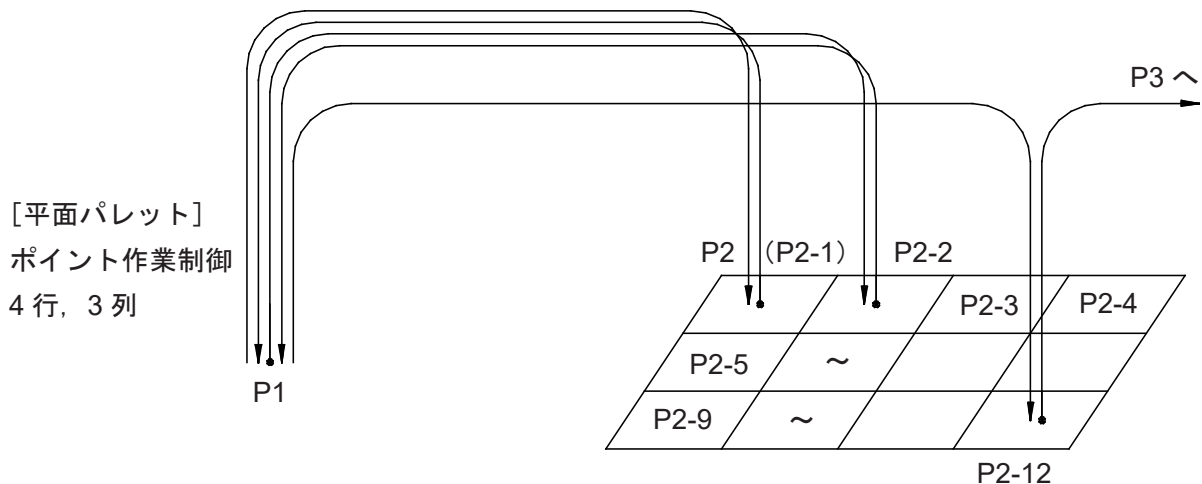
ポイント属性データ「パレット」には、カウンタを自動的に更新して行く（パレット内の次ポイントへ順次進んで行く）[自動インクリメント]と、ポイント作業で更新するよう設定しない限りカウンタが更新されない（パレットの次ポイントへ移動しない）[ポイント作業制御]の2種類があり、これを切り替えることができます。

自動インクリメントの場合は、パレット制御のためのポイント作業命令は必要ありません。パレット内の次ポイントへ順次進んで行き、自動的にパレットカウンタを更新します。ただし、自動インクリメントのパレットは、下図のように P2-1→P2-2→P2-3・・・と順に進んで行くことしかできません。

例 自動インクリメントのパレット



例 ポイント作業制御のパレット



ポイント作業制御のパレットの場合は、前図のようにパレットを進む度に P1 へ戻る（P1→ P2（P2-1）→P1→P2-2→P1→P2-3・・・）と言った動作ができます。

[ポイント作業制御] で使用するパレット命令には次の 3 つがあります。

命令カテゴリ	命令	必要パラメータ	作業
パレット	loopPallet	パレット番号， 飛び先ポイント番号	パレットのカウンタに 1 を加え、カウンタが最大値に満たない場合は、指定されたポイントへ移動します
	resPallet	パレット番号	パレットのカウンタを 0 にします。パレットフラグが 1（真）の場合、OFF（偽）になります（Ver.5 以降）。*
	incPallet	パレット番号	パレットのカウンタに 1 加えます

* resPallet 命令は、システムプログラムのバージョンにより仕様が異なります。

■ システムプログラム Ver.4 以前の resPallet 命令

- ・ パレットのカウンタを 0 にします
- ・ パレットフラグが ON（真）の場合、OFF（偽）になりません。

■ システムプログラム Ver.5 以降の resPallet 命令

- ・ パレットのカウンタを 0 にします
- ・ パレットフラグが ON（真）の場合、OFF（偽）になります。

また、パレット制御をするのに有効な変数として、以下の 2 つがあります。

[palletFlag (n)] : 以下の内容を持つブール型変数です。
 番号 n のパレットのパレットカウンタが最大 = ON（真）
 番号 n のパレットのパレットカウンタが最大値に満たない = OFF（偽）
 <パレットフラグのリセット>
 パレットフラグは電源投入時または運転開始時にリセットします。
 パレットフラグは一旦 ON（真）になると、次のリセットまで値を保持します。
 [palletCount (n)] : 番号 n のパレットのパレットカウンタの値を持つ数値型変数です。

例のポイント作業制御のパレットで、P1 で荷物を掴み（set #genOut1）、P2 のパレットへ置いて行く（reset #genOut1）ものとして、この場合のポイント作業データ例を以下に示します。

ポイント作業データ（P1 設定用）

set #genOut1

荷物を掴みます。

ポイント作業データ（P2 設定用）

reset #genOut1
loopPallet 10,1

荷物を置き（離し）ます。
 パレット 10 のカウンタに 1 加え、
 カウンタが最大になったら、次の命令（この場合、次の命令がないので、ポイント作業終了）へ進む。
 カウンタが最大値に満たなければ、P1 へ移動する。

注)「loopPallet」での移動(上記 P2 設定用ポイント作業データの場合は、P1 への移動)は、プログラムデータの「PTP 駆動条件」に従います。

命令「loopPallet」の代わりに「incPallet(指定したパレットカウンタに1をカウント)」を使用すると、パレット命令は以下のようになります。

〈LOOP を使わない場合〉

```
reset #genOut1
incPallet 10
if
  ldi #palletFlag (10)
then
  goPoint PTP3,1
endif
```

荷物を置き(離し)ます。
パレット 10 のカウンタに 1 加えます。
もし
パレット 10 のカウンタが最大値に満たなければ、
P1 へ移動(PTP 駆動条件 03 に従う)する。

注)「loopPallet」では、指定したポイントへの移動は PTP 駆動条件(プログラムデータ)に従いますが、「incPallet」を使うと「goPoint」または「goRPoint」命令が使用でき、PTP 駆動条件(ポイント属性データ)を選択することができます。

「incPallet」を使用すると、以下のように P1 へ移動する毎にパルス出力をする等もできます。

```
reset #genOut1
incPallet 10
if
  ldi #palletFlag (10)
then
  pulse #genOut5,200
  goPoint PTP0,1
endif
```

荷物を置き(離し)ます。
パレット 10 のカウンタに 1 加えます。
もし
パレット 10 のカウンタが最大値に満たなければ、
パルスを出力し
P1 へ移動する。

パレット番号を(「loopPallet」の場合は飛び先ポイント番号も)式で指定することができます。
例

```
declare numeric pal
if
  ld #genIn3
then
  let pal = 5
else
  let pal = 6
endif
reset #genOut1
loopPallet pal,1
```

ローカル変数「pal」の宣言。

もし

#genIn3 = ON

であれば

「pal」に 5 を代入。

でなければ

「pal」に 6 を代入。

荷物を置き(離し)ます。

パレット 5 / 6 のカウンタに 1 加え、
カウンタが最大になったら、次の命令(この場合、次の命令がないので、ポイント作業終了)へ進む。
カウンタが最大値に満たなければ、P1 へ移動する。

15. 実行流れ制御

15.1 種別設定作業をサブルーチンコール

■ callBase

カスタマイズモードで作成されたユーザ定義種別のポイントではポイント作業等を設定すると、種別に付加されているポイント作業等が実行されません。例えば「スタート待ち」は、スタート／ストップスイッチを押すかスタート信号を受信するまで待機するポイントですが、任意の「ポイント作業」を設定すると待機をしなくなります。

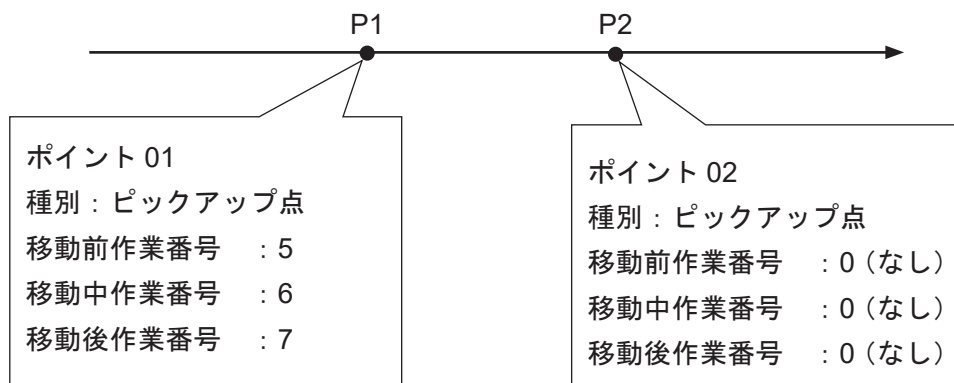
例 右のようなユーザ定義種別があるとします。

この時、下図の P1、P2 で運転時に実行されるポイント作業データは次のようになります

- P1) 移動前作業 : ポイント作業データ 5
移動中作業 : ポイント作業データ 6
移動後作業 : ポイント作業データ 7

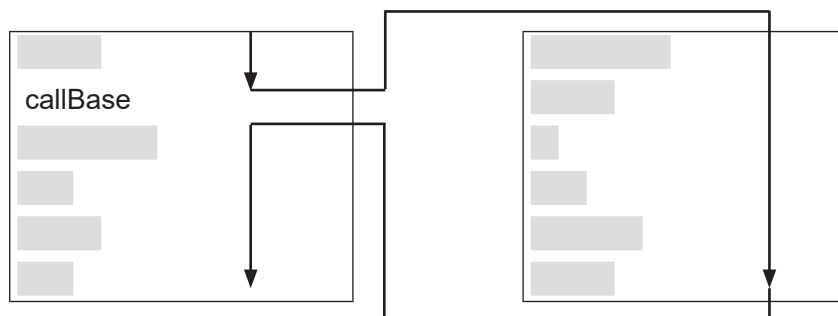
- P2) 移動前作業 : ユーザ定義種別に付加されている「移動前作業」
移動中作業 : ユーザ定義種別に付加されている「移動中作業」
移動後作業 : ユーザ定義種別に付加されている「移動後作業」

タイトル : ピックアップ点
基底種別 : PTP 駆動点
移動前作業 : 有り
移動中作業 : 有り
移動後作業 : 有り



このような場合、ユーザ定義種別のポイントに設定するポイント作業データで「callBase」命令を使用すると、種別に付加されているポイント作業等をサブルーチンコールすることができます。例の、ポイント作業データ 7 に「callBase」命令を使用すると、P1 のポイント作業実行時にポイント作業データ 7 でユーザ定義種別に付加されている「ポイント作業」の命令列をサブルーチンコールします。

(ポイント作業データ 7) (ユーザ定義種別に付加されている「ポイント作業」)



命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	callBase	-	ユーザ定義種別のポイントで、種別に付加されている作業命令列を呼び出して実行します

この「callBase」命令は、種別に付加されている作業命令列をサブルーチンコールする命令です。従って、移動前作業、移動中作業、CP 駆動中作業で「callBase」命令を実行すると、それぞれ種別に付加されている移動前作業、移動中作業、CP 駆動中作業の命令列がサブルーチンコールされます。

例の場合、ポイント作業データ 5 で使用すると、P1 の移動前作業実行時にポイント作業データ 5 で種別に付加されている移動前作業の命令列をサブルーチンコールします。

15.2 ポイント作業データをサブルーチンコール

■ callJob

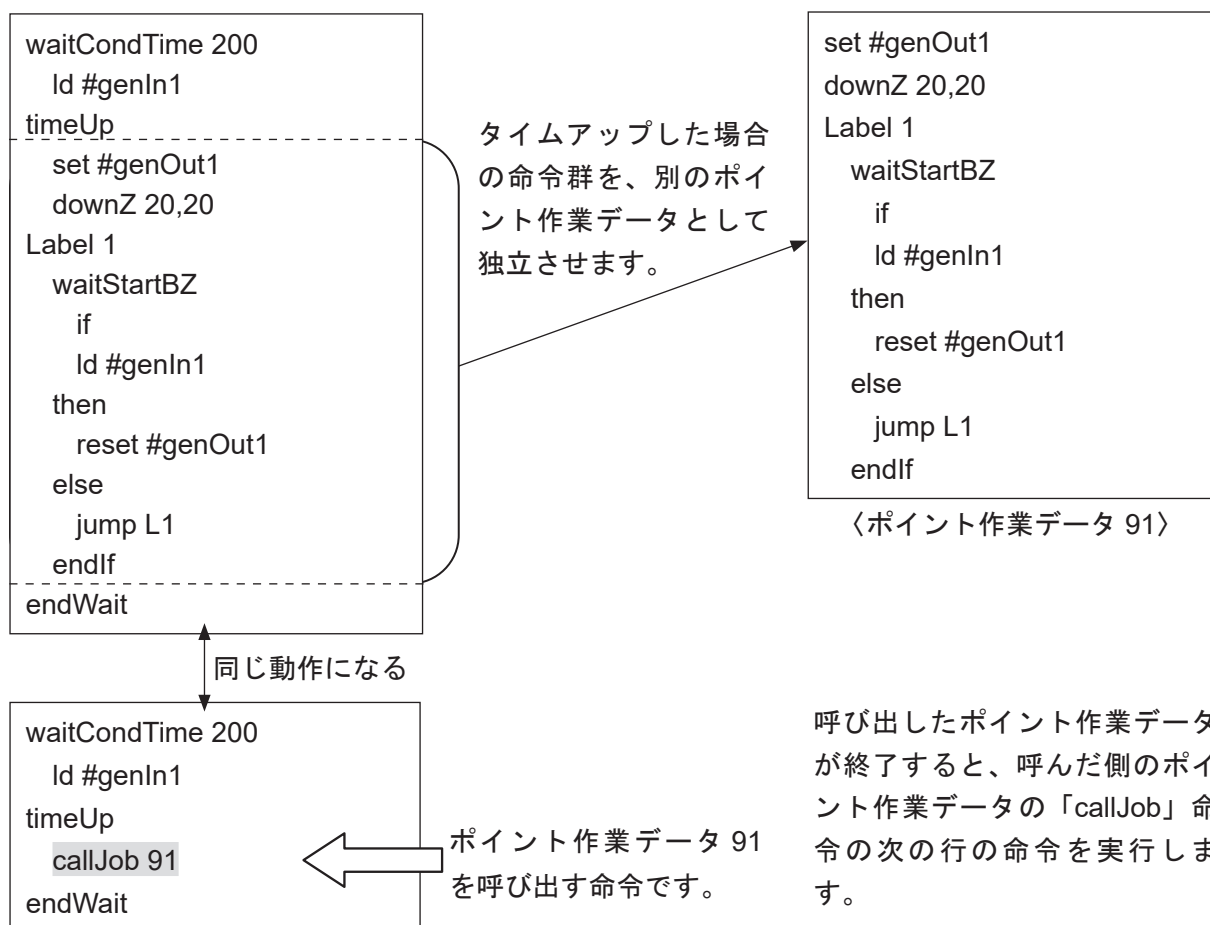
ポイント作業実行中に、別のポイント作業データを呼び出して実行することができます。

エラー処理など、複数のポイント作業に共通する部分を、1つのポイント作業データとし、他のポイント作業データから呼び出して使用すれば、ポイント作業データが短縮され、見やすくなります。

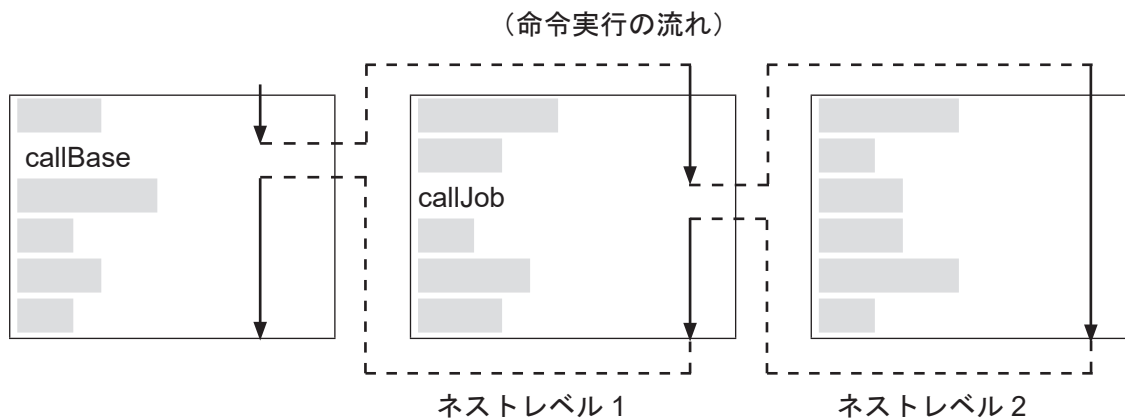
また、本来はポイント作業データの一部である命令群を1つのポイント作業データとすることで、その部分だけをテストすることもできます。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	callJob	ポイント作業データ番号	指定された番号のポイント作業データをサブルーチンコールします

注)「callJob」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。



注)「callJob」命令で呼び出すポイント作業データに、「callJob」命令が含まれている場合、ネストレベルが30を超えるとエラー (No.43) となります。(↓ネストレベル2の例)



ポイント作業データ番号を式で指定することもできます。

例)

```
declare numeric ejob
waitCondTime 200
  Id #genIn1
timeUp
  if
    Id #genIn2
  then
    let ejob = 9
  else
    let ejob = 10
  endif
  callJob ejob
endWait
```

ローカル変数「ejob」の宣言。

以下の条件が満たされるのを 0.2 秒間待つ。

#genIn1 = ON (条件)

0.2 秒たっても条件が満たされない場合、
もし

#genIn2 = ON

であれば

「ejob」に 9 を代入。

でなければ

「ejob」に 10 を代入。

番号9／10のポイント作業データをサブルーチンコール。

15.3 ポイント作業終了

■ returnJob

複雑な条件とそれに対応する処理があり、その中でもう処理がない場合、「returnJob」命令によって、そのポイント作業を終了することができます。

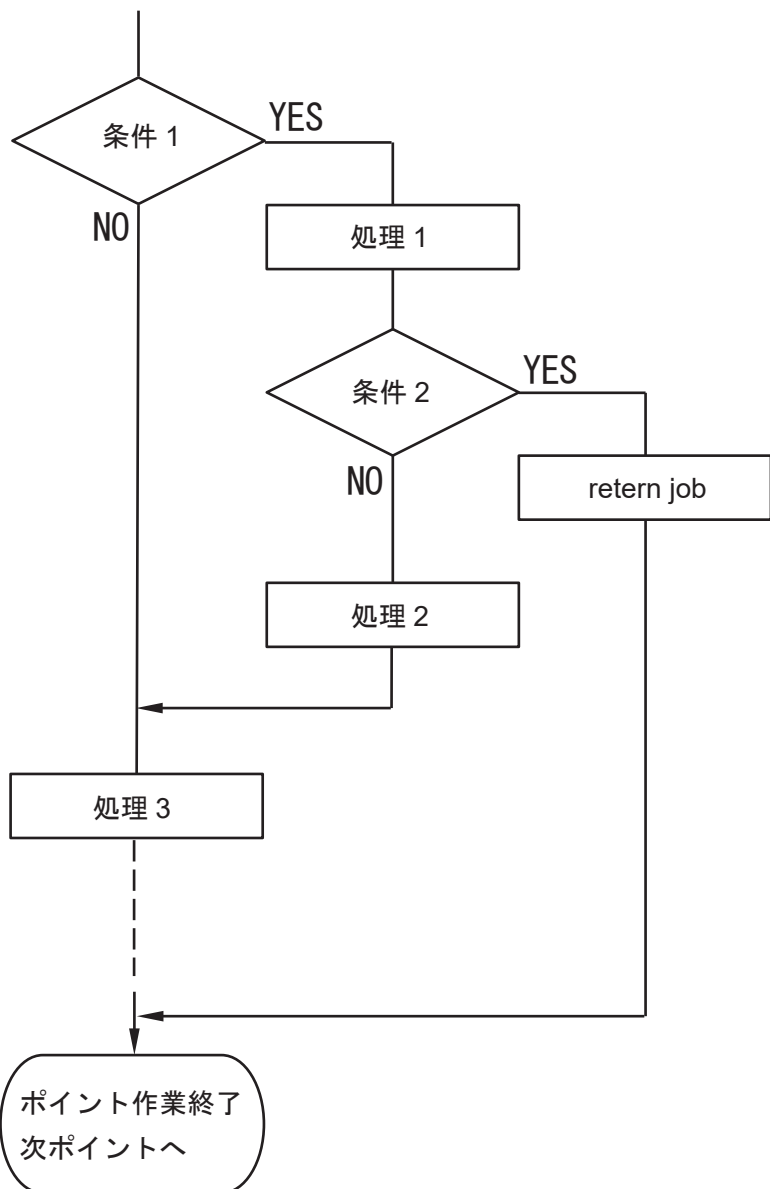
命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	returnJob	-	ポイント作業を終了します

例 右図のようなポイント作業をティーチングすると、以下ようになります。

```

if
  条件 1
then
  処理 1
  if
    条件 2
  then
    returnJob
  else
    処理 2
  endif
endif
処理 3

```



「returnJob」を外してしまうと、「条件 2」が ON (YES) の場合も、「処理 3」を実行してしまいます。

15.4 プログラムをサブルーチンコール

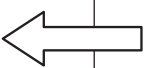
■ callProg

ポイント作業実行中に、別のプログラムを呼び出して実行することができます。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	callProg	プログラム番号	指定した番号のプログラムをサブルーチンコールします

注) 「callProg」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

```
waitCondTime 200
  Id #genIn1
timeUp
  callProg 17
endWait
```

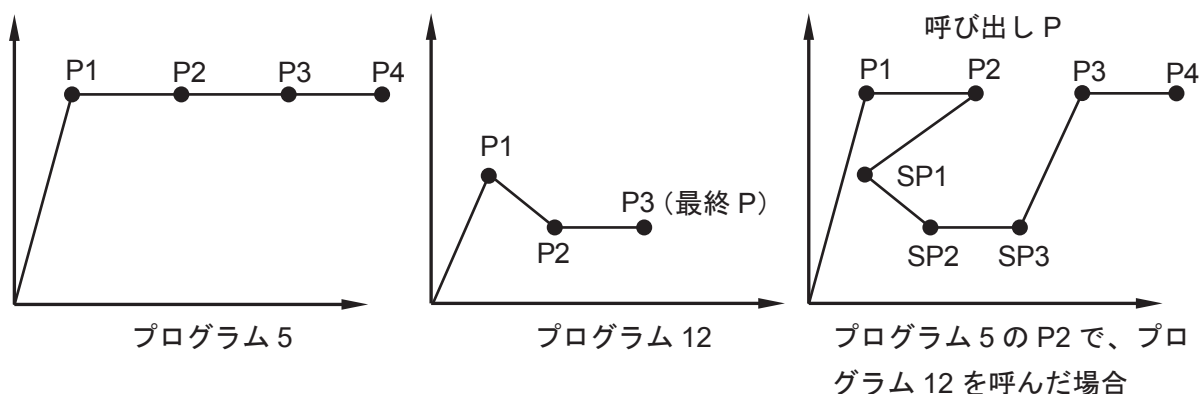


17 番のプログラムを
呼び出す命令です。

呼び出したプログラムが終了すると、呼んだ側のポイント作業データの「callProg」命令の次の行の命令を実行します。(左の場合、「endWait」)

以下、呼び出したプログラムを「サブプログラム」、サブプログラムのポイント 1 を「SP1」と呼んで説明します。

注) サブプログラムの [サイクルモード] が [連続運転] に設定されている場合、サブプログラムが連続運転されます。



また、プログラム番号を式で指定することもできます。

例

```
declare numeric eprg
waitCondTime 200
  Id #genIn1
timeUp
  if
    Id #genIn2
  then
    let eprg = 9
  else
    let eprg = 10
  endif
  callProg eprg
endWait
```

ローカル変数「eprg」の宣言。

以下の条件が満たされるのを 0.2 秒間待つ。

#genIn1 = ON（条件）

0.2 秒たっても条件が満たされない場合、

もし

#genIn2 = ON

であれば

「eprg」に 9 を代入。

でなければ

「eprg」に 10 を代入。

番号 9 / 10 のプログラムをサブルーチンコール。

■ 位置データ（プログラムデータ項目名）設定

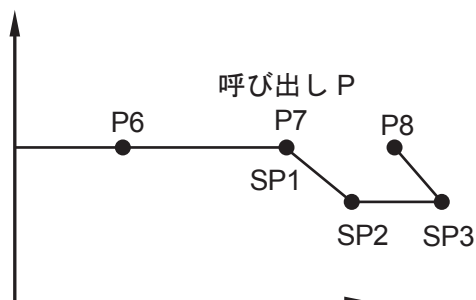
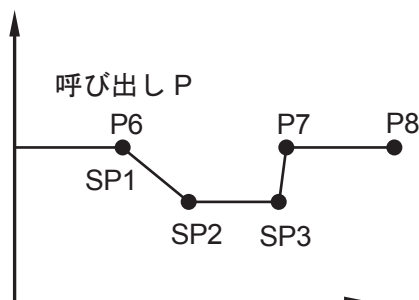
プログラムデータの [位置データ] で、ポイントデータに含まれる座標（位置データ）の扱い方を設定できます。次の 3 種類があります。

- ・ 絶対座標：位置データの数値を、ロボットの固定座標を指すものとして扱います。
- ・ 相対座標：位置データの数値を、プログラム開始座標からの距離を指すものとして扱います。
- ・ 移動量：位置データの数値を、次点までの距離を指すものとして扱います。

サブプログラムを「相対座標」、または「移動量」に設定しておけば、呼び出したポイント（「callProg」命令を含むポイント作業データを設定したポイント）から常に等距離で運転します。

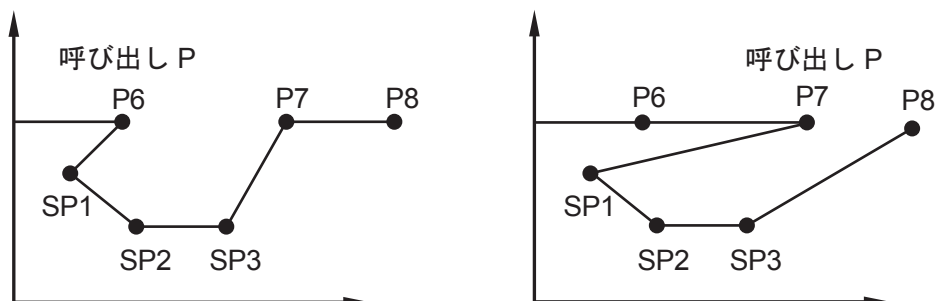
- ・ サブプログラムが [相対座標]、または [移動量] に設定されている例

現在のポイント（呼び出したポイント）を P1（SP1）とします（サブプログラムの P1 座標データは無視します）。作業原点は無視します。



- サブプログラムが「絶対座標」に設定されている例

呼び出したポイントの位置に関わらず、ポイントデータの座標で運転します。現在のポイント（呼び出したポイント）で、（サブプログラムの）作業原点の開始時ポイント作業を実行し（移動はしません）、P1（SP1）へ移動します。



注) 「callProg」命令で呼び出すプログラムに、「callProg」命令が含まれている場合、ネストレベルが 30 を超えるとエラーとなります。

ポイントデータに含まれる座標（位置データ）の扱い方を、「絶対座標」「相対座標」「移動量」の三種類から選択することができます。デフォルトは「絶対座標」が設定されています。

- 絶対座標：位置データの数値を、ロボットの固定座標を指すものとして扱います。
- 相対座標：位置データの数値を、プログラム開始座標からの距離を指すものとして扱います。
（開始座標が (0, 0) の場合は、絶対座標と同じ結果になります）
- 移動量：位置データの数値を、次点までの距離を指すものとして扱います。

位置データの扱い方により、数値は同じでも移動するポイントの位置は異なります。例を以下に記します。

ポイントデータ座標数値	(0, 0)	(15, 20)	(10, 20)	(5, 10)	
	P1	P2	P3	P4	P5
・ 絶対座標	●	●	●	●	●
	(10, 10)	(0, 0)	(15, 20)	(10, 20)	(5, 10) …ロボットの絶対座標
・ 相対座標	●	●	●	●	●
	(10, 10)	(10, 10)	(25, 30)	(20, 30)	(15, 20) …ロボットの絶対座標
・ 移動量	●	●	●	●	●
	(10, 10)	(10, 10)	(25, 30)	(35, 50)	(40, 60) …ロボットの絶対座標

プログラム開始座標

注) プログラムをサブプログラムとして運転する場合は、作業原点を無視します。また、[相対座標] [移動量] 指定のプログラムの場合も戻りません。

[絶対座標] 指定のプログラムを独立したものとして実行する（「callProg」での運転ではない）場合のみ、作業原点に戻ります。

■ [相対座標] 指定のプログラムのティーチング

JOG モードでポイントのティーチングをする場合には、位置データの設定に関わらず [絶対座標] でのティーチングとなります。

相対座標のプログラムを作成する場合は、ポイントをティーチングした後、1 ポイント目の座標を (0, 0, 0) にするように、全ポイント移動（オフセット）をしてください。

■ [移動量] 指定のプログラムのティーチング

ティーチングした座標を移動量に変換することはできません。ポイントのティーチングは位置数値入力で行ってください。

15.5 ポイント列呼び出し

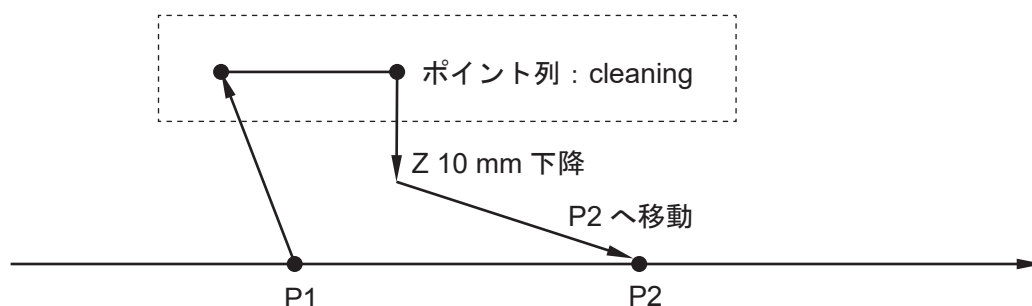
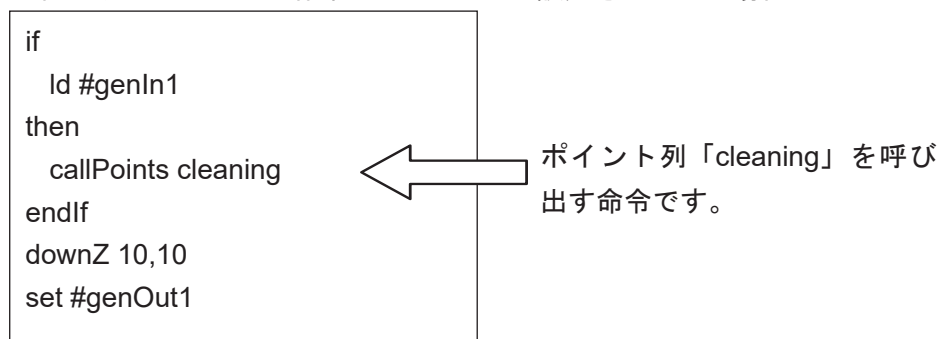
■ callPoints

識別子を持つポイント列（カスタマイズモードで定義）を呼び出して実行します。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	callPoints	ポイント列識別子	指定したポイント列をサブルーチンコールします

注) 「callPoints」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

例 以下のポイント作業データが P1 に設定されている場合



#genIn1 が ON ならポイント列「cleaning」へ呼び出し、ポイント列「cleaning」に設定されたポイント作業データ、ポイント属性データを実行します。実行後、Z（J3）軸を 10 mm 下降させ、#genOut1 へ ON 出力。その後、P2 へ移動します。

#genIn1 が OFF なら、そのまま Z（J3）軸を 10 mm 下降させ、#genOut1 へ ON 出力します。

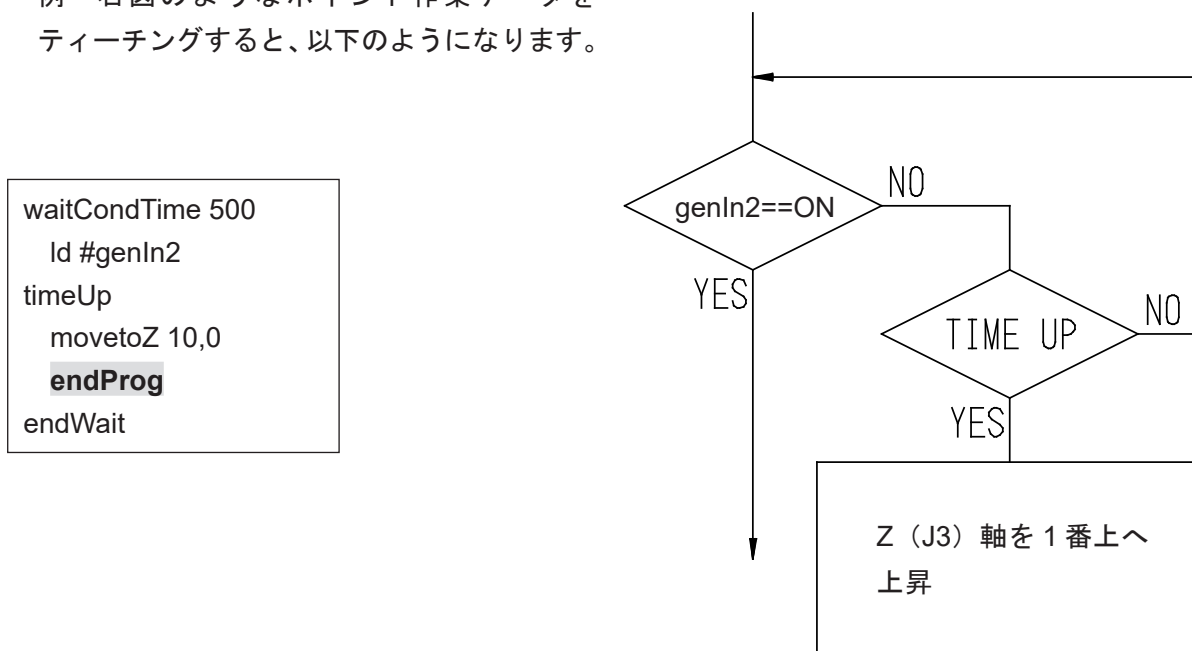
15.6 プログラムの強制終了

■ endProg

その場でプログラムを終了（運転終了）します。作業原点へは戻りません。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	endProg	-	その場でプログラムの運転を終了します

例 右図のようなポイント作業データを
ティーチングすると、以下のようになります。



「endProg」は、作業原点に戻らず、その場でプログラムを終了します。一旦停止ではないので、運転の続きを再開することはできません。新たにスタートすることになります。

注) 作業原点に戻ってからプログラムを終了させたい場合は、「goPoint」の命令を、飛び先番号 0 に設定してご使用ください。

15.7 関数の戻り値を代入

■ returnFunc

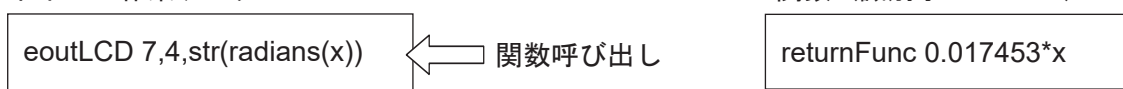
指定した式の値を戻り値へ代入し、関数を終了します。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	returnFunc	式	指定した式の値を戻り値として、関数を終了します

注) 「returnFunc」命令は、ポイント作業データでは使用できません。

ポイント作業データ

関数（識別子：radians）



* str () は、数値を 10 進数文字列に変換します。

ティーチングペンダント LCD に、引数 (x) の場合の関数「radians」の戻り値を表示。

15.8 指定ポイントへ飛ぶ

■ goPoint, goRPoint, goCRPoint

そのポイントのポイント作業を終了し、次ポイントでなく、指定したポイントへ飛ぶことができます。

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	goPoint	PTP 駆動条件番号、ポイント番号	指定したポイントへ移動します
	goRPoint	PTP 駆動条件番号、相対ポイント番号	相対指定したポイントへ移動します
	goCRPoint	PTP 駆動条件番号、移動先選択	CP 駆動の途中から、選択した移動先へ移動します

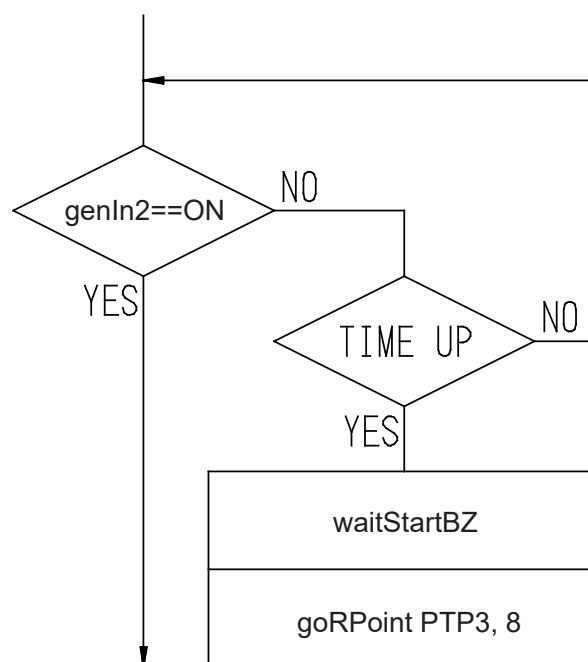
注)

- 「goPoint」「goRPoint」「goCRPoint」命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。
- 「goPoint」「goRPoint」命令のポイント番号、相対ポイント番号は、変数や式で指定することもできます。

「goCRPoint」命令の移動先選択も変数や式で指定することができますが、この場合には必ず値が「0」または「1」のどちらかになるようにしてください。

例 右図のようなポイント作業データを
ティーチングすると、以下ようになります。

```
waitCondTime 500
  Id #genIn2
timeUp
  waitStartBZ
  goRPoint PTP3, 8
endWait
```



このポイント作業の意味は、以下の通りです。

「#genIn2 が、0.5 秒待っても ON にならない場合は、ブザーを鳴らしてスタート待ちにする。その後、スタート信号を受信したら現在ポイント番号 +8 のポイントから運転を再開する。

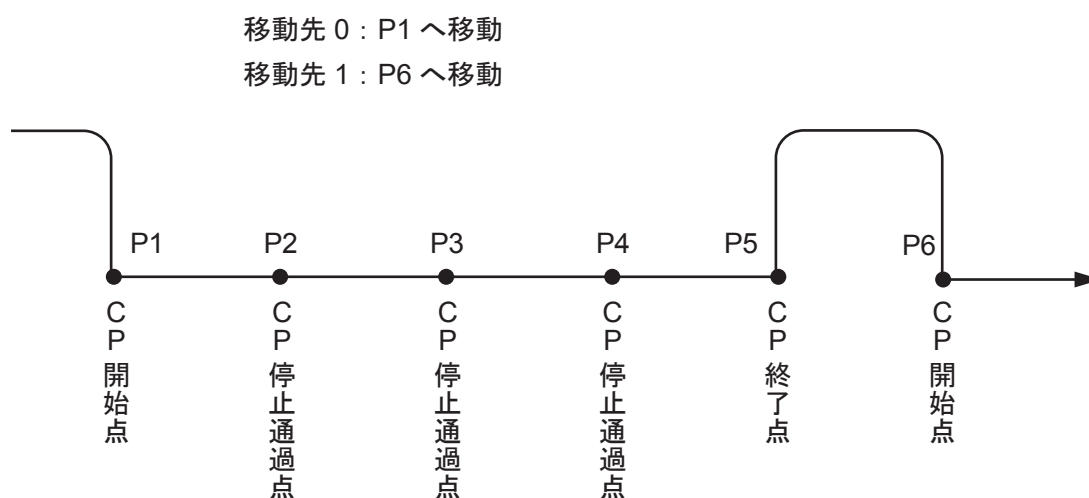
[goPoint PTP3, 25] : 番号 25 のポイントへ飛びます。(PTP 駆動条件 03 に従って移動)
PTP 駆動条件番号を 0 にすると、プログラムデータに設定されている
PTP 駆動条件の設定に従います。
ポイント 0 を指定すると、作業原点へ飛びます。
(指定した番号のポイントへ飛びます。)

[goRPoint PTP3, -4] : 現在のポイント番号に、-4 した番号のポイント (4 ポイント前) へ飛び
ます。(PTP 駆動条件 03 に従って移動)
PTP 駆動条件番号を 0 にすると、プログラムデータに設定されている
PTP 駆動条件の設定に従います。
相対ポイント 0 を指定すると、同ポイントからの再開になります。
(相対的に指定した番号のポイントへ飛びます。)

[goCRPoint PTP3, 1] : CP 駆動の途中から移動する場合にこの命令を使用します。

CP 開始点～CP 終了点を 1 つの動作とし、移動先 0 を指定した場合は現在の CP 駆動を開始したポイント（CP 開始点）へ戻ります（PTP 駆動条件 03 に従って移動）。1 を指定した場合は、CP 終了点の次のポイントへ飛びます。（PTP 駆動条件 03 に従って移動） PTP 駆動条件番号を 0 にすると、プログラムデータに設定されている PTP 駆動条件の設定に従います。

例 以下のようにポイントが並んでいるとして、P1 ～ P5 の間でこの命令が実行された場合は、次の結果になります。

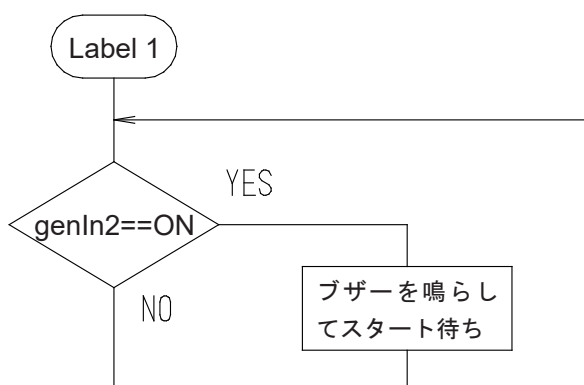


15.9 指定した命令へ飛ぶ

■ jump, Label

命令カテゴリ	命令	必要パラメータ	作業
実行流れ制御	jump	ラベル番号	指定した番号の「Label」へ飛びます
	Label	ラベル番号	「jump」での飛び先の目印になります

例 #genIn2 が ON の場合、ブザーを鳴らしてスタート指示があるまで待機。
#genIn2 が ON でなかったら、そのまま次の作業へ。



Label 1

```

if
  ld #genIn2
then
  waitStartBZ
  jump L1
endif
  
```

(飛び先目印)

以下の条件が真なら then へ、偽なら endif の次へ。

genIn2==ON

条件が真の場合、以下の命令を実行。

ブザーを鳴らし、スタート指示があるまでその場で待機。

[Label 1] へ (スタート指示があったら)。

条件分岐命令終了。

注)

- ・「Label」命令は、if ~ endif, waitCond ~ endWait の途中には設定できません。
- ・「Label 1」 ~ 「Label 99」まで設定できます。

16. 繰り返し

■ for, next, exitFor, do, loop, exitDo

命令カテゴリ	命令	必要パラメータ	作業
繰り返し	for	制御変数、初期値、終了値、増分	for から next までの命令を、指定した変数が初期値から終了値になるまで繰り返す
	next	-	
	exitFor	-	for 文から抜け出す
	do	-	
	loop	-	do から loop までの命令を繰り返す
	exitDo	-	do 文から抜け出す

■ for ～ exitFor ～ next

「for」は繰り返し回数を指定できる命令です。

```
for ival=1 to 8 step 1
  (繰り返し内容)
next
```

変数 ival の初期値を 1 として、1 周毎に 1 を足し、for ～ next を ival=8 になるまで繰り返します。

```
for ival=1 to 8 step 1
  (繰り返し内容)
  if
    ld #genIn1
  then
    exitFor
  endif
next
```

「exitFor」は for ～ next の繰り返しから抜け出し、「next」の次の命令を読みに行きます。

条件：#genIn1=ON の場合、ival=8 でなくても、for ～ next の繰り返しから抜け出し、「next」の次の命令を読みに行きます。

繰り返し途中で条件が ON になった場合、繰り返し内容を 1 サイクル実行後、次へ移行します。
(設定条件が繰り返し命令 for ～ next の間に設定されている場合のみ)

「for」命令のパラメータ（初期値、終了値、増分）は変数や式で指定することもできます。

```
declare numeric loop
  if
    ld #genIn1
  then
    let loop = 5
  else
    let loop = 10
  endif
  for ival=1 to loop step 1
    (繰り返し内容)
  next
```

ローカル変数 loop の宣言。

もし、

#genIn1 = ON

であれば

loop に 5 を代入。

でなければ

loop に 10 を代入。

変数 ival の初期値を 1 として、1 周毎に 1 を足し、for ~ next を ival に値が変数 loop (5 / 10) と同じになるまで繰り返します。

注) for~next の繰り返しを途中終了をしたい場合には、exitFor 命令を実行すると、ival の値が変数 loop (5 または 10) と同じにならなくても、繰り返しを終了することができます。
exitFor 命令は、**for~next** の「繰り返し内容」の中で、If 文などで条件を指定して使用してください。

■ do ~ exitDo ~ loop

「do」から「loop」までを、「exitDo」で抜け出すまで繰り返します。

```
do
  (繰り返し内容)
loop
```

繰り返し終了条件無しの場合、このままでは無限ループになります。

```
do
  (繰り返し内容)
  if
    ld #genIn1
  then
    exitDo
  endif
  (繰り返し内容)
loop
```

注) (繰り返し内容) は条件の前後どちらでも可。

条件 : #genIn1=ON の場合、do ~ loop の繰り返しから抜け出し、「loop」の次の命令を読みに行きます。

注)

- ・ 繰り返し命令はネストレベルが 10 を超えた場合、エラーとなります。
- ・ 繰り返し命令を CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントに、ポイント作業として設定した場合、繰り返し回数によっては停止する場合があります。

17. 駆動制御

17.1 Z (J3) 軸だけを上昇／下降

■ upZ, downZ, movetoZ

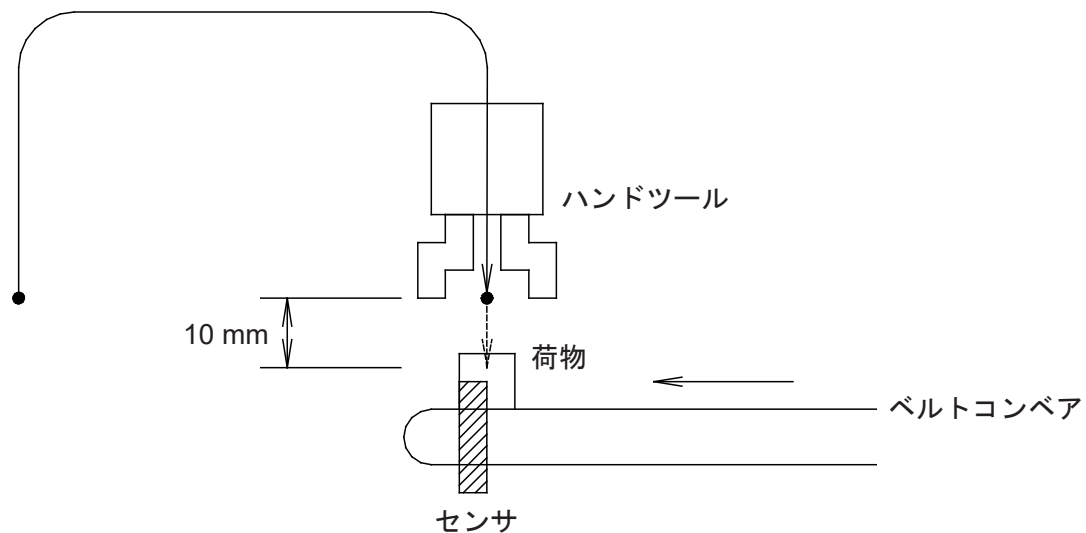
ポイント作業で、Z (J3) 軸だけを上昇／下降させることができます。命令の種類（カテゴリ）は、[駆動制御] に属します。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	upZ	移動速度、移動距離	指定距離を Z (J3) 軸のみ上昇します。
	downZ	移動速度、移動距離	指定距離を Z (J3) 軸のみ下降します。
	movetoZ	移動速度、Z 移動位置	指定 Z 座標（絶対座標）へ上昇、または下降します。

注)

- ・ 駆動制御命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。
- ・ ポイント種別に関係なく、[移動中作業]、[CP 駆動中作業] に設定しても無効です。

例 ハンドツールで物を掴む時に、「PTP 駆動で高速のまま掴む位置まで移動させず、手前で一旦止め、物の有無をセンサで確認後、低速で下降させて掴みたい」という場合。



```
waitCond
  Id #genIn2
endWait
downZ 20, 10
```

以下の条件が満たされるまで、その場で待つ。

#genIn2=ON (条件)

条件記載行終了

Z (J3) 軸のみ 速度 20 mm/sec で 10 mm 下降。

移動距離、移動速度を変数や式で指定することもできます。

```
waitCond
  Id #genIn2
endWait
downZ 20, #P_Z(1) -#point_Z
```

以下の条件が満たされるまで、その場で待つ。

#genIn2=ON (条件)

条件記載行終了

Z (J3) 軸のみ速度 20 mm/sec で、P1 の Z 座標から現在ポイントの Z 座標を減算した値分下降、または上昇。

#P_Z(1) : 現在プログラム P1 の Z 座標を値に持つ変数

#point_Z : 現在ポイントの Z 座標を値に持つ変数

注) 移動前作業で変数「#jobStartHight」に値を代入 (let) すると、設定ポイントの Z 座標より、代入した値分高い位置から移動後作業を開始します。

つまり、事前に移動後作業の開始するタイミングを早めることができます。

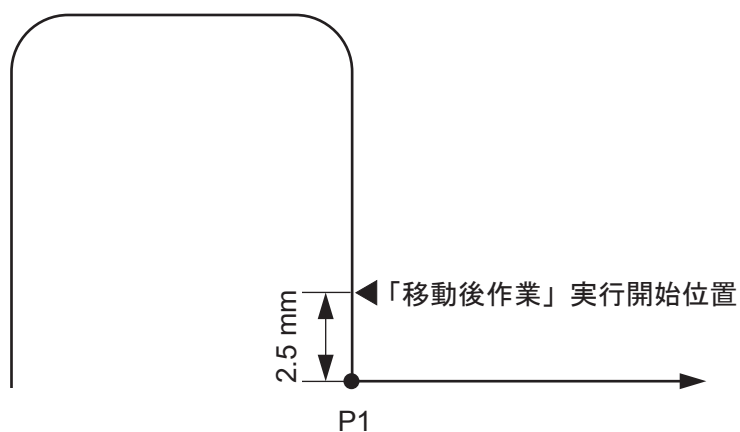
例

P1
種別 : CP 開始点
移動前作業 : ポイント作業データ 3
移動後作業 : ポイント作業データ 12

ポイント作業データ 3

```
let #jobStartHight = 2.5
```

注) ポイント作業データ 12 は任意で設定してください。



17.2 ポイント作業で CP 直線駆動

■ lineMove, lineMoveStopIf

ポイント作業データの命令で、直線移動（CP 駆動）ができます。

CP 速度と各座標軸の移動量を指定できます。また条件を設定し、途中で移動を終了させることもできます。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	lineMove	移動速度（CP 速度） X 移動距離 Y 移動距離 Z 移動距離 R 回転角度	入力した距離を CP 駆動で移動します。

注）駆動制御命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

各移動距離は、座標ではなく、現在のポイント位置から移動したい距離を入力してください。移動させたくない方向には「0」を入力してください。

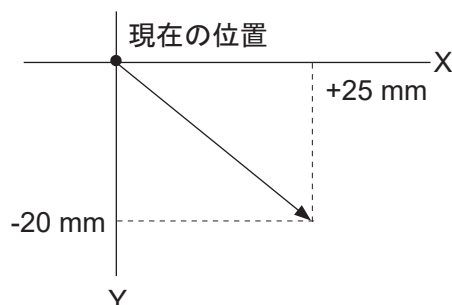
また、移動距離を数値でなく変数や式で入力することもできます。

この命令を入力すると、「lineMoveSpeed」から「endLineMove」まで、右図のように命令が展開します。

```
lineMoveSpeed 20
lineMoveX 25
lineMoveY -20
lineMoveZ 5
lineMoveR 0
endLineMove
```

注）設定途中で ESC を押してしまうと押した項目の前の設定で終了してしまうため、途中項目の追加はできません（初めの項目から設定し直しが必要です）。

命令は座標軸ごとに表示されますが、移動は一括で実行します。



■ 「lineMove」で、条件により途中で移動を終了させたい場合

```
lineMoveSpeed 3
  lineMoveX 20
  lineMoveY 0
  lineMoveZ 0
  lineMoveR 0
lineMoveStopIf
  Id #sysIn1
endLineMove
callJob11
```

移動終了の条件

X方向の移動が+20に達する前でも、#sysIn1がONになったら、移動を終了し、次の命令（callJob11）へ進みます。

この場合に、指定した距離を移動して終了したかは、システムフラグ（#sysFlag34）で参照できます。

指定した距離を移動：0、条件により途中で移動終了：1

■ 「lineMove」で、移動結果がソフトリミットを超えてしまう場合

移動結果がソフトリミットを超えてしまう場合は、ソフトリミットに達した位置で移動を終了し、次の命令に進みます。

この場合に、指定した距離を移動して終了したか、システムフラグ（#sysFlag（33））で参照できます。

正常：0、距離が指定より短い：1

指定した移動距離を移動し終わる前にソフトリミットに達した場合は、ブザーを鳴らして停止（スタート待ち）。

```
lineMoveSpeed 3
  lineMoveX 20
  lineMoveY 0
  lineMoveZ 0
  lineMoveR 0
endLineMove
if
  Id #sysFlag（33）
then
  waitStartBZ
endif
```

17.3 ポイント作業でメカ初期化実行

■ initMec

メカ初期化（ロボットの電源を ON した時に実行する初期化）をポイント作業で実行できます。メカ初期化を実行すると、外力で位置ズレが起きた場合でも絶対座標（x:0, y:0, z:0, r:0）へ戻すことができます。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	initMec	軸指定	指定した軸をメカ初期化します。

注)

- ・ 駆動制御命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。
- ・ 移動中、CP 駆動中作業は無効です。

軸指定	内容	
	JR3000 シリーズ、JC-3 標準仕様	JC-3 アブソリュート仕様
all	全軸をメカ初期化します。	補助軸（MT1, MT2）をメカ初期化します。
x	X 軸をメカ初期化します。	X 軸の作業原点移動は実行せず、即時終了します。
y	Y 軸をメカ初期化します。	Y 軸の作業原点移動は実行せず、即時終了します。
z	Z 軸をメカ初期化します。	Z 軸の作業原点移動は実行せず、即時終了します。
r	R 軸をメカ初期化します。	R 軸の作業原点移動は実行せず、即時終了します。
MT1	MT1 をメカ初期化します。	
MT2	MT2 をメカ初期化します。	

JS3 シリーズ	
軸指定	内容
all	補助軸（MT1, MT2）をメカ初期化します。
J1	J1 軸の作業原点移動は実行せず、即時終了します。
J2	J2 軸の作業原点移動は実行せず、即時終了します。
J3	J3 軸の作業原点移動は実行せず、即時終了します。
J4	J4 軸の作業原点移動は実行せず、即時終了します。
MT1	MT1 をメカ初期化します。
MT2	MT2 をメカ初期化します。

注)

- ・ メカ初期化は、低速移動により実行されます。
- ・ 円周開始点に設定しないでください。
- ・ 補助軸のメカ初期化は、軸コンフィグレーション設定に従います。軸コンフィグレーションでメカ初期化を[無し]に設定した場合は実行せず、コマンドは即時終了（正常終了扱い）します。

17.4 位置ずれ検出

■ checkPos

以下の命令は JR3000/JC-3 シリーズのみ有効です。JS3 シリーズには以下の命令はありません。

位置ずれ検出をポイント作業で実行できます。

位置ずれ検出命令を実行すると、現在の座標に関わらず絶対座標 (x:0, y:0, z:0, r:0) へ移動します。位置ずれ検出後、次ポイントへ向かいます。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	checkPos	-	位置ずれ検出します。

注) 駆動制御命令は、CP 連続通過点、および CP 連続通過点を基底種別とするポイント種別のポイントでは無効です。

位置ずれ検出の結果は、システムフラグ (#sysFlag (35)) で参照できます。

正常 : 0、位置ずれ : 1

例

```
checkPos
if
  ld #sysFlag (35)
then
  waitStartBZ
endif
```

位置ずれ検出を実行し、位置ずれを起こしていたと判断した場合、ブザーを鳴らしてスタート待ち。

また、位置ずれが検出された場合は、#sysOut8 (位置ずれエラー) 信号が ON します。

注) JC-3 アブソリュート仕様では位置ずれ検出を実行せず、即時終了します。

17.5 指定した 1 軸だけを駆動

■ monoMove, endMonoMove, monoMoveStopIf

指定された 1 軸だけの駆動を行います。ロボットの X, Y, Z, R 軸および補助軸 MT1, MT2 を指定できます。位置座標値は mMovePosition、距離は mMoveDistance で指定してください。速度、加速度は mMoveSpeed, mMoveAccelRate, mMoveAccelTime で指定できます。省略することも可能です。省略した場合はデフォルトの速度、加速度で駆動します。詳細は mMoveSpeed, mMoveAccelRate, mMoveAccelTime の説明をご参照ください。

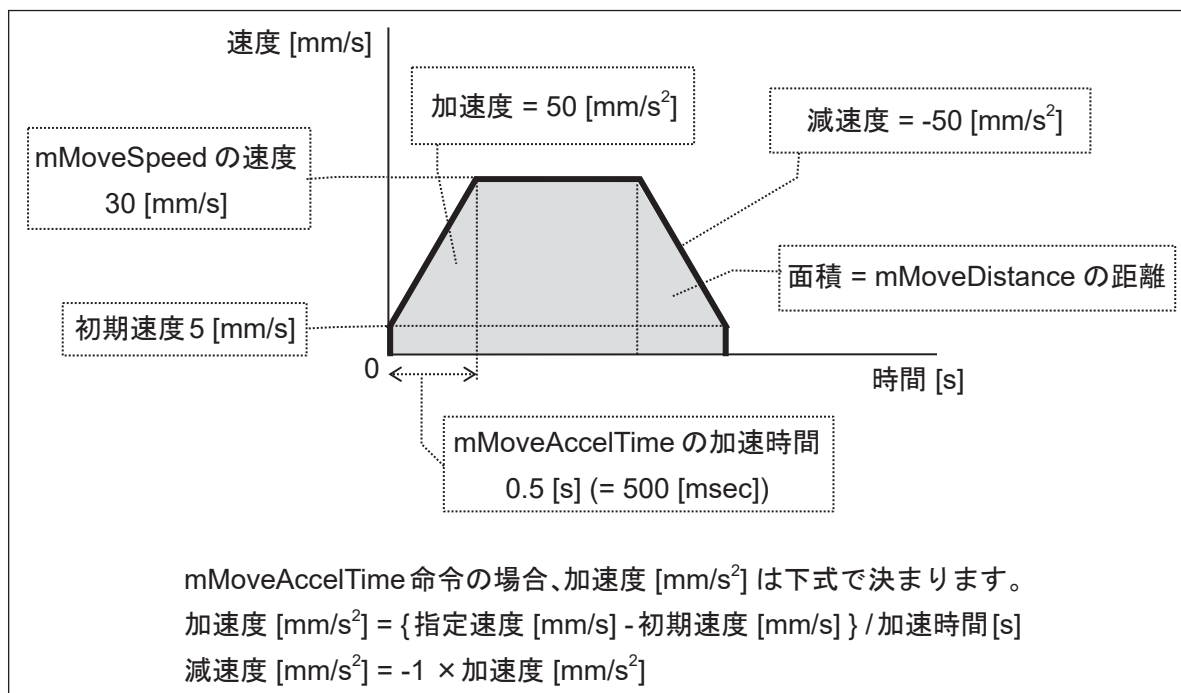
命令カテゴリ	命令	必要パラメータ	作業
駆動制御	monoMove	軸指定	指定された 1 軸だけの駆動を行います。
	endMonoMove	-	monoMove 命令の終了を示します。
	monoMoveStopIf	-	monoMove 命令による駆動を条件により停止させます。条件による停止をしない場合は、この命令を入力する必要はありません。

例

```
monoMove Z
  mMoveDistance 50
  mMoveSpeed 30
  mMoveAccelTime 500
monoMoveStopIf
  Id #genIn1
endMonoMove
```

Z 軸の単独駆動
距離 50 [mm]
速度 30 [mm/s]
加速時間 500 [msec]
(加速度 = 30 [mm/s] / 0.5 [s] = 120 [mm/s²])
停止条件
#genIn1 が ON になったら停止する

この例の場合の速度変化は次ページ図のようになります。



指定された距離で停止することを優先する調整のため、実際の速度、加加速度、加速時間および減速度、減速時間には誤差がある場合があります。分解能が大きいほど誤差が大きくなる傾向があります。

ロボットの X, Y, Z, R 軸の場合、[初期速度] は機種ごとに固有の値です。補助軸 MT1, MT2 の場合は補助軸コンフィギュレーションの [初期速度] に設定された値です。

monoMove 命令の駆動に対し、以下のシステムフラグで駆動結果の情報を得られます。

No.	識別子	内容	
33	#FIMoveOutOfRange	真 (1)	可動範囲またはソフトリミットを超えるため、範囲に達した位置まで駆動した。
		偽 (0)	可動範囲およびソフトリミットの範囲内で駆動が完了した。
34	#FIMoveStop	真 (1)	monoMoveStopIf の条件を満たし、駆動を停止した。
		偽 (0)	monoMoveStopIf の条件を満たさなかった。

■ mMovePosition

monoMove 命令で駆動する位置座標値を指定します。

mMovePosition と mMoveDistance の両方を定義した場合は mMoveDistance が有効になり、mMovePosition は無効になります。

パラメータの単位系は軸ごとに異なります。X, Y, Z 軸は [mm]、R 軸は [deg] です。補助軸 MT1, MT2 は補助軸コンフィグレーションの「単位表示」に設定された単位です。

パラメータの入力範囲は +9999.999 ~ 0 です。最小入力単位は 0.001 です。式入力の場合は入力範囲外の値や最小入力単位未満の値を指定可能です。

各軸の可動範囲またはソフトリミットを超える場合は、範囲に達する位置まで駆動します。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	mMovePosition	位置座標値	monoMove 命令で駆動する位置座標値を指定します。

■ mMoveDistance

monoMove 命令で駆動する距離を指定します。

速度や加速度と異なり、距離は必ず指定する必要があります。mMoveDistance を省略した場合は式評価エラーとなり、駆動は行われません。

パラメータの単位系は軸ごとに異なります。X, Y, Z 軸は [mm]、R 軸は [deg] です。補助軸 MT1, MT2 は補助軸コンフィグレーションの「単位表示」に設定された単位です。

パラメータの入力範囲は +9999.999 ~ -9999.999 です。最小入力単位は 0.001 です。式入力の場合は入力範囲外の値や最小入力単位未満の値を指定可能です。

正の値では正方向に、負の値では負方向に駆動します。

各軸の可動範囲またはソフトリミットを超える場合は、範囲に達する位置まで駆動します。

距離が 0 または各軸の最小分解能未満の場合は、駆動は行われません。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	mMoveDistance	距離	monoMove 命令で駆動する距離を指定します。

■ mMoveSpeed

monoMove 命令で駆動するときの速度を指定します。

mMoveSpeed は省略可能です。省略した場合はデフォルトの速度となります。

ロボットの X, Y, Z, R 軸の場合、デフォルトの速度は機種ごとに固有の最大速度です。補助軸 MT1, MT2 の場合は補助軸コンフィグレーションの「制限速度」に設定された速度です。パラメータの単位系は軸ごとに異なります。X, Y, Z 軸は [mm/s]、R 軸は [deg/s] です。補助軸 MT1, MT2 は補助軸コンフィグレーションに設定された「単位表示」に基づく速度の単位です。

パラメータの入力範囲は 0 ～ 9999.9 です。最小入力単位は 0.1 です。式入力の場合は入力範囲外の値や最小入力単位未満の値を指定可能です。

デフォルトを超える速度が指定された場合は、デフォルトの速度で駆動します。

速度が 0 またはパルス速度 [PPS] に換算して 0 になる場合は駆動しません。

また、負の値では駆動しません。負方向に駆動させたい場合は、mMoveDistance により負方向の距離を指定してください。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	mMoveSpeed	速度	monoMove 命令で駆動するときの速度を指定します。

■ mMoveAccelRate, mMoveAccelTime

monoMove 命令で駆動するときの加速度を指定します。

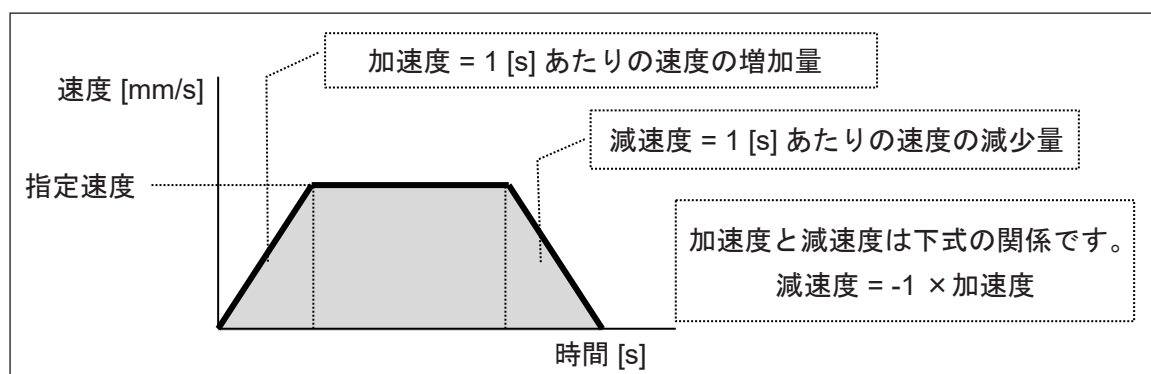
加速度を指定する手段には、加速度率による mMoveAccelRate 命令と、加速時間による mMoveAccelTime 命令があります。

どちらか一方の命令で指定してください。両方の命令を入力した場合は mMoveAccelRate が優先されます。両方を省略した場合はデフォルトの加速度となります。

ロボットの X,Y,Z,R 軸の場合、デフォルトの加速度は機種ごとに固有の最大加速度です。補助軸 MT1, MT2 の場合は補助軸コンフィギュレーションの「最大加速度」に設定された加速度です。

命令カテゴリ	命令	必要パラメータ	作業
駆動制御	mMoveAccelRate	加速度率 [%]	monoMove 命令で駆動するときの加速度を指定します。
	mMoveAccelTime	加速時間 [msec]	

減速時には、下図のように加速度と同じ変化量で速度が減少します。減速度を個別に指定することはできません。



mMoveAccelRate 命令（加速度率 [%]）は、デフォルトの加速度に対する比率を [%] 単位（百分率）で指定してください。

パラメータの入力範囲は 1 ～ 100 [%] です。最小入力単位は 0.1 [%] です。式入力の場合は入力範囲外の値や最小入力単位未満の値を指定可能です。

100 [%]（デフォルトの加速度）を超える加速度が指定された場合は、100 [%] で駆動します。加速度が 0 または負の値の場合は、駆動は行われません。

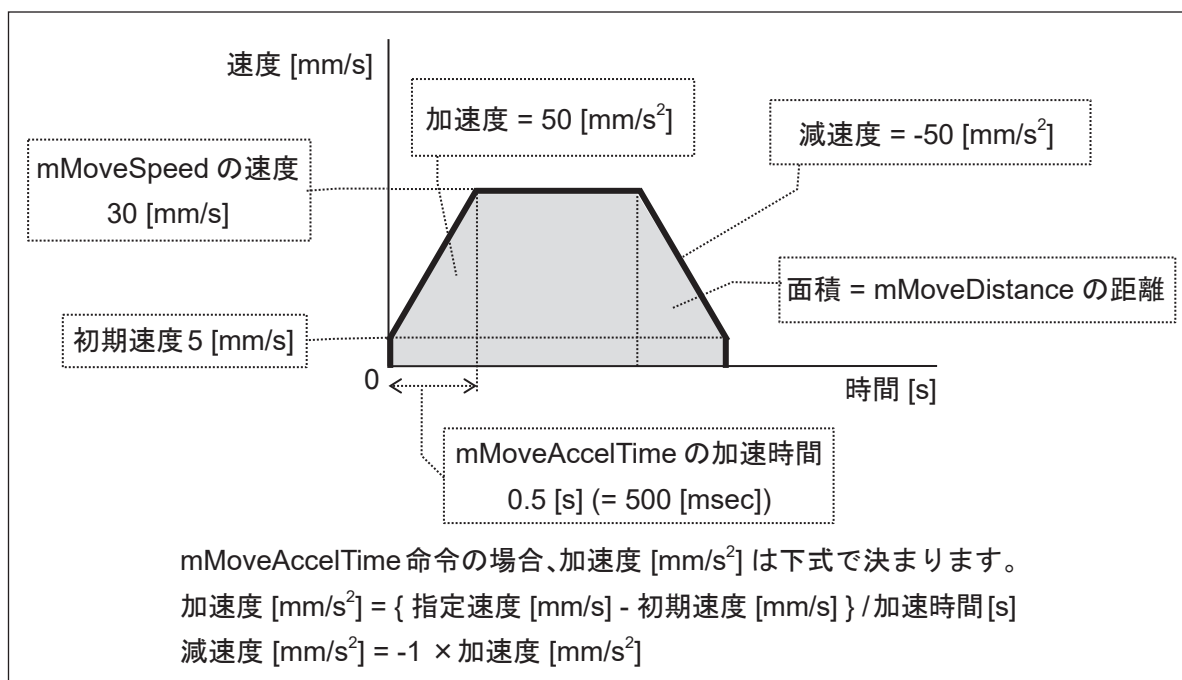
mMoveAccelTime 命令（加速時間 [msec]）は、mMoveSpeed で指定された速度に到達するまでの時間を [msec] 単位で指定してください。

パラメータの入力範囲は 0 ～ 999999999 [msec] です。最小入力単位は 1 [msec] です。式入力の場合は入力範囲外の値や最小入力単位未満の値を指定可能です。

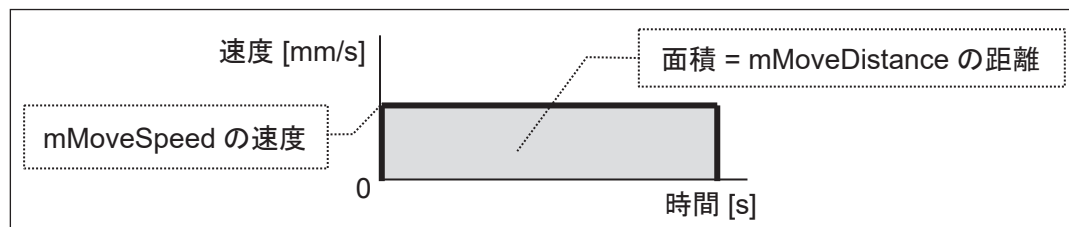
負の値が指定された場合は 0 として扱います。

mMoveAccelTime 命令では、指定された加速時間と速度から加速度（= 1 [s] あたりの速度の増加量）が決まります。下図に例を示します。

(Z 軸, mMoveSpeed = 30 [mm/s], mMoveAccelTime = 500 [msec])



加速時間 0 [msec] は特別な意味を持ちます。加速時間が 0 [msec] の場合は、下図のように加減速の区間の無い駆動となります。



注) デフォルトの加速度（100 % の加速度）を超える場合の注意・制限事項

ロボットの X,Y,Z,R 軸の場合、加速度は最大でデフォルトの 500 [%] に制限されます。

また、JR3000/JC-3 シリーズの場合、デフォルトの加速度(100 [%] の加速度)を超える場合、

または、加速時間が 0 [msec] の場合は、速度がメカ初期化時と同じ速度に制限されます。

補助軸 MT1, MT2 の場合、加速度は最大でデフォルトの 500 [%] に制限されます。

加速時間が 0 [msec] の場合は加速時間がなく即座に指定速度となります。

⚠ 注意



補助軸 I/O-MT の駆動の場合は、接続対象（外部モータ等の装置）の仕様・性能を超えるような急激な加速とならないようご注意ください。

mMoveAccelTime 命令による高加速や加速区間の無い駆動は、ステッピングモータを自起動運転させるために用いることを推奨いたします。

18. LCD、7SLED

18.1 ティーチングペンダントに指定文字列を表示

■ clrLCD, clrLineLCD, outLCD, eoutLCD

ティーチングペンダントの LCD 画面に入力した文字を表示させたり、表示を消したりすることができます。

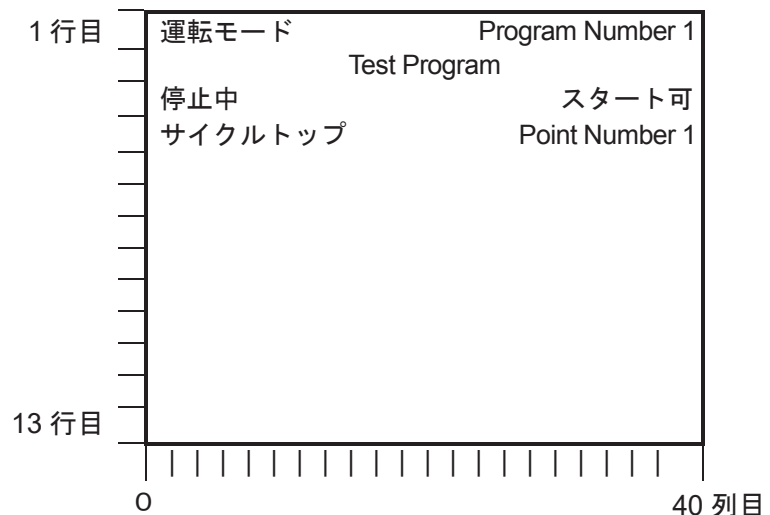
命令カテゴリ	命令	必要パラメータ	作業
LCD 表示	clrLCD	無し	LCD 画面全体をクリア
	clrLineLCD	行	LCD 画面の指定した行をクリア
	outLCD	行、列、文字列	LCD 画面の指定した位置に入力した文字列を表示
	eoutLCD	行、列、文字列式	LCD 画面の指定した位置に入力した文字列式の結果を表示

注) 行や列数を式で指定することもできます。また、命令「eoutLCD」は表示内容を文字列式で与えることもできます。

outLCD 7, 4, "PULSE" : ティーチングペンダントの LCD に文字列「PULSE」を表示。

eoutLCD 7, 4, #sv (24) & #sv (25) : ティーチングペンダントの LCD に文字列型変数 #sv (24) と #sv (25) の値を結合させた文字列を表示。

ティーチングペンダントの LCD 画面は、13 行×40 列です。
半角文字の場合は 1 行に最大 40 文字、全角文字の場合は最大 20 文字を表示させることができます。



18.2 7セグメント LED に任意の数値を表示

■ sys7SLED, out7SLED

「out7SLED」命令で、本体前面（JR3000 シリーズ）またはスイッチボックス／操作ボックスの 7SLED に任意の数値を表示させることができます。

「sys7SLED」命令を実行するかプログラムを切り替えると、プログラム番号の表示に戻ります。

命令カテゴリ	命令	必要パラメータ	作業
LCD, 7SLED	sys7SLED	-	「out7SLED」で変更した表示を戻す
	out7SLED	表示種類、表示数値	7SLED へ出力

「out7SLED」のパラメータの表示種類は、以下の 4 種類から選択できます。表示数値は、変数や式で指定することもできます。

表示種類	内容
Num	指定した数値（0 ～ 999）を 7SLED に表示 例：1 桁の場合 '001' → 表示は「1」 2 桁の場合 '011' → 表示は「11」
Er	「Er」と指定した数値を 7SLED に交互に表示
St	「St」と指定した数値を 7SLED に交互に表示
Ur	「Ur」と指定した数値を 7SLED に交互に表示

例 out7SLED Num, 10 : 数値「10」を表示
out7SLED Er, 20 : 「Er」と数値「20」を交互に表示
out7SLED St,nMyNum : 「St」と変数「nMyNum」の値を表示

19. COM/Ethernet 入出力

- outCOM, eoutCOM, setWTCOM, inCOM, cmpCOM, ecmpCOM, clrCOM, shiftCOM
COM からデータを出力、または入力することができます。また、イーサネットクライアントポートでの入出力（送受信）も行うことができます。詳細は「[21.1 イーサネットクライアント機能](#)」をご参照ください。

命令カテゴリ	命令	必要パラメータ	作業
COM/Ethernet 入出力	outCOM	ポート、文字列	COM およびイーサネットから文字を出力
	eoutCOM	ポート、文字列式	COM およびイーサネットから文字列式の結果を出力
	setWTCOM	ポート、待ち時間	COM およびイーサネット受信待ち時間（タイムアウト時間）を設定
	inCOM	変数名、ポート、文字数	COM およびイーサネット受信データを指定変数へ代入
	cmpCOM	ポート、文字列	COM およびイーサネット受信データと文字列とを比較 結果はシステムフラグ sysFlag (1) ~ (15) に入る
	ecmpCOM	ポート、文字列式	COM およびイーサネット受信データと文字列式とを比較 結果はシステムフラグ sysFlag (1) ~ (15) に入る
	clrCOM	ポート	COM およびイーサネット受信バッファをクリア
	shiftCOM	ポート、シフト数	COM およびイーサネット受信データをシフト 先頭からシフト数バイト分のデータを捨てる

- COM 出力 : outCOM, eoutCOM
COM およびイーサネットから文字列を出力することができます。文字列は 255 文字まで出力できます。
[outCOM] / [eoutCOM] を選択し、ポート（COM 番号）を選択すると、文字入力画面が表示されます。出力したい文字列を入力し、ENTR キーを押してください（文字入力画面でのキー操作は、取扱説明書『ティーチングペンダント操作編』「3.4 文字、式入力」をご参照ください）。
出力する文字列は「`”`」で囲います（例 1）。eoutCOM では、変数や式を出力することもできます。この場合には「`”`」で囲わないでください（例 2）。
例 1 : eoutCOM port2, “ERROR” : ERROR と出力
例 2 : eoutCOM port2, #sv(24) & #sv(25) : 文字列型変数 #sv(24) と #sv(25) の値を結合させた値を出力。

また、[eoutCOM] 命令では「%」を使用することで、文字を 16 進コードで指定することもできます（例 3）。ただし、「%」の後に 0～9、A～F、%以外の文字がある場合は、「%」をそのままの文字として扱います（例 4）。「%」の後に 0～9、A～F の文字があって「%」をそのまま出力したい場合は、「%%」と入力してください（例 5）。

例 3 : eoutCOM port2,“%0D%0A” : CR・LF コード（改行コード）の出力

例 4 : eoutCOM port2,“%G01” : %G01 と出力

例 5 : eoutCOM port2,“%%300” : %300 と出力

■ COM 受信待ち時間設定 : setWTCOM

「inCOM」「cmpCOM」で、受信待ちをする時の受信待ち時間を設定します。ここで設定した時間だけ待っても受信しない場合は、タイムアウト（システムフラグ ON）となります。1 msec 単位で設定できます。

setWTCOM のデフォルト値は 100 msec です。別の値を設定した場合、ロボットの電源を切るまで維持されます。ロボットの電源を再投入すると、100 msec のデフォルト値に戻ります。

■ COM 入力 : inCOM

COM およびイーサネットから受信したデータを指定した文字数だけ変数へ代入します。受信データが指定の文字数を超過している場合は、先頭から数えて指定文字数までを代入します。受信データが指定文字数に満たない場合は、「setWTCOM」で指定した待ち時間だけ待機し、その時点で受信しているデータを変数へ代入します。「setWTCOM」を使用しない場合は、100 msec を待ち時間とします。

注) COM 入力命令を含むポイント作業データが CP 連続通過点に設定されている場合、受信待ち時間は 0 sec として動作します。

■ COM およびイーサネット受信データ比較 : cmpCOM, ecmpCOM

COM およびイーサネット受信バッファ（受信バッファは受信データを格納しておく場所です）と指定文字列を比較します。比較した結果は、システムフラグに反映されます。

先頭の文字から 1 文字ずつ順次比較して行き、等しくない場合、指定文字数ぶん比較が終了した場合、また、設定した受信待ち時間だけ待っても受信がない場合にそれぞれ対応するシステムフラグを ON します（下表参照）。

受信待ち時間は「setWTCOM」で設定します。「setWTCOM」を使用しない場合は 100 msec を待ち時間とします。

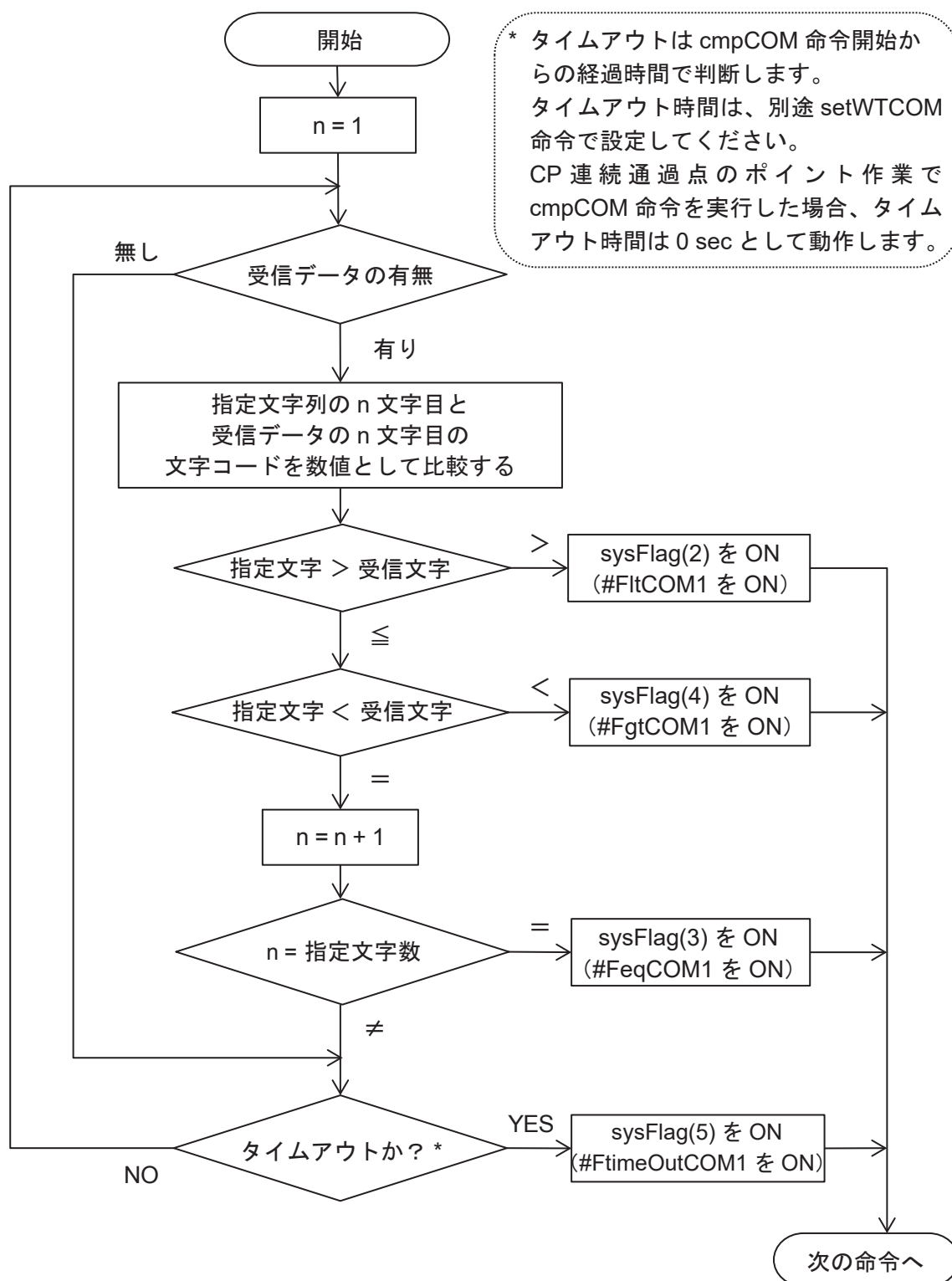
「ecmpCOM」命令では受信バッファと比較する文字列を文字列式で指定することができます。

対応するシステムフラグ

	COM1	COM2	COM3 (JR3000/JC-3 シリーズのみ)
受信データあり	sysFlag(1)	sysFlag(6)	sysFlag(11)
指定文字 > 受信バッファ	sysFlag(2)	sysFlag(7)	sysFlag(12)
指定文字 = 受信バッファ	sysFlag(3)	sysFlag(8)	sysFlag(13)
指定文字 < 受信バッファ	sysFlag(4)	sysFlag(9)	sysFlag(14)
タイムアウト	sysFlag(5)	sysFlag(10)	sysFlag(15)

	クライアント ポート 1	クライアント ポート 2	クライアント ポート 3
受信データあり	sysFlag (300)	sysFlag (306)	sysFlag (312)
指定文字 > 受信バッファ	sysFlag (301)	sysFlag (307)	sysFlag (313)
指定文字 = 受信バッファ	sysFlag (302)	sysFlag (308)	sysFlag (314)
指定文字 < 受信バッファ	sysFlag (303)	sysFlag (309)	sysFlag (315)
タイムアウト	sysFlag (304)	sysFlag (310)	sysFlag (316)

COM1 の場合の cmpCOM 命令の例



■ COM およびイーサネット受信バッファをクリア : clrCOM

受信バッファは受信したデータを格納しておく場所です。各ポートに1つずつ受信バッファがあります（各 8 Kbyte）。受信したデータは上書きされず、受信したデータの後ろへ書き込まれていきます。

受信バッファは電源を落とすか、「clrCOM」命令を実行するとクリアされます。

■ COM およびイーサネット受信データのシフト : shiftCOM

指定したバイト数ぶん、受信バッファのデータをシフトします。

例 2 バイトシフト

受信バッファ



システムフラグにより、各受信バッファにデータが格納されているか確認できます。

受信データがある場合は、対応するシステムフラグが ON します。

■ COM1 コマンド通信機能が有効になっている場合の注意

COM1 コマンド通信機能が有効になっている場合、「inCOM」「outCOM」などの命令を使用して COM1 で任意の通信処理を行うことはできません。ただし、以下を行うと、COM1 でコマンド通信機能が有効でも、一時的に「inCOM」「outCOM」などの命令を使用することができます。

- (1) 「inCOM」「outCOM」などの命令前に事前に「stopPC」でコマンド通信機能を停止してください。停止中はコマンド通信機能の影響を受けず「inCOM」「outCOM」などの命令で任意の通信を行うことができます。
- (2) 「inCOM」「outCOM」などの命令後に「startPC」でコマンド通信機能を再開してください。

20. 変数、コメント、システム制御

20.1 変数宣言、代入

■ declare, let

宣言命令を含むポイント作業データやユーザ定義関数の中でのみ有効な変数をローカル変数と呼びます。

ローカル変数は宣言時に型と識別子を設定します。識別子は変数の名前にあたり、型は「数値型」と「文字列型」どちらかの選択となります。

また、ローカル変数は3次元までの配列として宣言することもできます。

「let」命令は右辺の数値、変数の値、または式の計算結果を左辺の変数へ代入する命令です。

命令カテゴリ	命令	必要パラメータ	作業
変数、コメント、システム制御	declare	型, 識別子	ローカル変数宣言
	let	代入式	右辺の式の計算結果を左辺の変数へ代入

例 declare 命令

```
declare numeric abc  
declare string def
```

数値型変数「abc」の宣言

文字列型変数「def」の宣言

例 let 命令

```
let count = 0  
let count = count + 1  
let count = in - out  
  
let total = nin * 365  
  
let tsuki = total / 12  
  
let fullname=name1 & name2
```

変数「count」に0を代入

変数「count」に1を足し込み

変数「count」にinの値からoutの値を引いた数値を代入

変数「total」に、ninの値に365を掛けた数値を代入

変数「tsuki」に、totalの値を12で割った数値を代入

変数「fullname」にname1とname2をつなげた文字列を代入

次の2つのポイント作業データの場合、どちらも「count」というローカル変数を使用していますが、ローカル変数は宣言命令を含むポイント作業データ内でのみ有効な関数なので、干渉することはありません。例えばポイント作業データ24で「count」に0を代入しますが、ポイント作業データ05で使用している「count」の値は変化しません。逆も同様です。

例 ポイント作業データ 05

```
declare numeric count
let count=0
do
  let count=count+1
  callJob 24
  if
    ld count>=10
  then
    exitDo
  endif
loop
```

「count」のローカル変数宣言（数値型）
「count」に初期値0を設定
loop 命令までを繰り返す
「count」に1を足し込み
ポイント作業データ24を実行
もし、
「count」の値が10以上の場合、

loop 命令の次の命令へ飛ぶ

do 命令へ戻る

例 ポイント作業データ 24

```
declare numeric count
let count=0
Label 1
pulse #genOut1,250
let count=count+1
if
  ld count<=3
then
  jump L1
endif
```

「count」のローカル変数宣言（数値型）
「count」に初期値0を設定
ラベル1（飛び先目印）
#genOut1にONパルス出力
「count」に1を足し込み
もし、
「count」の値が3以下の場合、

ラベル1へ飛ぶ

注）ローカル変数と反対に、どのプログラム、どのポイントからでも参照できる変数をグローバル変数と呼びます。「declare」で宣言したローカル変数以外はすべてグローバル変数です。

20.2 コメント挿入

■ rem, crem

ポイント作業データや PLC プログラムの命令に、コメントを付けることができます。

命令カテゴリ	命令	必要パラメータ	作業
変数、コメント、システム制御	rem	文字列	1 行コメント
	crem		行末コメント(逆コンパイル時の表示のみ)

注) T/P での表示は rem、crem 共に同じ表示

例 1 : rem 使用例 (T/P 表示)

```
if
  ld #genIn1
  rem #genIn1 は障害物センサ
then
  waitStartBZ
```

もし、
#genIn1 が真の場合、
(#genIn1 は障害物センサ) : コメント行

スタート指示があるまでブザーを鳴らして
待機

crem を使用した場合は、入力時には例 1 と同様に別の行に表示されますが、「JR C-Points II」で逆コンパイルした場合は、例 2 のように表示されます。

例 2 : crem 使用例 (逆コンパイル時)

```
if
  ld #genIn1 // #genIn1 は障害物センサ
then
  waitStartBZ
```

もし、
#genIn1 が真の場合、(#genIn1 は障害物
センサ) : コメント行
スタート指示があるまでブザーを鳴らして
待機

20.3 ポイント作業でプログラム番号変更

■ setProgNum

ポイント作業で、現在選択されているプログラムの番号を変更できます。次のような場合に利用できます。

- 電源投入時のポイント作業で番号をセットすることにより、運転モードで電源を入れた時にはいつも同じプログラム番号にすることができます。

- 作業原点の終了時ポイント作業で番号をセットすることにより、次に実行するプログラム番号に切り替えることができます。一連のプログラムを順に実行したい場合に便利です。例えば、「プログラム 1→ プログラム 2→ プログラム 3」が一連の作業工程で、これを繰り返す場合があります。プログラム 1 の作業原点の終了時ポイント作業で「setProgNum2」を実行すると、プログラム 1 を運転した後はプログラム 2 に切り替わります。同様にプログラム 2 からプログラム 3 へ、プログラム 3 からプログラム 1 に切り替わるように設定できます。
- 運転待機時のポイント作業で、COM からの入力に従い、番号をセットすることができます。COM にバーコードリーダーをつなぎ、読み取った値に従ってプログラムを切り替えることもできます。

プログラム運転中に番号を変更しても、即座に運転しているプログラムが切り替わることはありません。番号変更後、運転待機状態からスタート（運転開始）で、切り替わります。
プログラム運転中に別のプログラムを実行したい場合は「callProg」命令をご使用ください。

命令カテゴリ	命令	必要パラメータ	作業
変数、コメント、システム制御	setProgNum	プログラム番号	次に運転待機状態からスタートした時に、プログラム番号を切り替えます

注) プログラム番号を、変数や式で指定することもできます。

20.4 ポイント作業で PLC プログラム変更

■ setSeqNum

ポイント作業で、現在選択されている PLC プログラムの番号を変更できます。

PLC プログラムの最大命令数は 1000 ステップなので、あまり複雑なものは作れません。そこで、電源投入時、運転待機時、運転中に実行する PLC プログラムをそれぞれ別にして作成し、「setSeqNum」で切り替えて使うことができます。

例えばサイクル開始時共通作業（運転モード作業）で「setSeqNum2」とし、サイクル終了時作業（運転モード作業）では「setSeqNum1」とすれば運転中は 2 番、待機中は 1 番の PLC プログラムが実行されます。

プログラム運転中に番号を変更しても、即座に PLC プログラムが切り替わることはありません。番号変更後、運転待機状態からスタート（運転開始）で、切り替わります。

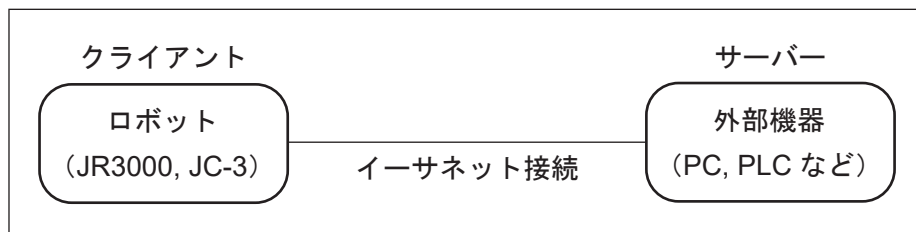
命令カテゴリ	命令	必要パラメータ	作業
変数、コメント、システム制御	setSeqNum	PLC プログラム番号	次に運転待機状態からスタートした時に、PLC プログラム番号を切り替えます

注) PLC プログラム番号を、変数や式で指定することもできます。

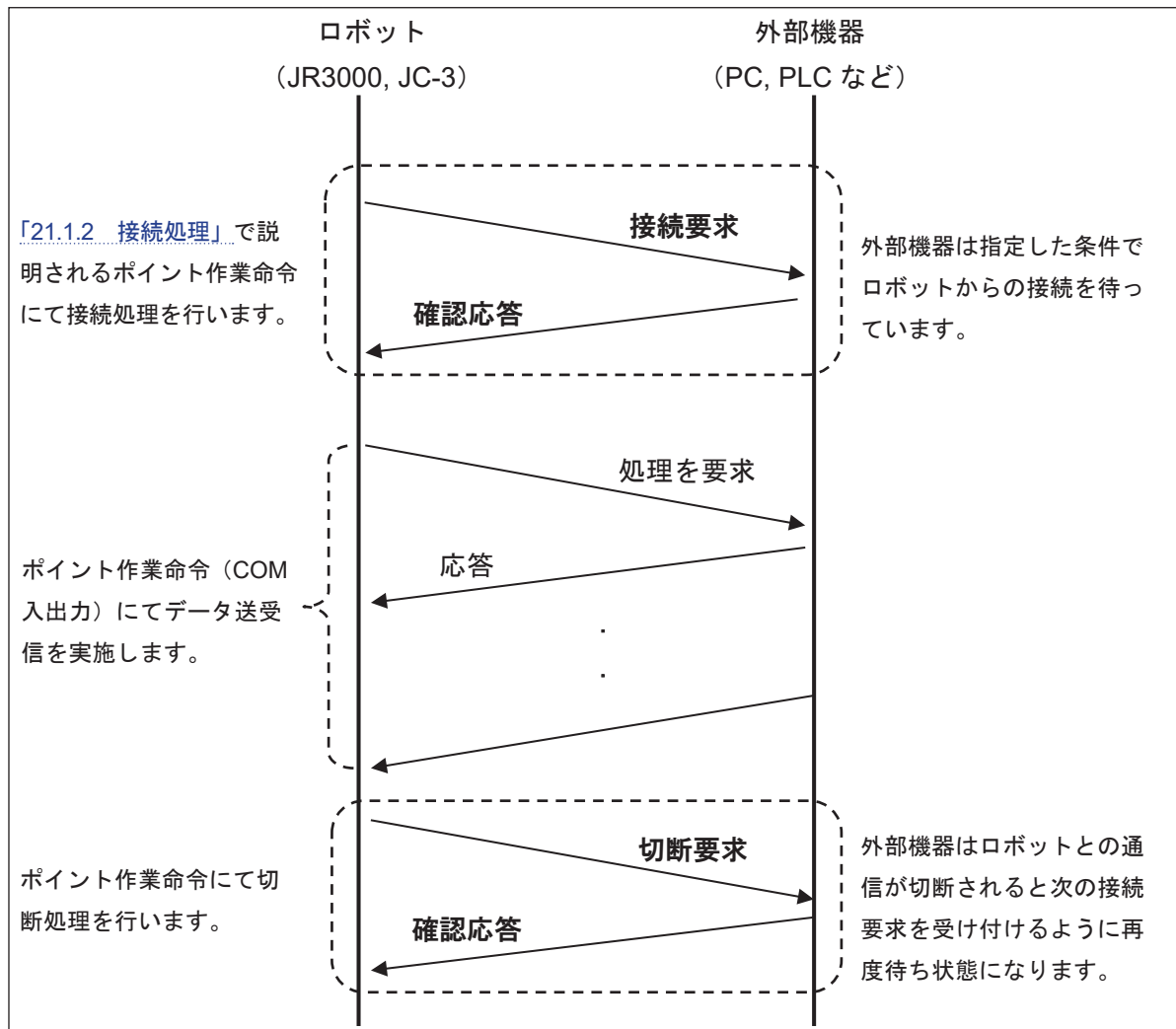
21. Appendix

21.1 イーサネットクライアント機能

本機能は、クライアントサーバーシステムにおけるクライアント側の機能です。ロボットがクライアントとなり、サーバーとなる外部機器との通信を行うことができます。データの送受信は、ポイント作業命令の inCOM や outCOM といった COM 通信と同じ命令体系で行います。



ロボットと外部機器の通信の流れは下図のようになります。



はじめに、ロボット（JR3000 または JC-3）が外部機器に対して接続要求を送ります。外部機器は、この接続要求を受け取ることができるように接続待ちの状態となっている必要があります。なお、接続開始の処理を行わないままロボットがデータの送信処理をしても、データは外部機器には受信されません。

外部機器は、最大 3 台まで接続できます。設定方法については、[「21.1.1 イーサネットクライアントポートの設定」](#)をご参照ください。

21.1.1 イーサネットクライアントポートの設定

イーサネットクライアント機能では、接続先（サーバー）の IP アドレスとポート番号を設定する必要があります。設定は、ティーチングペンダントもしくは PC ソフト「JR C-Points II」を使って行います。

ティーチングペンダントの場合は以下の通りです。

TP **MODE** [管理モード]（JR3000/JC-3 シリーズ）
UTILITY [モード切り替え] [管理モード]（JS3 シリーズ）
[管理設定モード]
[Ethernet 通信設定]
[クライアントポート 1]
[クライアントポート 2]
[クライアントポート 3]

設定完了して管理モードを終了すると、ロボットは自動的に再起動します。再起動後、ネットワークアドレスが設定されます。

PC ソフトの場合は以下の通りです。

PC [ロボット] → [管理設定] → [管理設定] → [COM/Ethernet 設定]

[設定] ボタンを押すと、ロボットに設定を送信します。新しい設定は、ロボットの電源を再投入すると反映されます。

ティーチングペンダント、PC ソフト共に、IP アドレスをすべてゼロもしくは、ポート番号をゼロに設定すると、そのクライアントポートは無効となります。

デフォルト値は、無効扱いとして、IP アドレス（0.0.0.0）、ポート番号（0）が設定されています。また、IP アドレスとして通常利用できないアドレス（127.0.0.0 など）を指定した場合には、エラーとして設定できません。

21.1.2 接続処理

ロボットと外部機器が接続を行うには connect を、切断を行うには disconnect 命令を行います。

命令	必要パラメータ	作業
connect	ポート、数値	通信相手に接続する
disconnect	ポート、数値	通信相手から切断する

■ 接続時：connect

第 1 引数で指定したクライアントポートの接続相手に接続を行います。接続相手およびネットワークの状況によって、すぐに接続完了とならない可能性がありますので第 2 引数でタイムアウト値（単位は、ミリ秒）を指定することができます。

ネットワーク規模が大きい場合には、相対的に接続が完了するまで時間を要しますのでタイムアウト値を大きめに設定しておくことを推奨します。（100 ミリ秒程度を推奨）

■ 切断時：disconnect

第 1 引数で指定したクライアントポートの接続相手から切断します。接続相手およびネットワークの状況によって、すぐに切断完了とならない可能性がありますので第 2 引数でタイムアウト値（単位はミリ秒）を指定することができます。

ネットワーク規模が大きい場合には、相対的に切断が完了するまで時間を要しますのでタイムアウト値を大きめに設定しておくことを推奨します。（100 ミリ秒程度を推奨）

接続または切断されたかを確認するには、システムフラグを使います。接続済みの場合にはシステムフラグが ON となります

ポート	状態確認用システムフラグ
クライアントポート 1	sysFlag (305)
クライアントポート 2	sysFlag (311)
クライアントポート 3	sysFlag (317)

「19. COM/Ethernet 入出力」記載の COM/Ethernet 入出力命令にてクライアントポートを利用する場合には、ポート指定において、[ether1] , [ether2] もしくは、[ether3] を選択してください。それぞれ、クライアントポート 1～3 に対応します。

また、cmpCOM, ecmpCOM 命令の比較結果を保存するシステムフラグは、クライアントポート毎に下表のようになります。

	クライアント ポート 1	クライアント ポート 2	クライアント ポート 3
受信データあり	sysFlag (300)	sysFlag (306)	sysFlag (312)
指定文字 > 受信バッファ	sysFlag (301)	sysFlag (307)	sysFlag (313)
指定文字 = 受信バッファ	sysFlag (302)	sysFlag (308)	sysFlag (314)
指定文字 < 受信バッファ	sysFlag (303)	sysFlag (309)	sysFlag (315)
タイムアウト	sysFlag (304)	sysFlag (310)	sysFlag (316)

■ ポイント作業命令による接続例

クライアントポート 1 に設定されている接続先へ接続を行い、接続完了をなすまで繰り返す場合の例を示します。なお、下記の例では 1 秒間隔で 1 度接続を試し、完了するまで無制限に接続試行を行います。

```
do
  connect ether1, 100    クライアントポート 1 へ接続
  if
    ld #sysFlag (305)    クライアントポート 1 の接続状況を確認
  then
    exitDo               接続済みである場合には do ~ loop から抜ける
  endif
  delay 900              未接続の場合には 900 msec 待つ
loop
```

■ ポイント作業命令による切断例

接続済みの状態から切断するとき場合の例を示します。

```
if
  ld #sysFlag (305)      クライアントポート 1 が接続済みの場合
then
  disconnect ether1, 100  クライアントポート 1 を切断
endif
```

注) 頻繁に接続、切断を実施する場合には、上記命令をユーザー関数として作成しておくと、作業効率が向上します (取扱説明書『機能編 IV (カスタマイズ)』参照)。

21.1.3 実用例

ネットワーク上に準備されたログサーバーに対して、ロボット本体の移動情報を通知する場合を例に挙げます。

ログデータ送信は、プログラム運転完了時（サイクル終了時）に行われるものとします。

■ ログサーバー

対象とするログサーバーは、以下の条件でネットワークからのアクセスが行えるものとします。

プロトコル	TCP/IP, 無手順通信
ログサーバー IP アドレス	192.168.200.20
ログサーバーポート番号	1234

■ クライアントポート設定（ロボット側の設定）

本例では、ロボットの［クライアントポート 1］を使用するものとします。

上記条件のログサーバーと接続するため、［クライアントポート 1］を下表のように設定します。

接続先 IP アドレス	192.168.200.20
接続先ポート番号	1234

TP

MODE [管理モード]（JR3000/JC-3 シリーズ）

UTILITY [モード切り替え] [管理モード]（JS3 シリーズ）

[管理設定モード]

[Ethernet 通信設定]

[クライアントポート 1]

PC

[ロボット] → [管理設定] → [管理設定] → [COM/Ethernet 設定]

以下のようなポイント作業を作成し、[サイクル終了時作業]（取扱説明書『ティーチングペンダント操作編』参照）として設定します。

```
declare string logstr
declare num pno
let pno = currentMainProgNumber()
let logstr = "Program Finish!! [ProgNo="
let logstr = logstr & str(pno) & "]"

connect ether1, 100
if
  ld #sysFlag(305)
then
  eoutCOM ether1, logstr
  eoutCOM ether1, "%0D%0A"
  delay 200
  disconnect ether1, 100
endif
```

現在のプログラム番号を取得
ログ文字列作成
プログラム番号を連結

ログサーバに接続を試行

接続できている場合

ログ文字列を送信
改行文字（CR+LF）を送信
送信完了まで少し待つ
ログサーバから切断

ロボットのプログラム運転終了時、外部機器には下記のデータが受信されます。

```
Program Finish [ProgNo=10]
Program Finish [ProgNo=10]
Program Finish [ProgNo=10]
Program Finish [ProgNo=10]
```

■ 補足

- 本機が対応するイーサネット通信は、Internet Protocol Version.4（インターネットプロトコルバージョン4：IPv4）となります。Internet Protocol Version.6（インターネットプロトコルバージョン6：IPv6）のみの環境ではご使用いただけません。
同様にロボット（JR3000/JC-3/JS3 シリーズ）の通信先の外部機器（PC や PLC）についても IPv4 に対応している必要があります。

- 本機能で指定するポート番号は、通信先の外部機器（PC, PLC など）の受け待ちポート番号を指定します。

一般に、ポート番号には下表に示すように 3 種類のカテゴリが存在します。

ポート番号（範囲）	内容
1 ～ 1023	ウェルノウンプート
1024 ～ 49151	登録済みポート
49152 ～ 65535	動的・私用ポート

通常ウェルノウンプートおよび登録済みポートはサーバー（本機能では PC, PLC などの外部機器）側が、動的・私的ポートはクライアント側が利用することを想定しています。ウェルノウンプート範囲の番号で受け待ちを行っているサーバー機器はネットワークに対して重要な機能を提供している可能性が高いため、本機能で通信する PC, PLC などの外部機器は、登録済みポート番号の範囲にて受け待ちを行うことを推奨します。

- 本機能では通信において特定プロトコル（TELNET や HTTP 等）の規定を行わない「無手順通信」のみに対応しています。プロトコルが必要となる場合には、ポイント作業命令にて個別に対応する必要があります。

株式会社ジャノメ
産業機器営業本部

〒 193-0941 東京都八王子市狭間町 1463 番地
TEL 042-661-2123 / FAX 042-665-3354

テクニカルサポート TEL: 042-661-2247
E-mail: janomeie03@gm.janome.co.jp

製品改良のため、断り無く仕様を変更することがありますので、ご了承ください。
本書の内容の一部または全部を無断で複写・転載することを禁じます。

© 2014-2026 株式会社ジャノメ

170813004 / 202607-J05